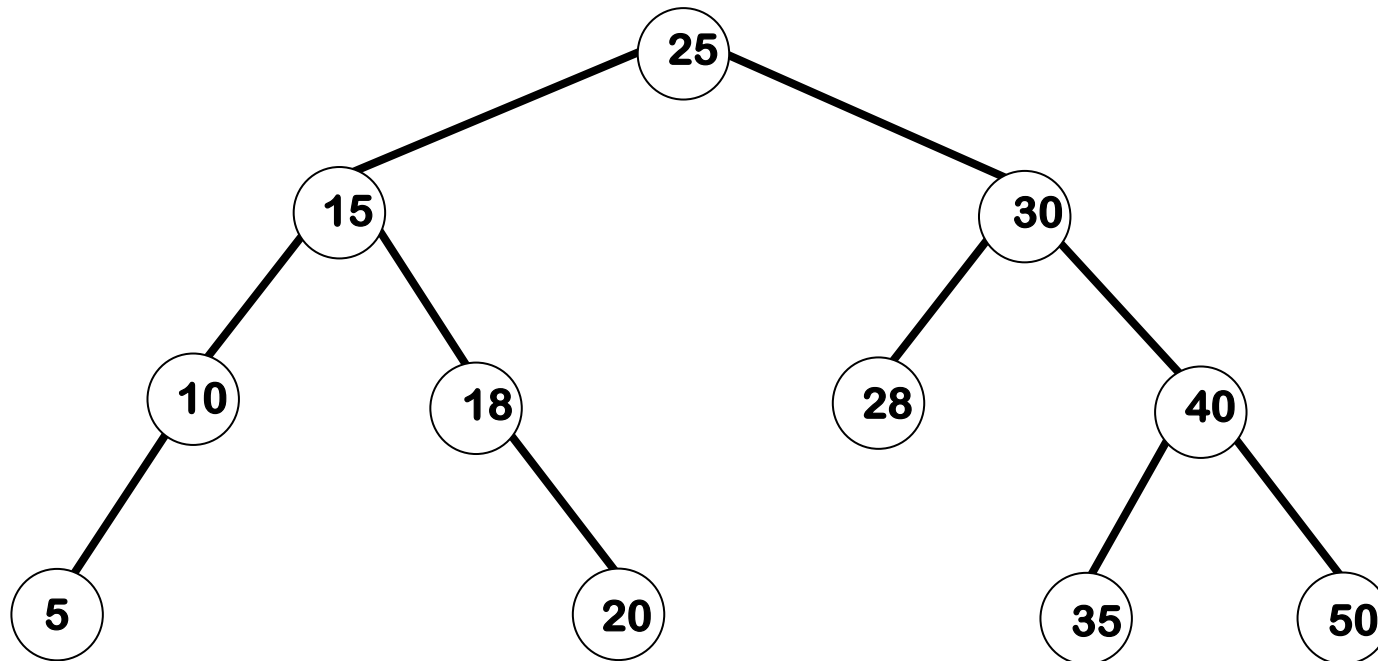




BALANCED BINARY SEARCH TREES

MARZIEH ESKANDARI
NJIT

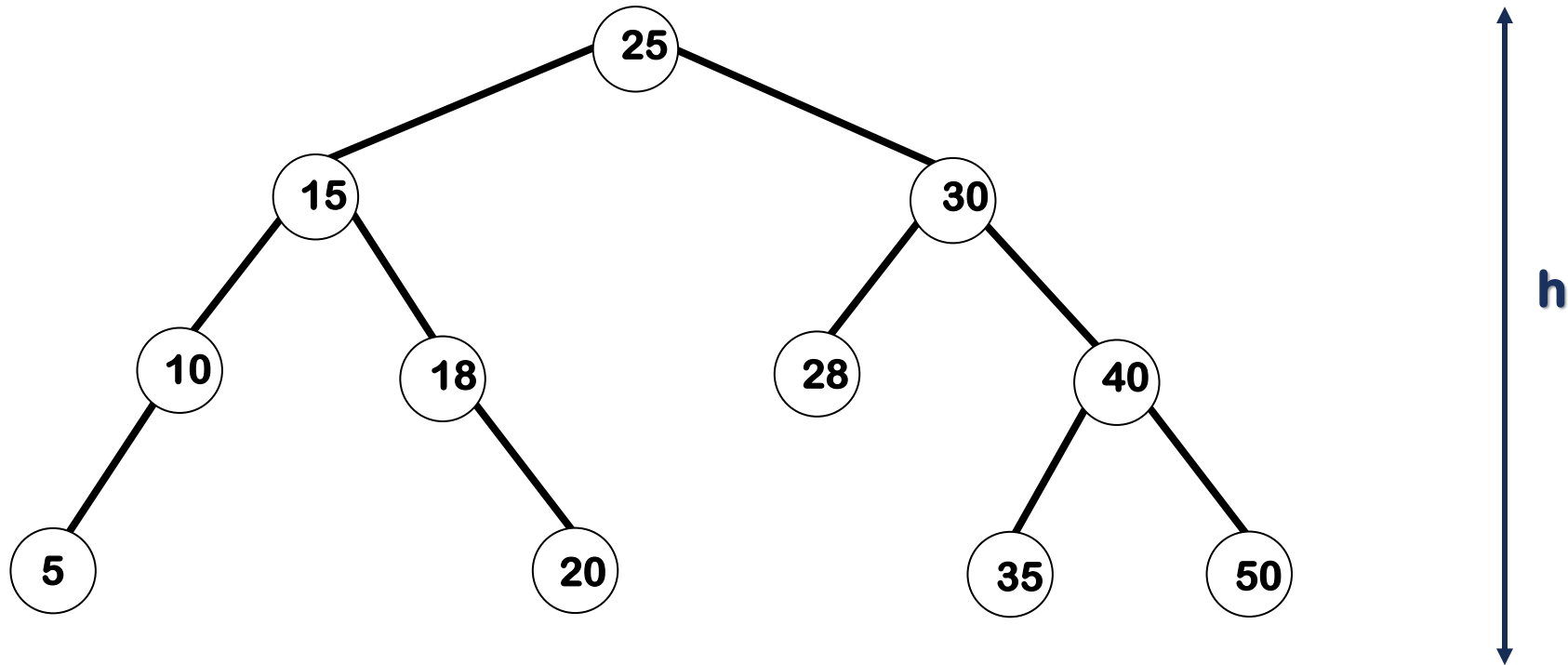
BINARY SEARCH TREE



Binary Search Tree Property: key of a node is less than the keys of its right subtree and greater than the keys of its left subtree.

METHODS

- Search(k): $O(h)$
- Insert(k) : $O(h)$
- Delete(k) : $O(h)$



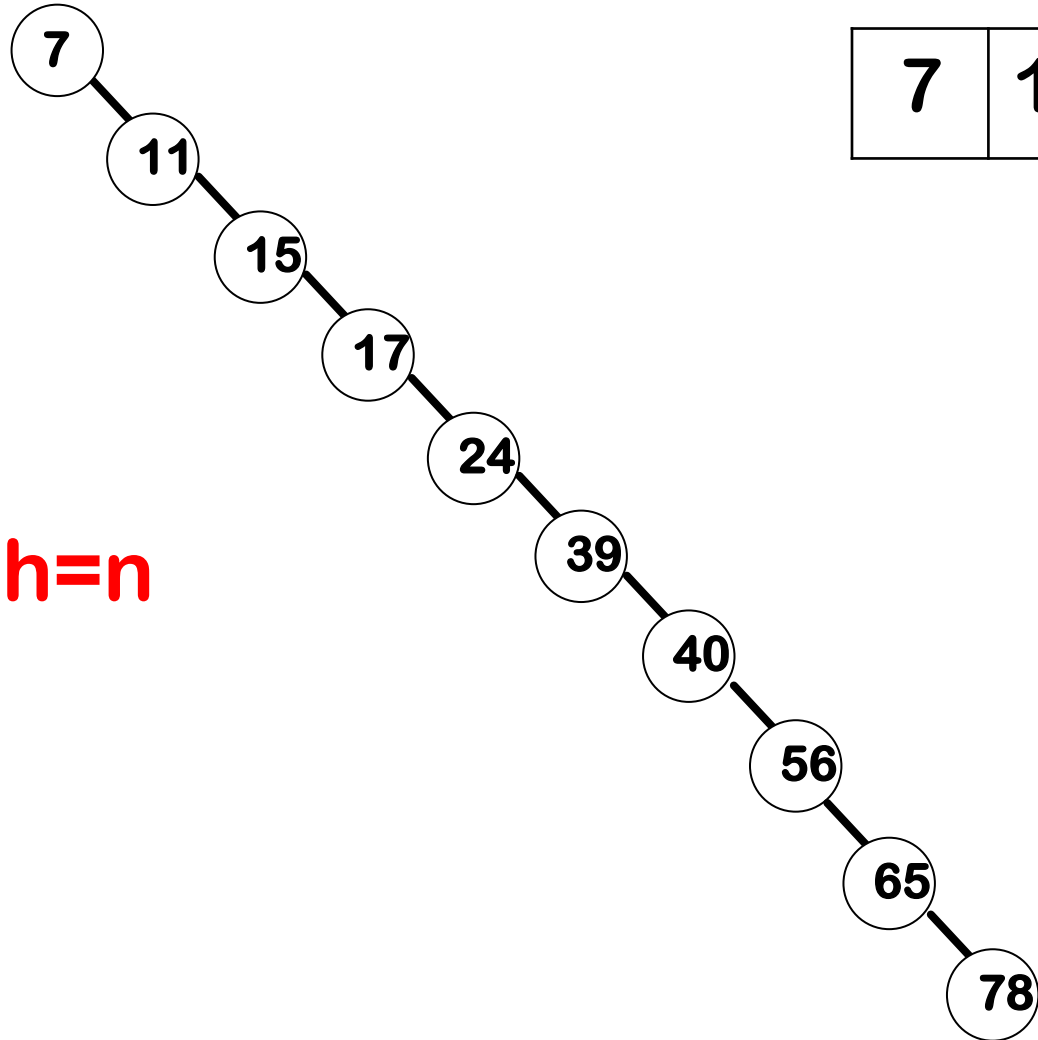
BAD TREES FOR SEARCH

7

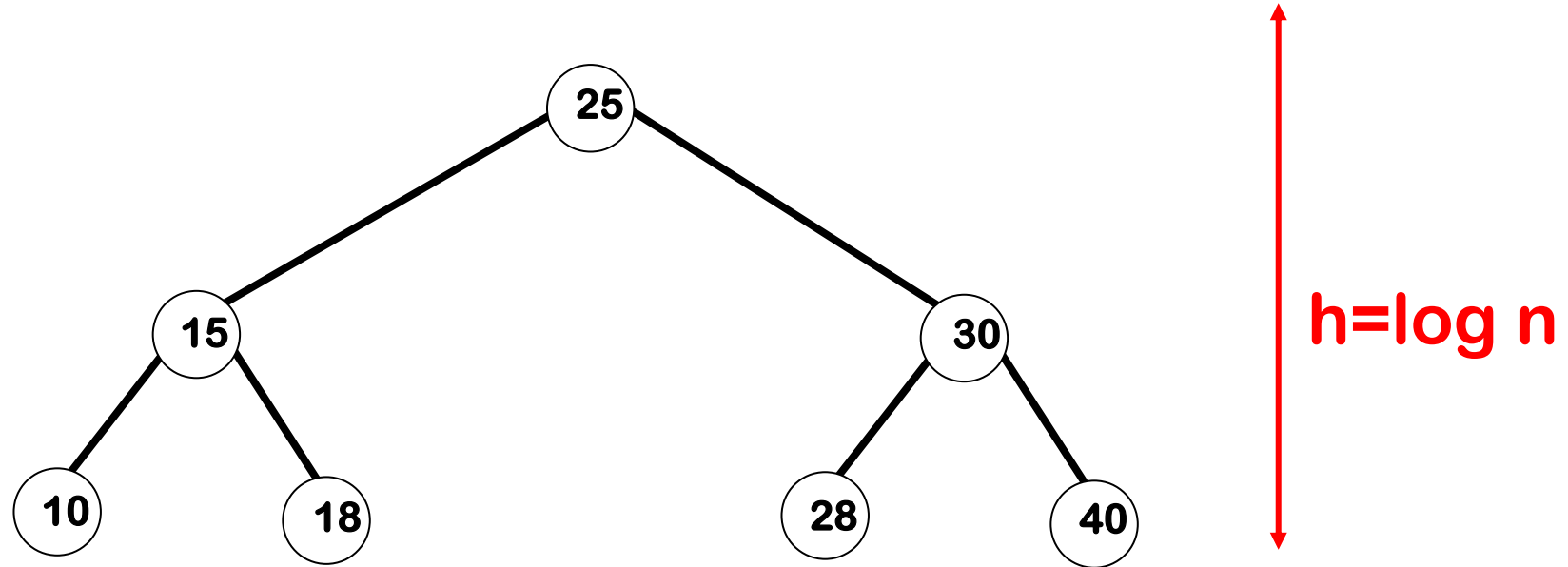
7	11	15	17	24	39	40	56	65	78
---	----	----	----	----	----	----	----	----	----

BAD TREES FOR SEARCH

7	11	15	17	24	39	40	56	65	78
---	----	----	----	----	----	----	----	----	----

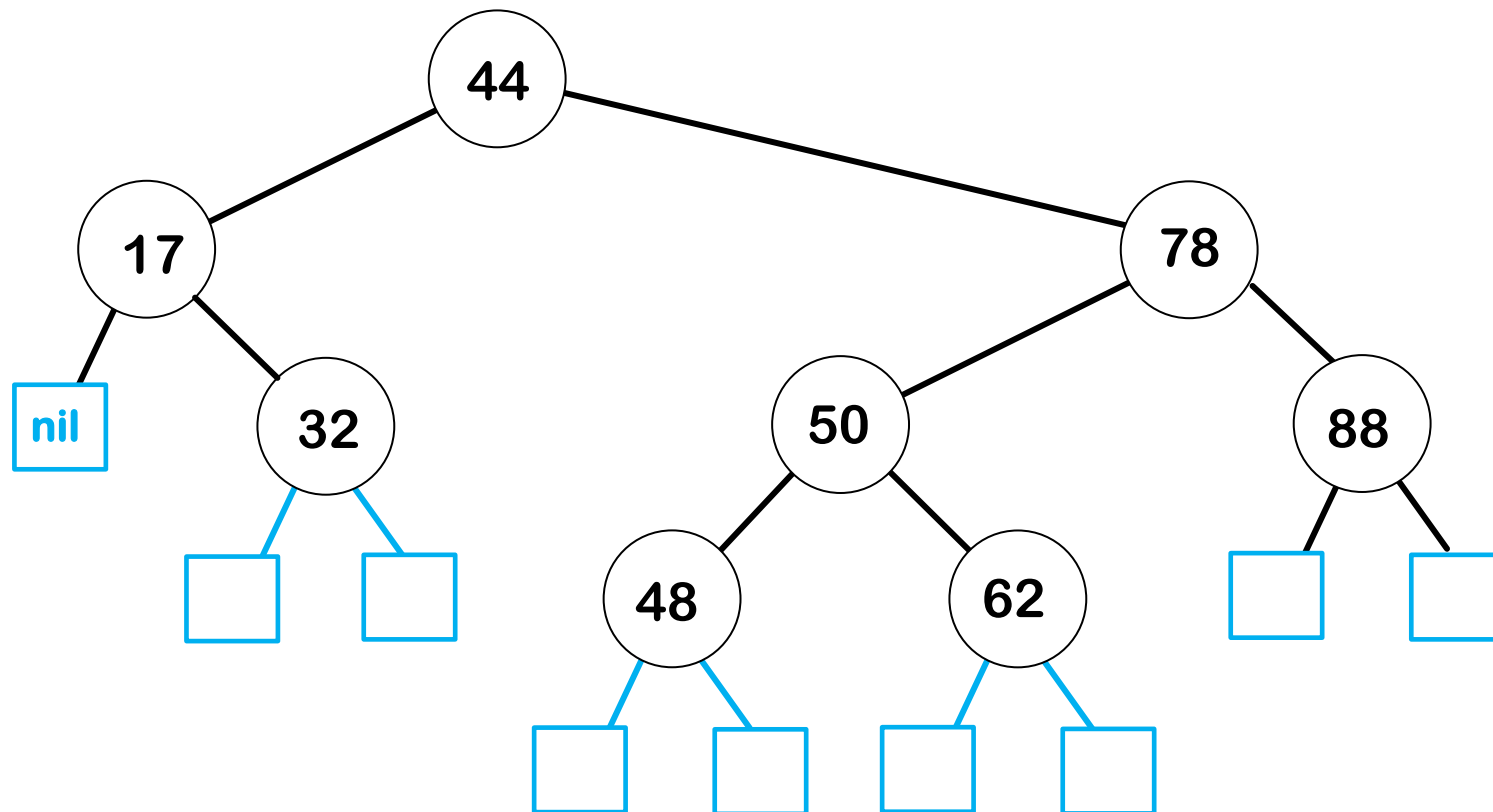


GOOD TREES FOR SEARCH



In order to avoid the poor performance, we need ways of maintaining the height of a search tree to be logarithmic in the number of nodes it has.

GENERALIZATION



DEFINITION

- **Rank-balanced tree** is a binary tree with logarithmic height.
 1. AVL trees
 2. Red-black trees
- By performing a certain kind of restructuring actions (**rotation**), we change a tree to a rank-balanced tree.

BALANCE FACTOR

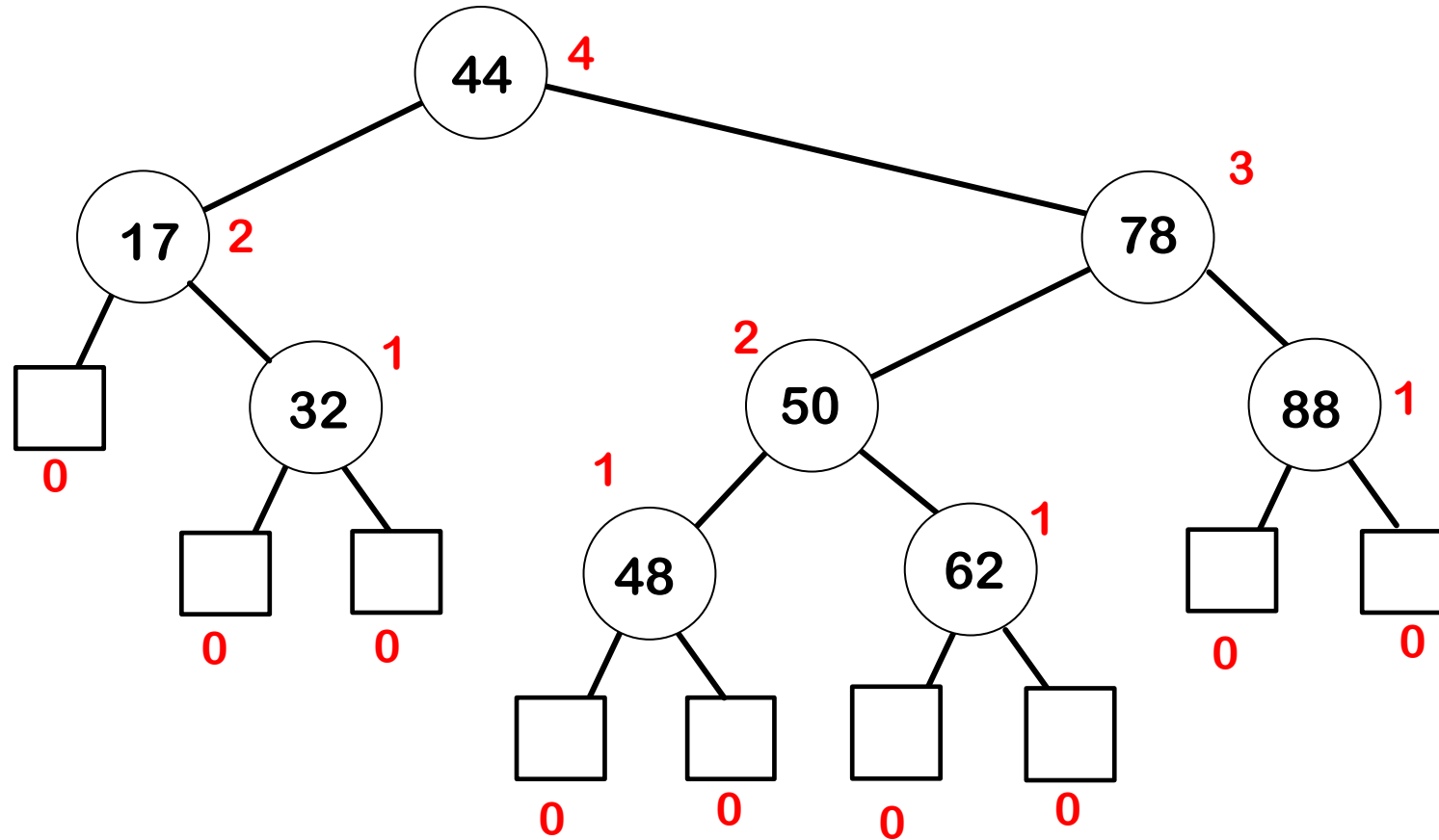
- Lets define the **Balance factor** of a node v as follows:

$$BF(v) = \text{height}(\text{left-subtree}) - \text{height}(\text{right-subtree})$$

- If for all nodes v , $|BF(v)| \leq 1$ the tree is called **AVL**.

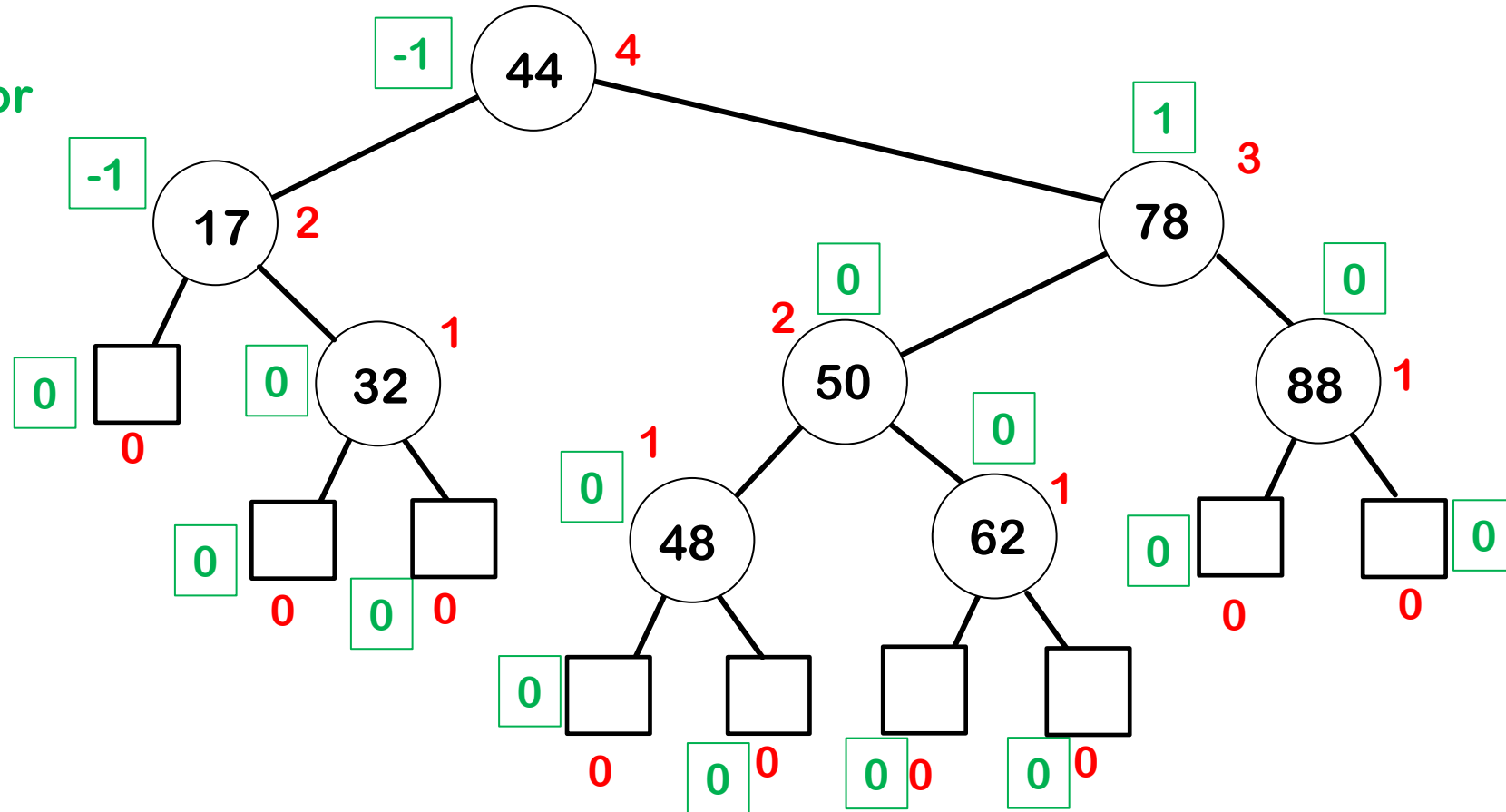
AVL: EXAMPLE

heights



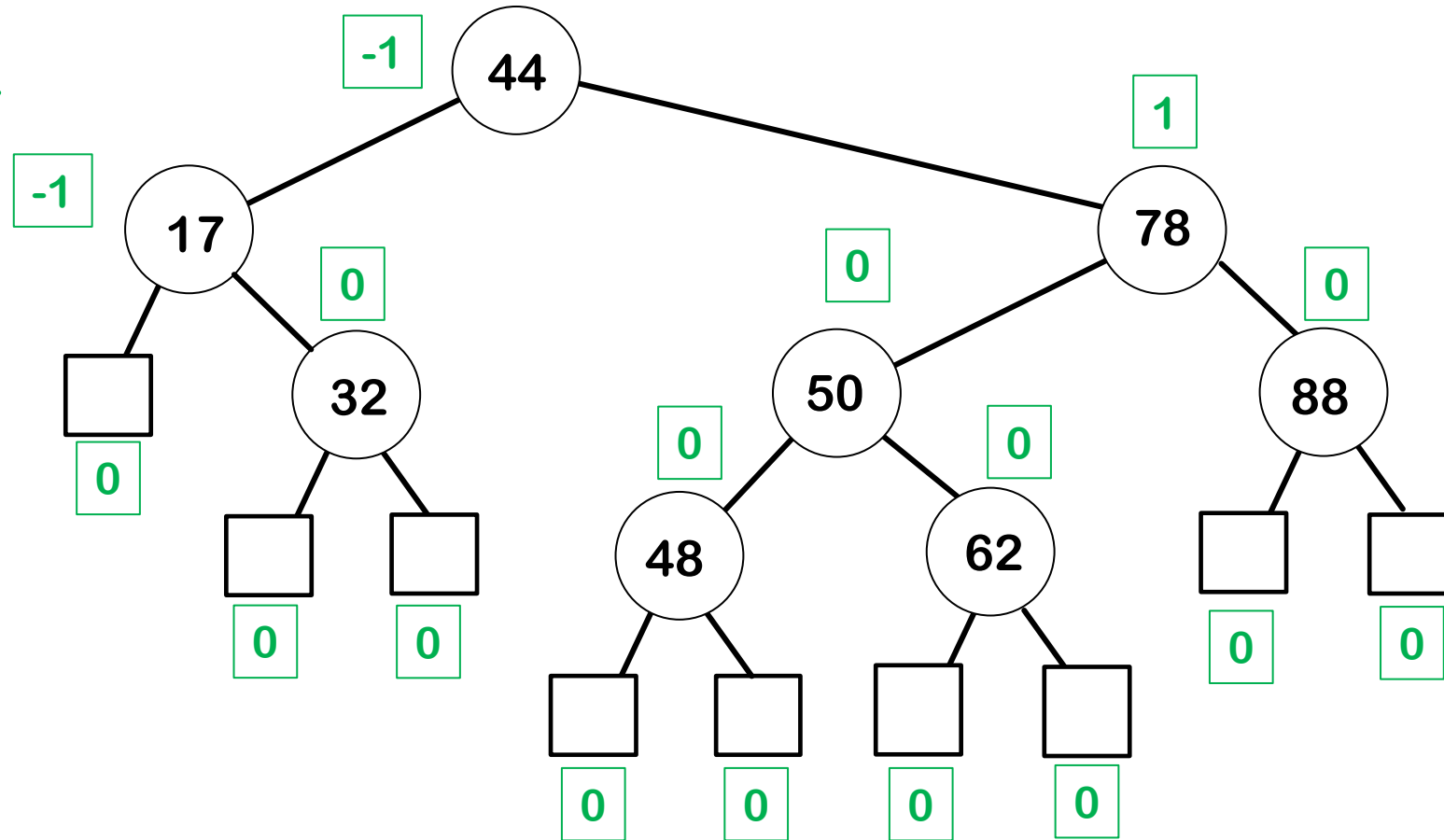
AVL: EXAMPLE

Balance factor



AVL: EXAMPLE

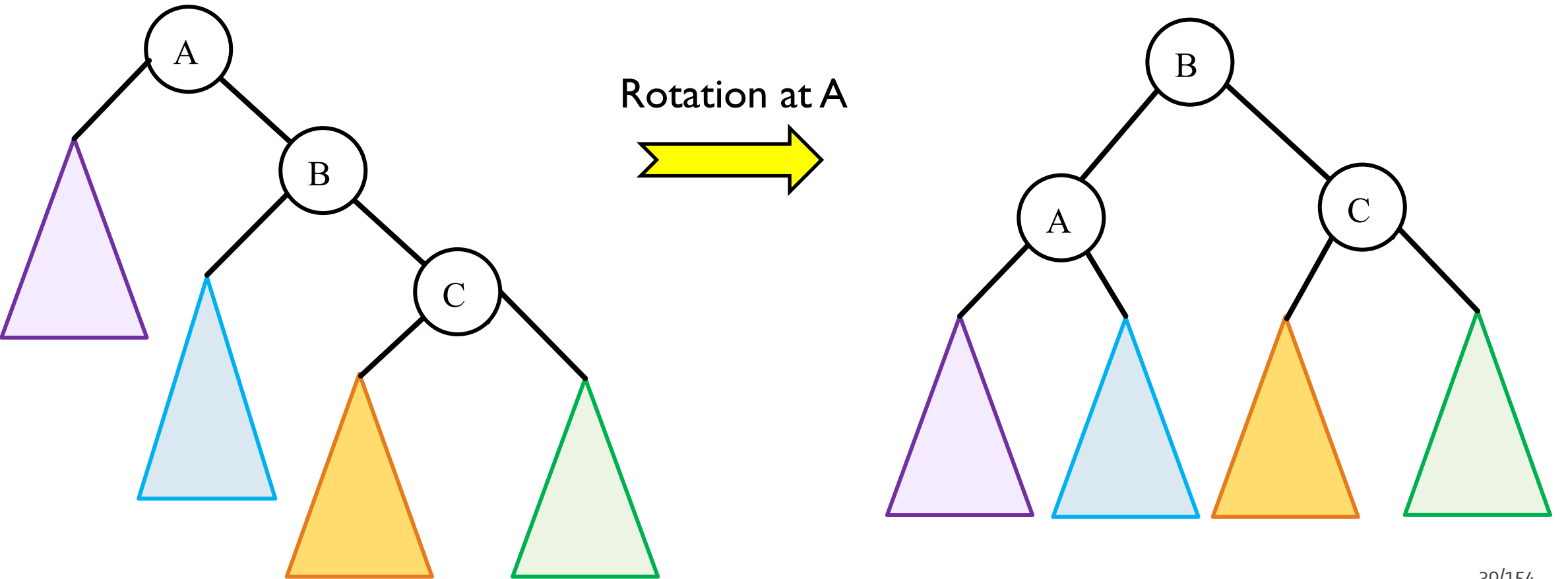
Balance factor



ROTATIONS

- An AVL tree may perform the following four kinds of rotations, when it is getting unbalanced:
 1. Single rotations
 - a) Left rotation
 - b) Right rotation
 2. Double rotations
 - a) Left-Right rotation
 - b) Right-Left rotation

ROTATIONS?

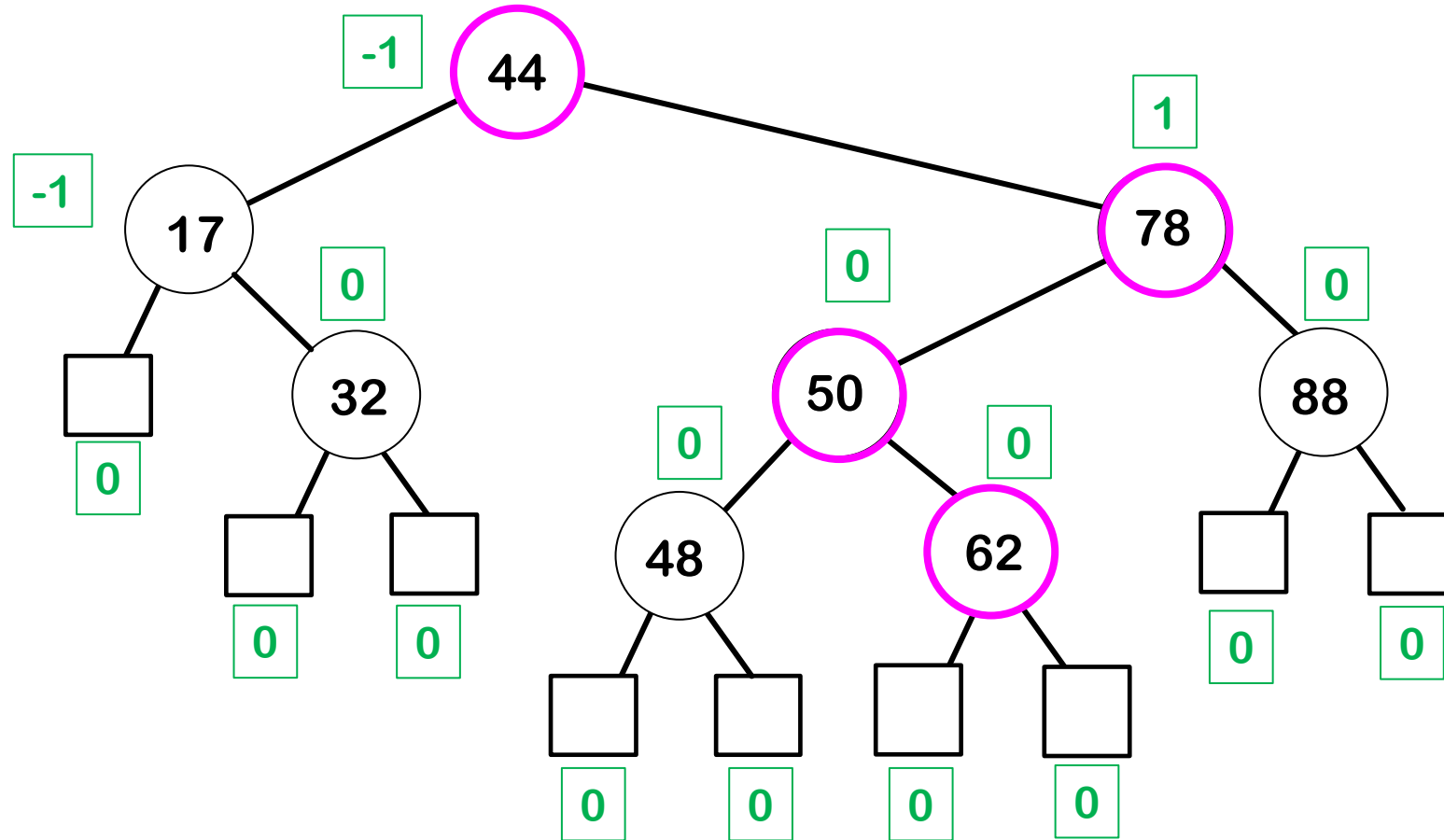


INSERTION

- By changing the way a key is inserted to a tree, the tree would be an AVL.
 1. Insert new key as usual
 2. If tree is not an AVL, Perform a **rotation**

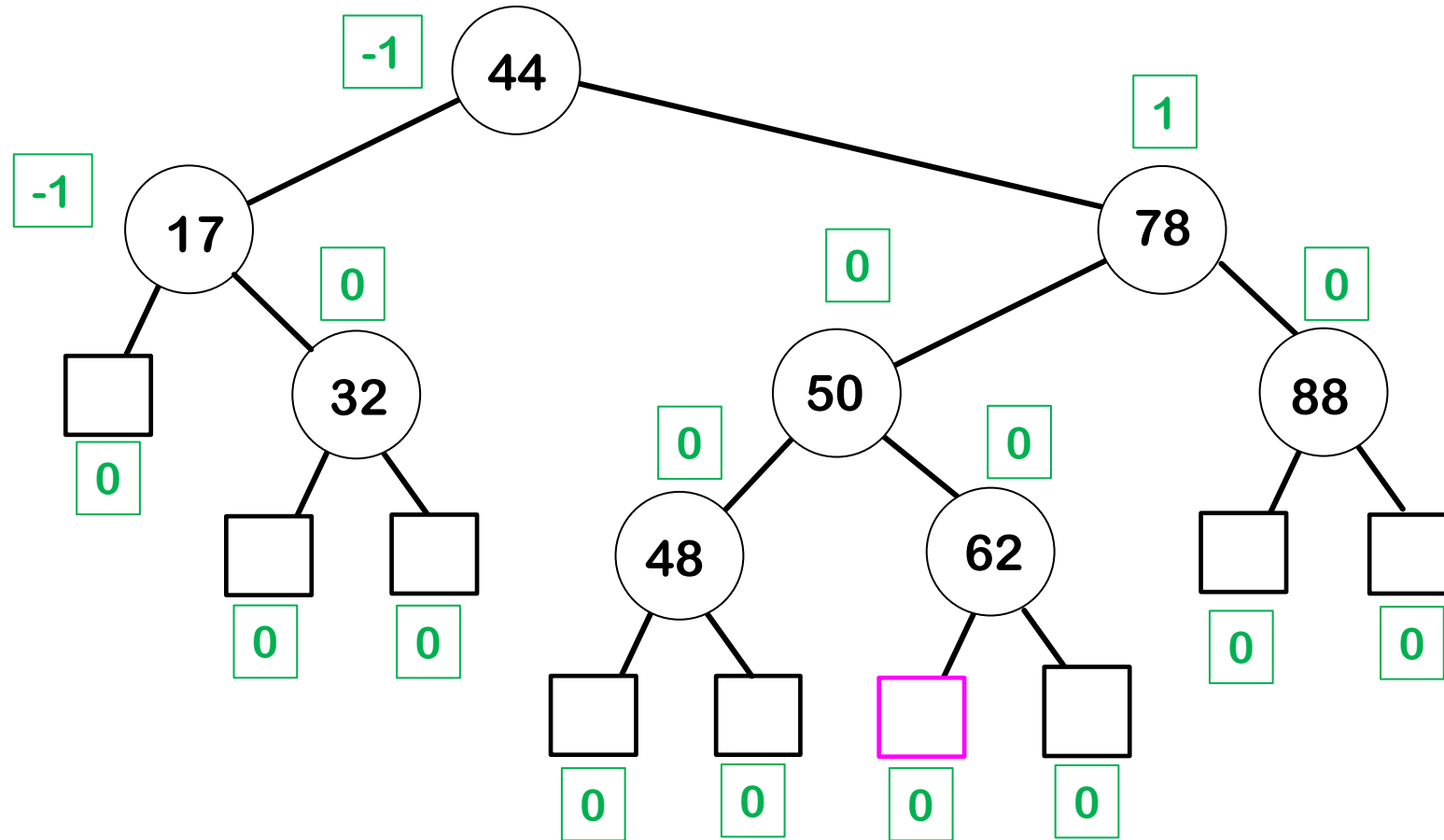
AVL: EXAMPLE

1. Insert(54)



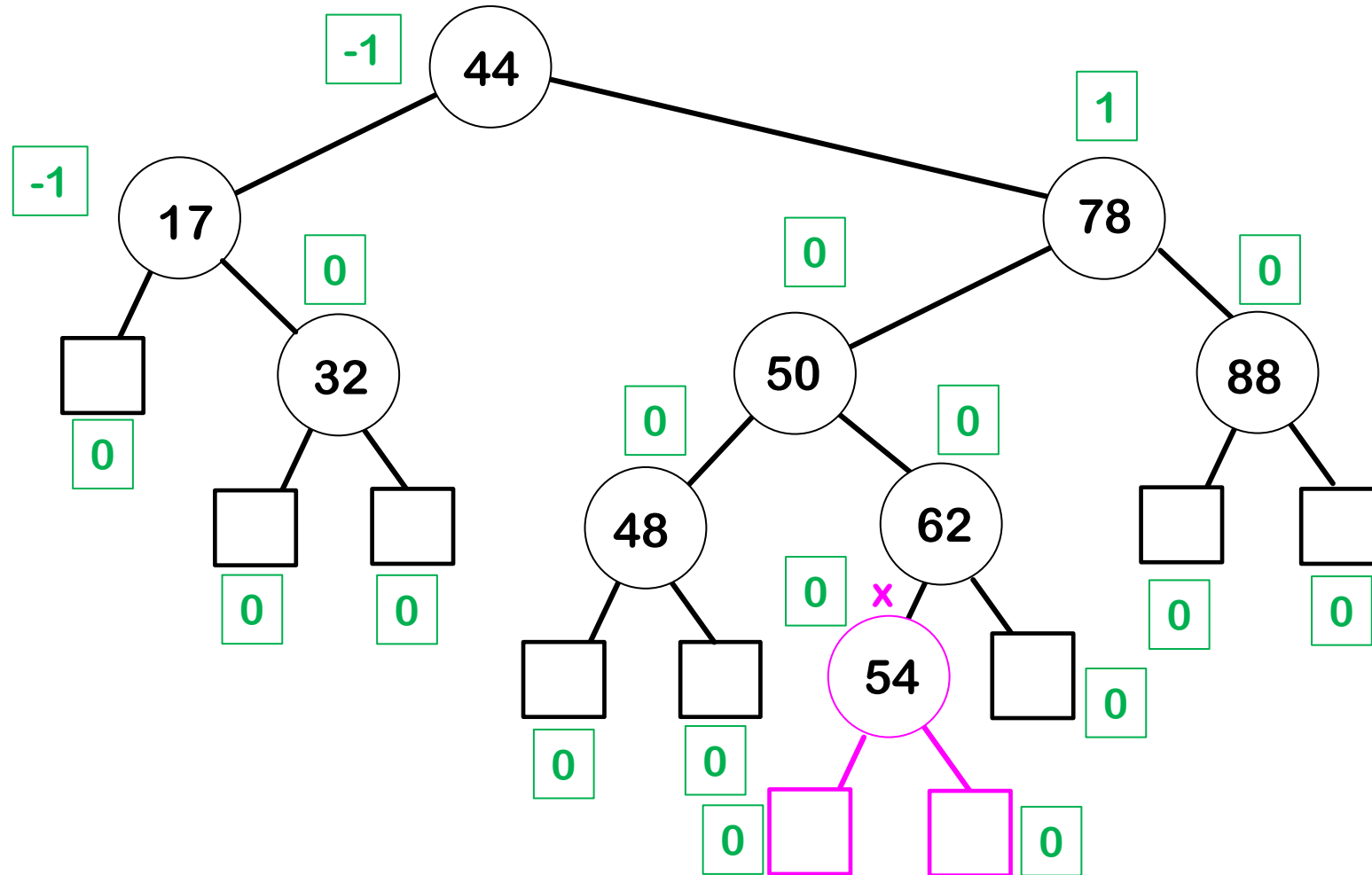
AVL: EXAMPLE

1. Insert(54)



AVL: EXAMPLE

1. Insert(54)

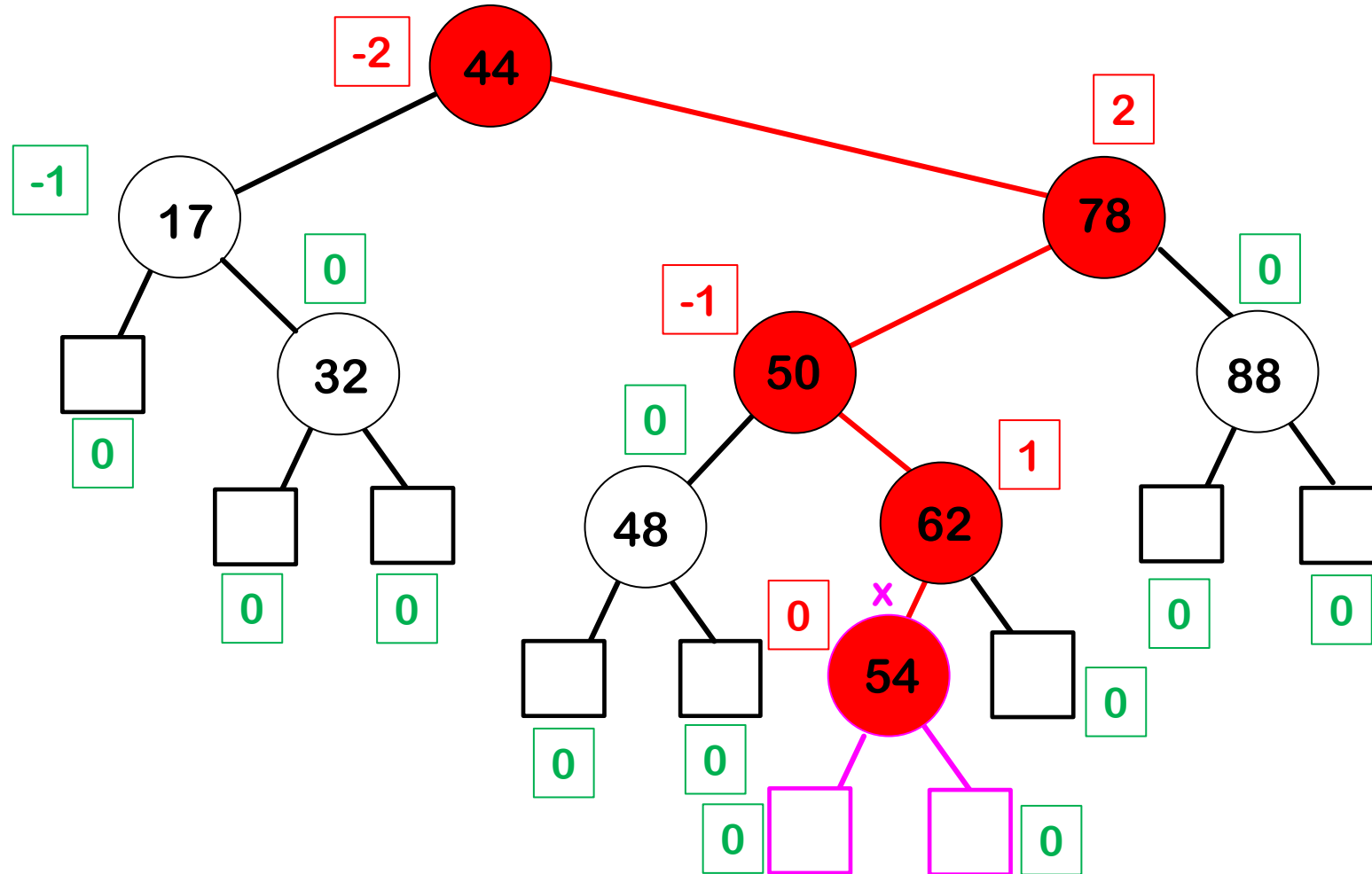


IS TREE STILL AN AVL?

- I. On the path from new node (x) to the root, update the balance factors.

AVL: EXAMPLE

1. Insert(54)
2. **Update BFs**

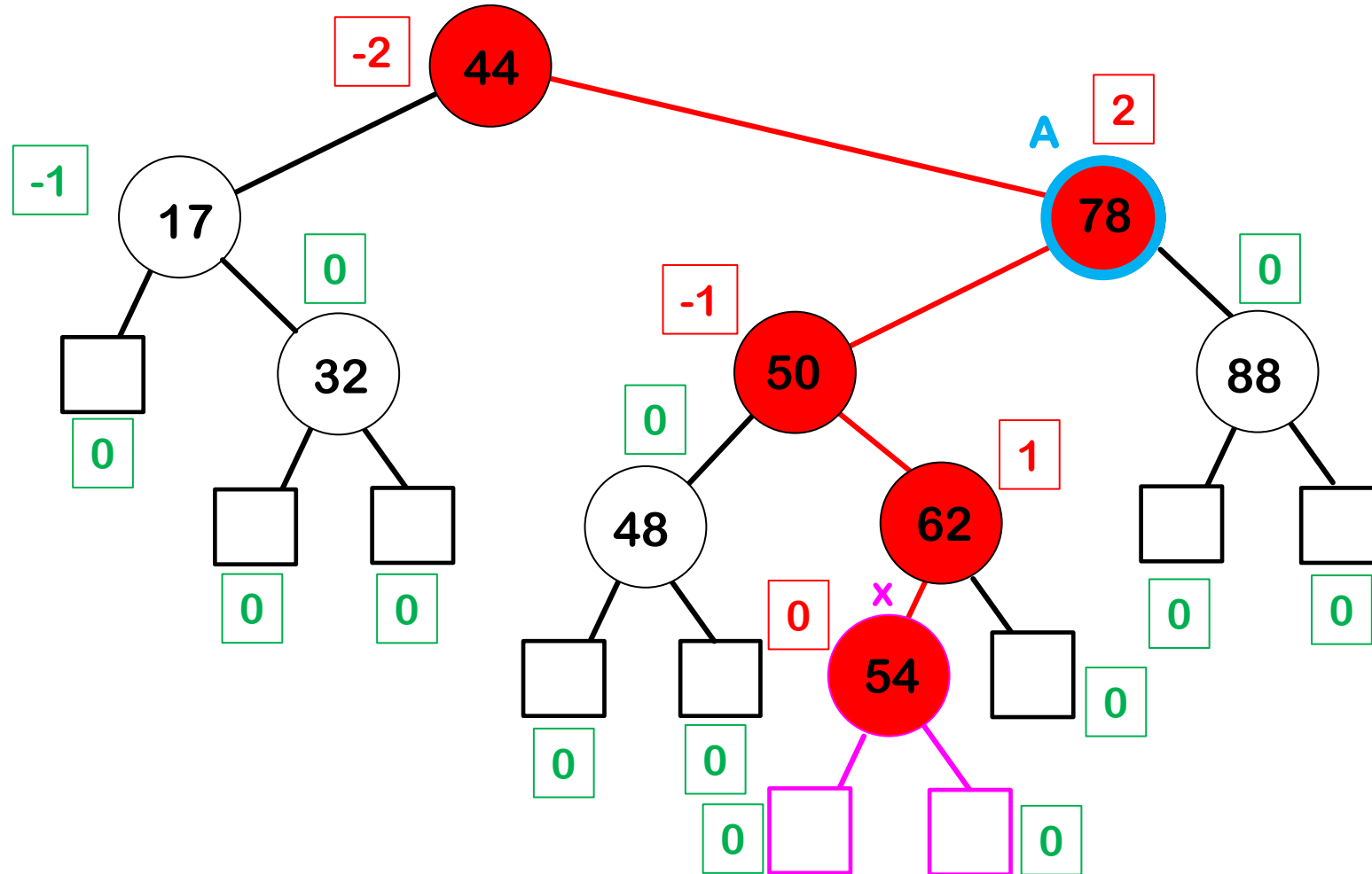


IS TREE STILL AN AVL?

1. On the path from **new node (x)** to the root, update the balance factors.
2. Find the first node with balance factor ± 2 . Denote this by A.

AVL: EXAMPLE

1. Insert(54)
2. **Update BFs**
3. **Find the first ancestor with BF of 2**

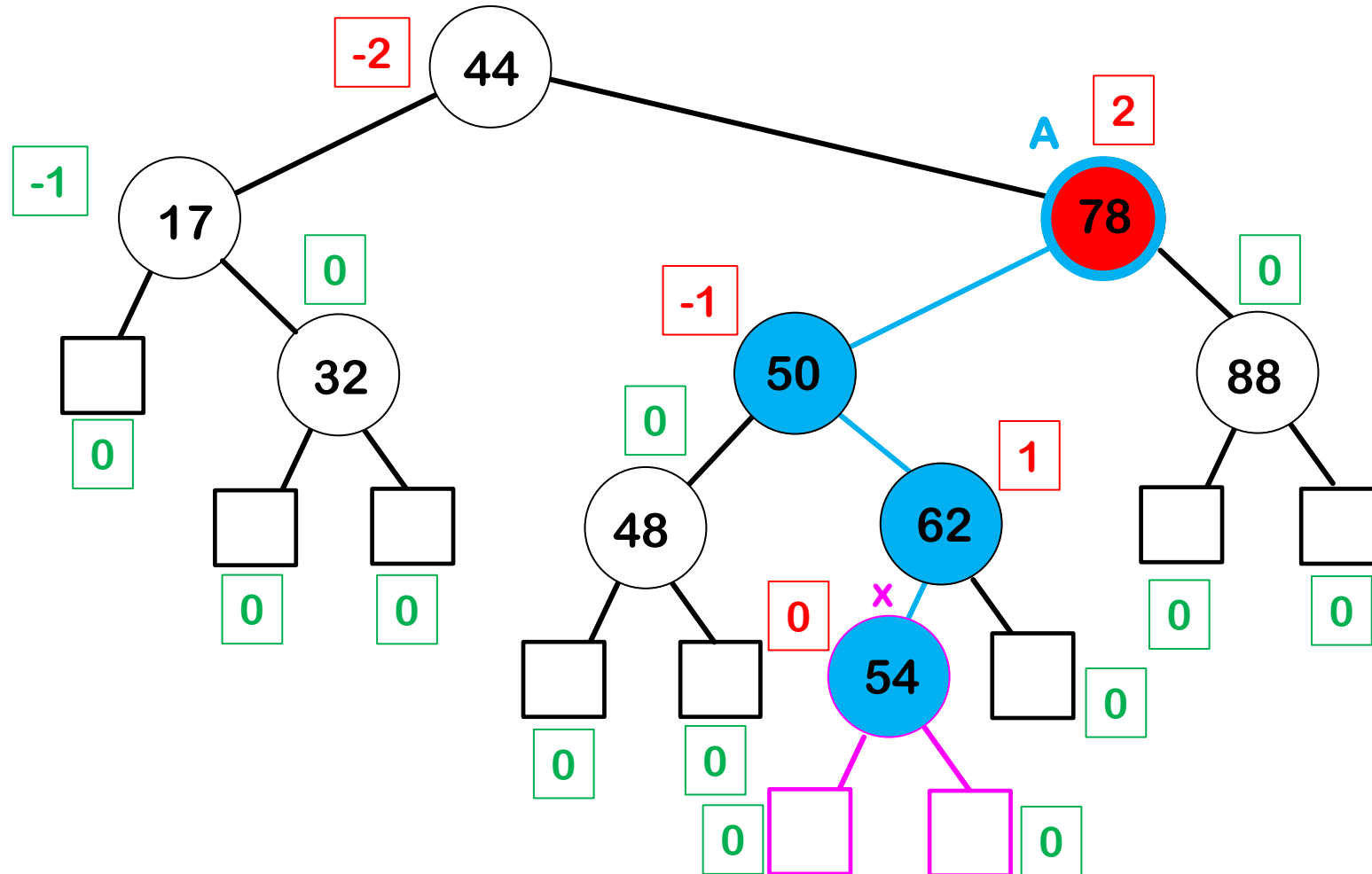


IS TREE STILL AN AVL?

1. On the path from **new node (x)** to the root, update the balance factors.
2. Find the first node with balance factor ± 2 . Denote this by A.
3. On the path from x to A, denote the child and grandchild of A by B and C respectively.

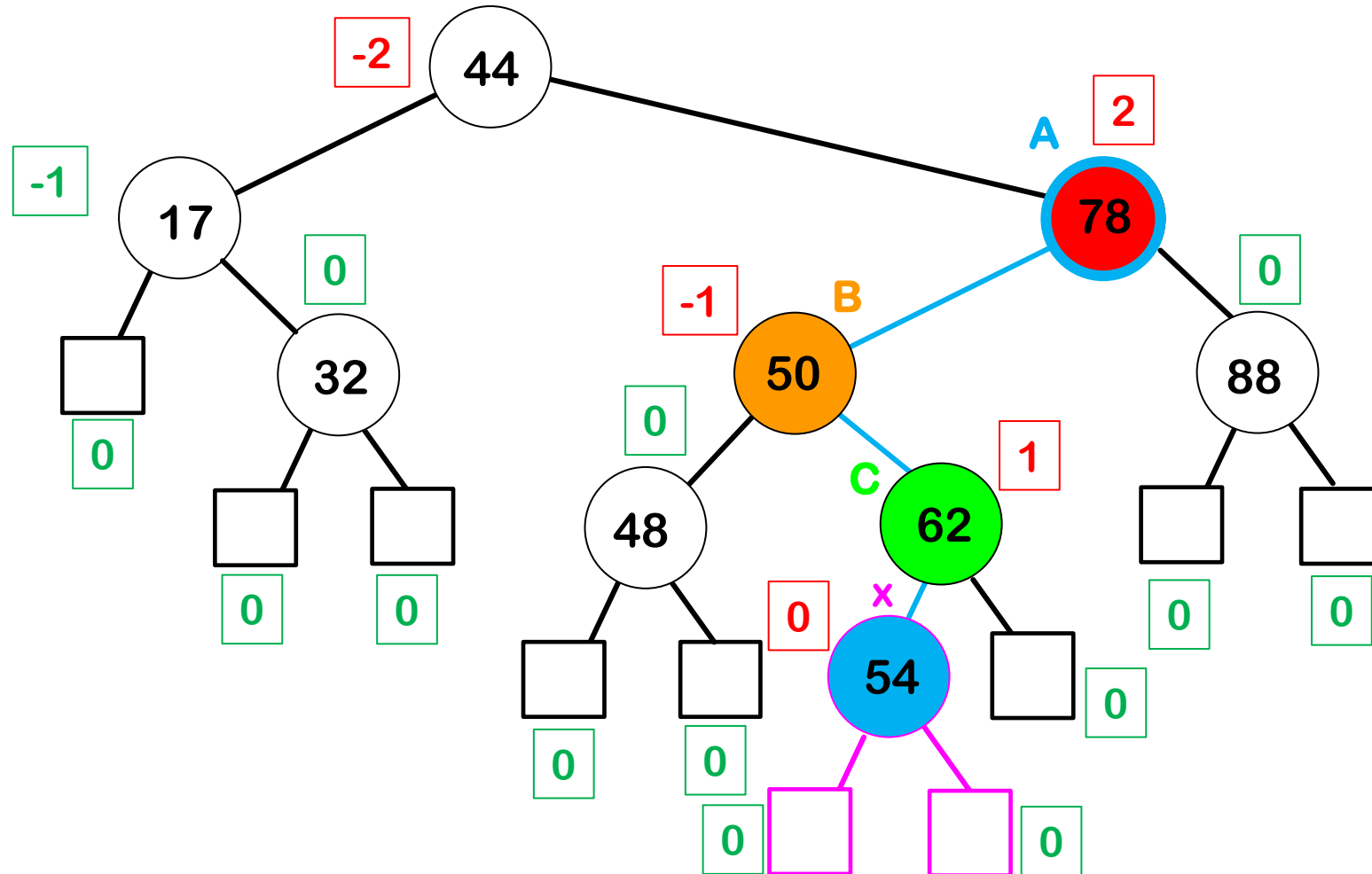
AVL: EXAMPLE

1. Insert(54)
2. **Update BFs**
3. **Find the first ancestor with BF of 2**
4. B and C?



AVL: EXAMPLE

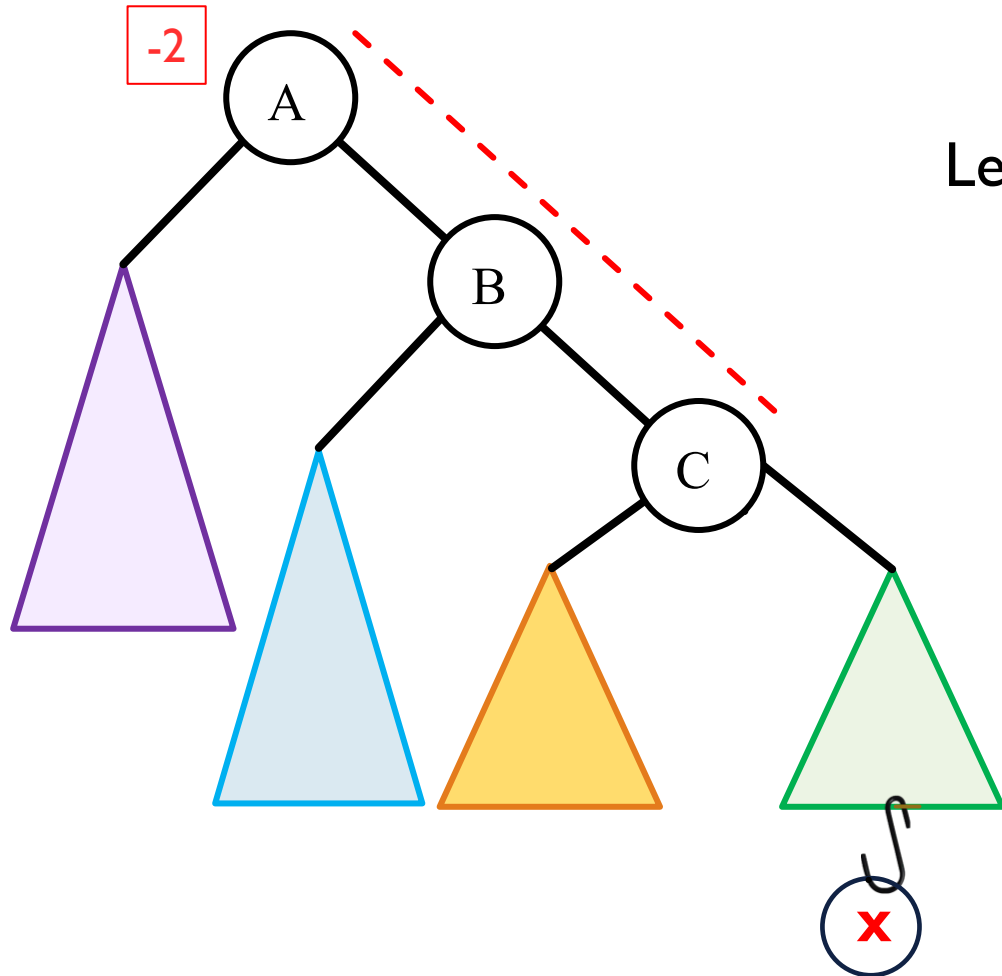
1. Insert(54)
2. **Update BFs**
3. **Find the first ancestor with BF of 2**
4. B and C: Rotation?



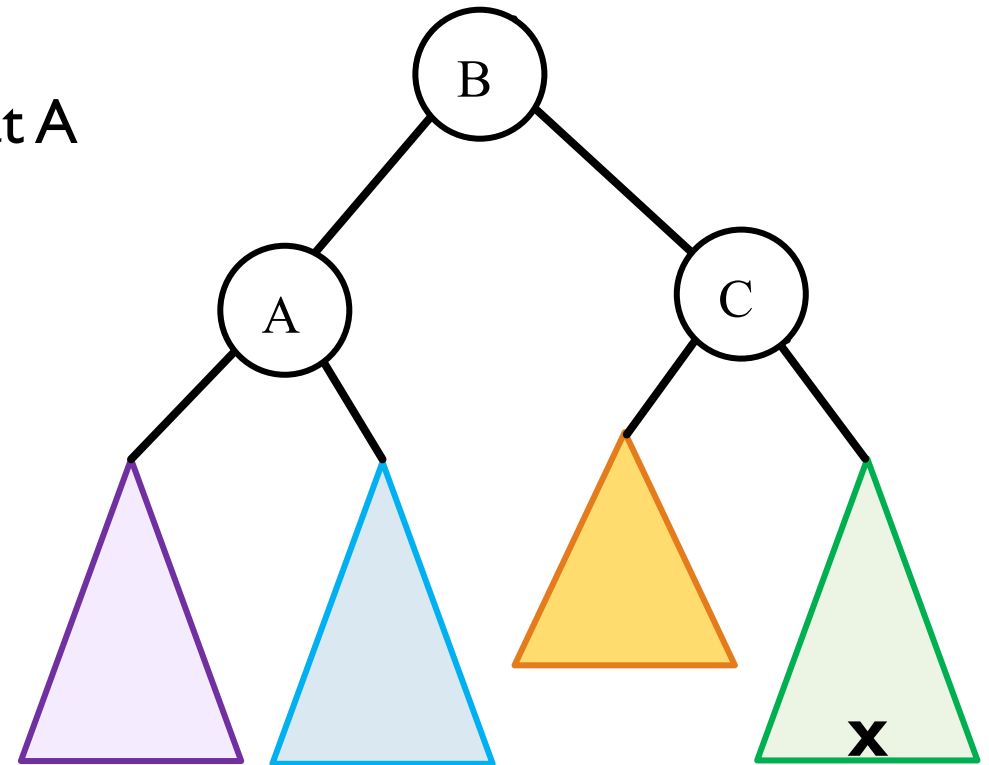
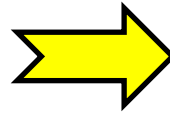
IS TREE STILL AN AVL?

1. On the path from **new node (x)** to the root, update the balance factors.
2. Find the first node with balance factor ± 2 . Denote this by A.
3. On the path from A to x, denote the child and grandchild of A by B and C respectively.
4. Based on the following cases perform a kind of rotation:
 - I. B is left child of A and C is left child of B
 - II. B is left child of A and C is right child of B
 - III. B is right child of A and C is right child of B
 - IV. B is right child of A and C is left child of B

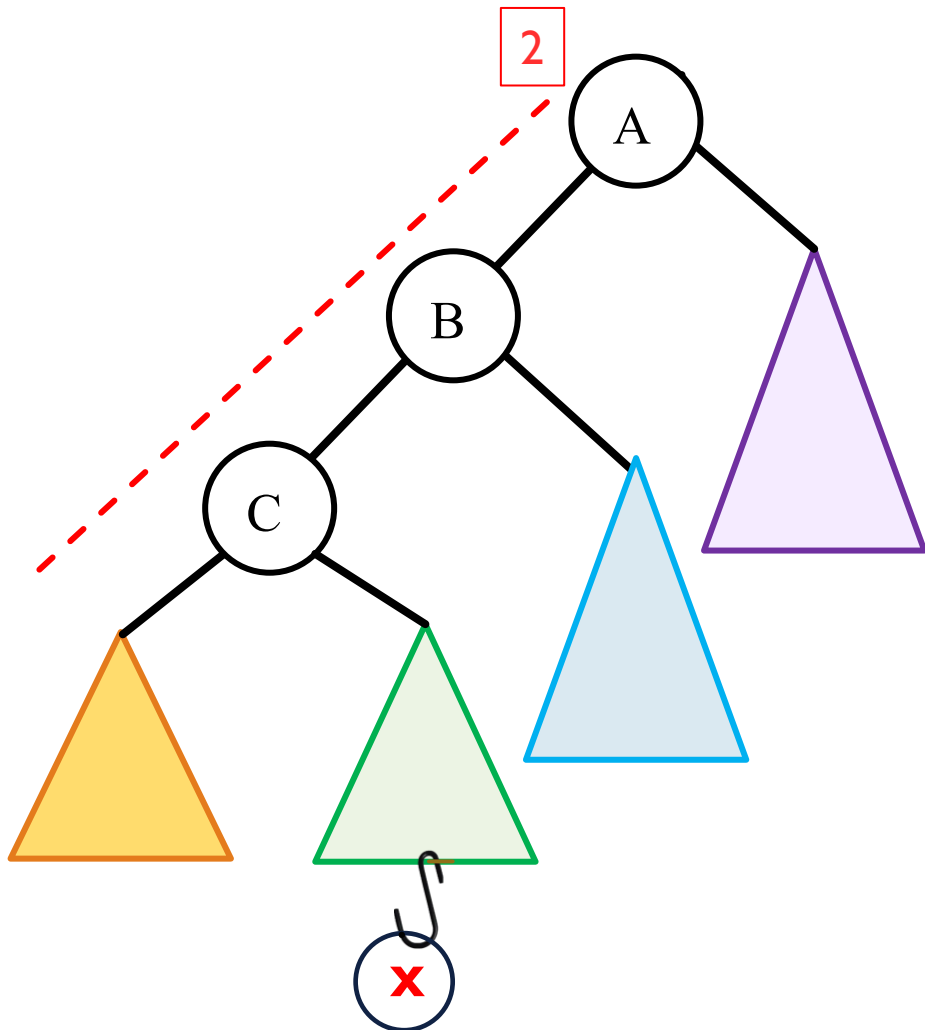
LEFT ROTATION



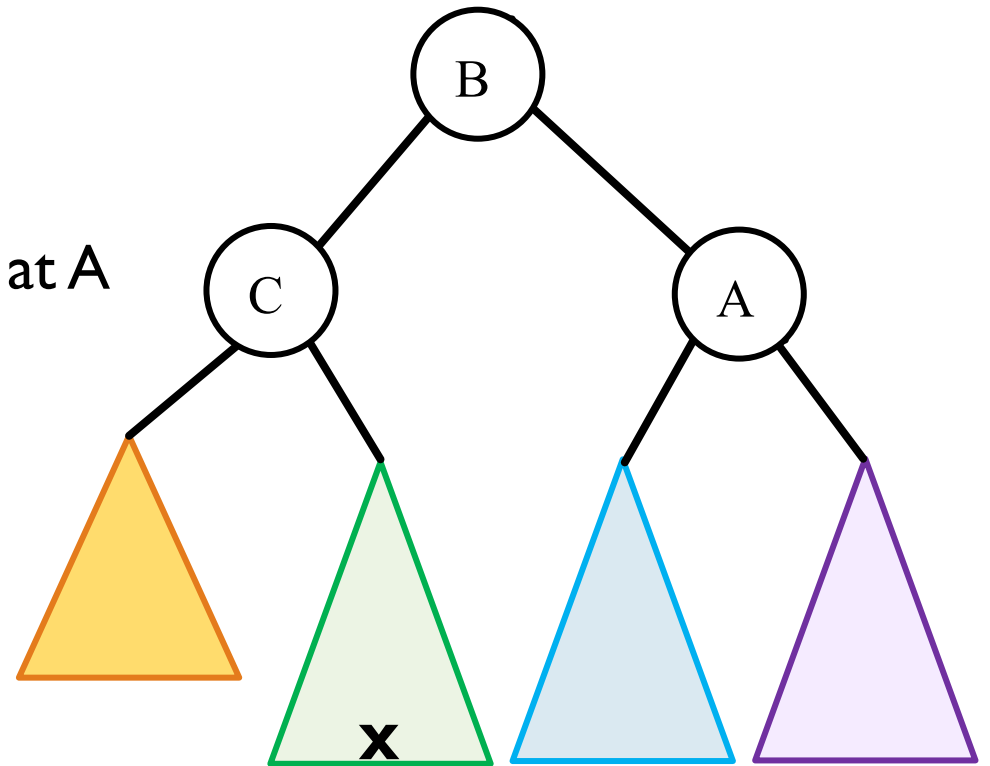
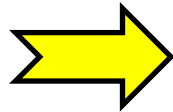
Left Rotation at A



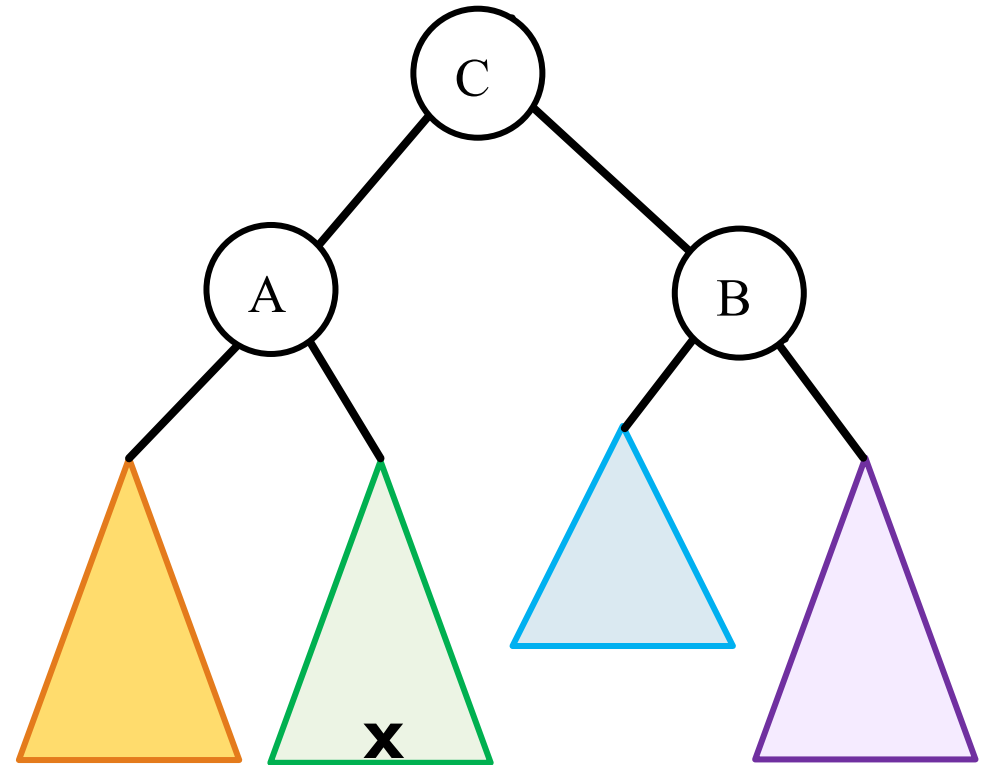
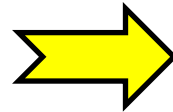
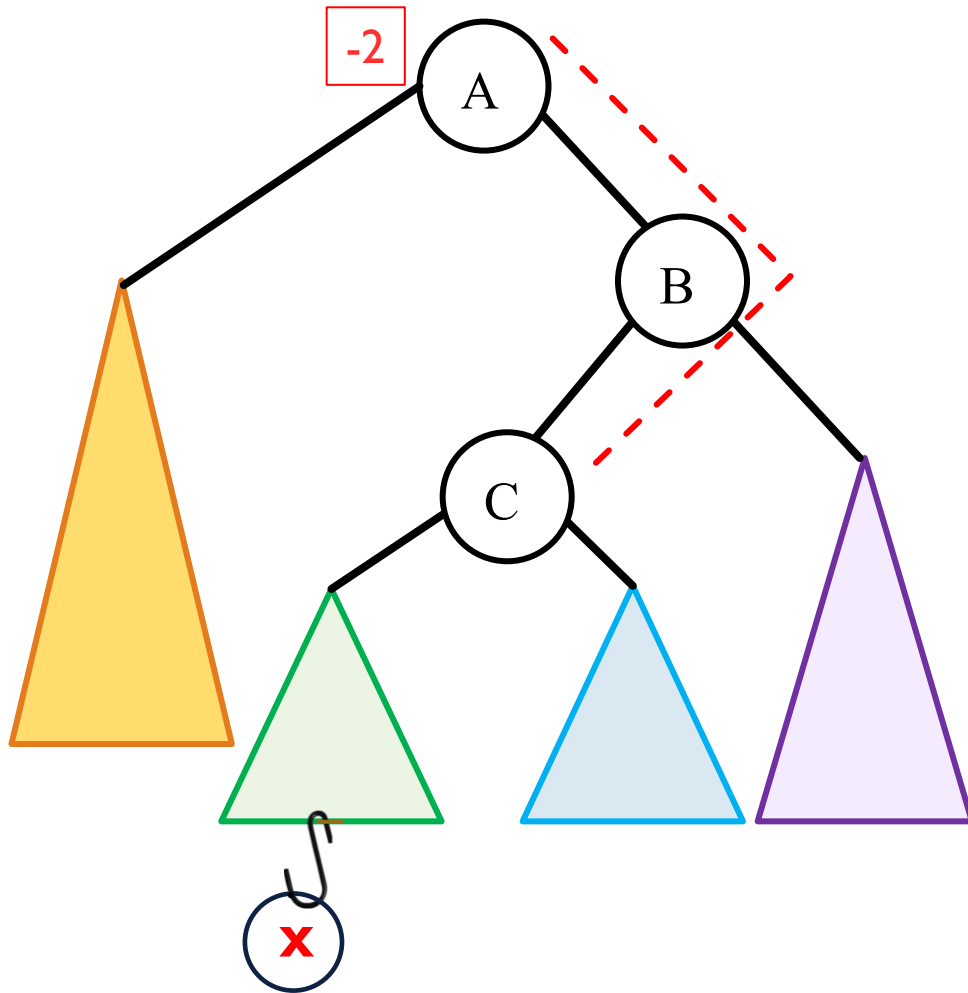
RIGHT ROTATION



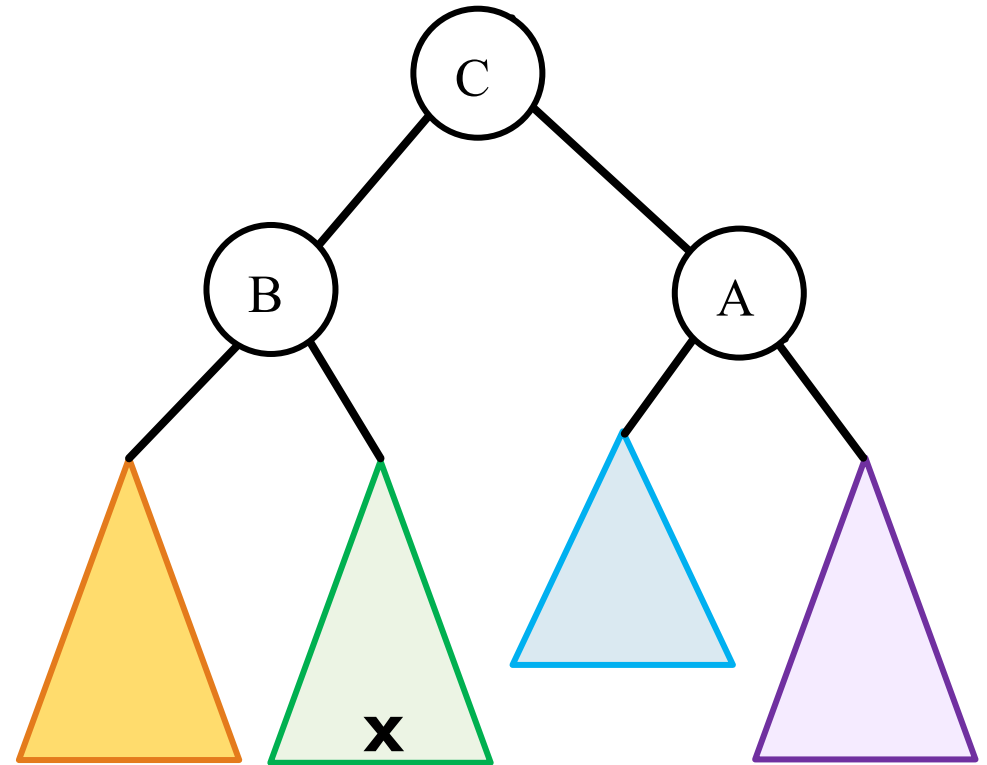
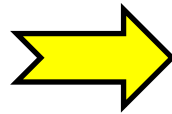
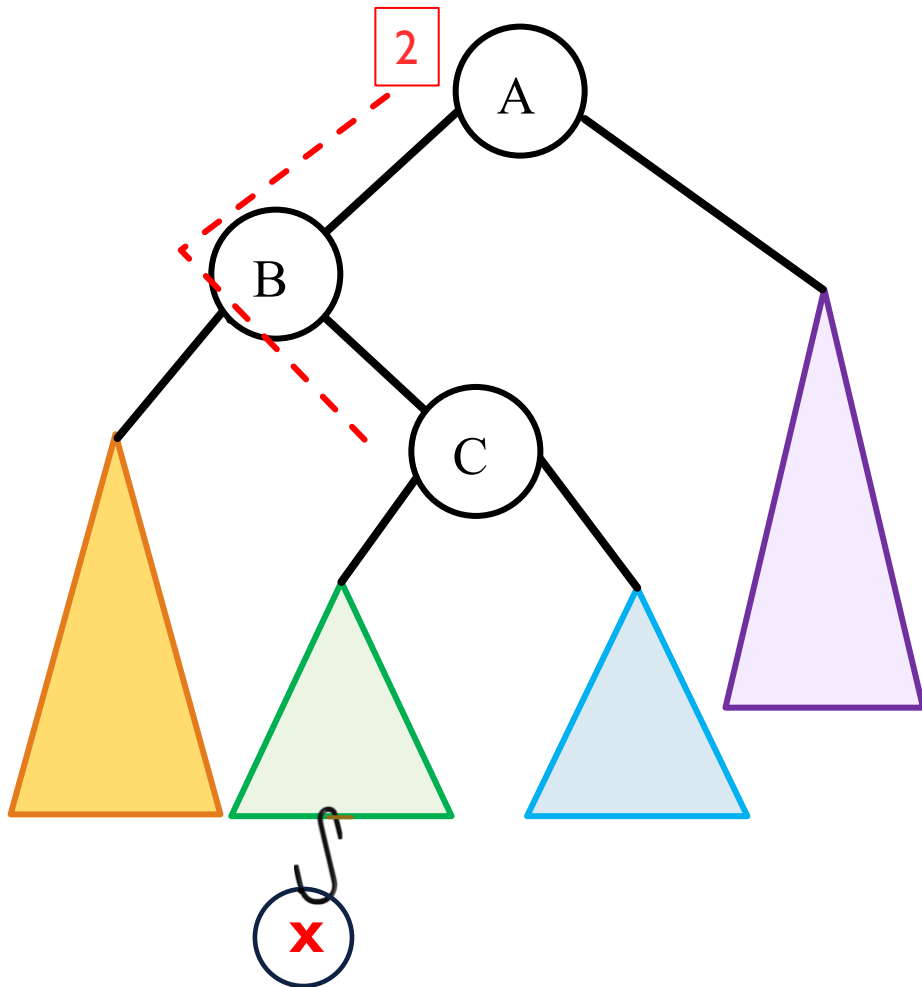
Right Rotation at A



RIGHT-LEFT ROTATION

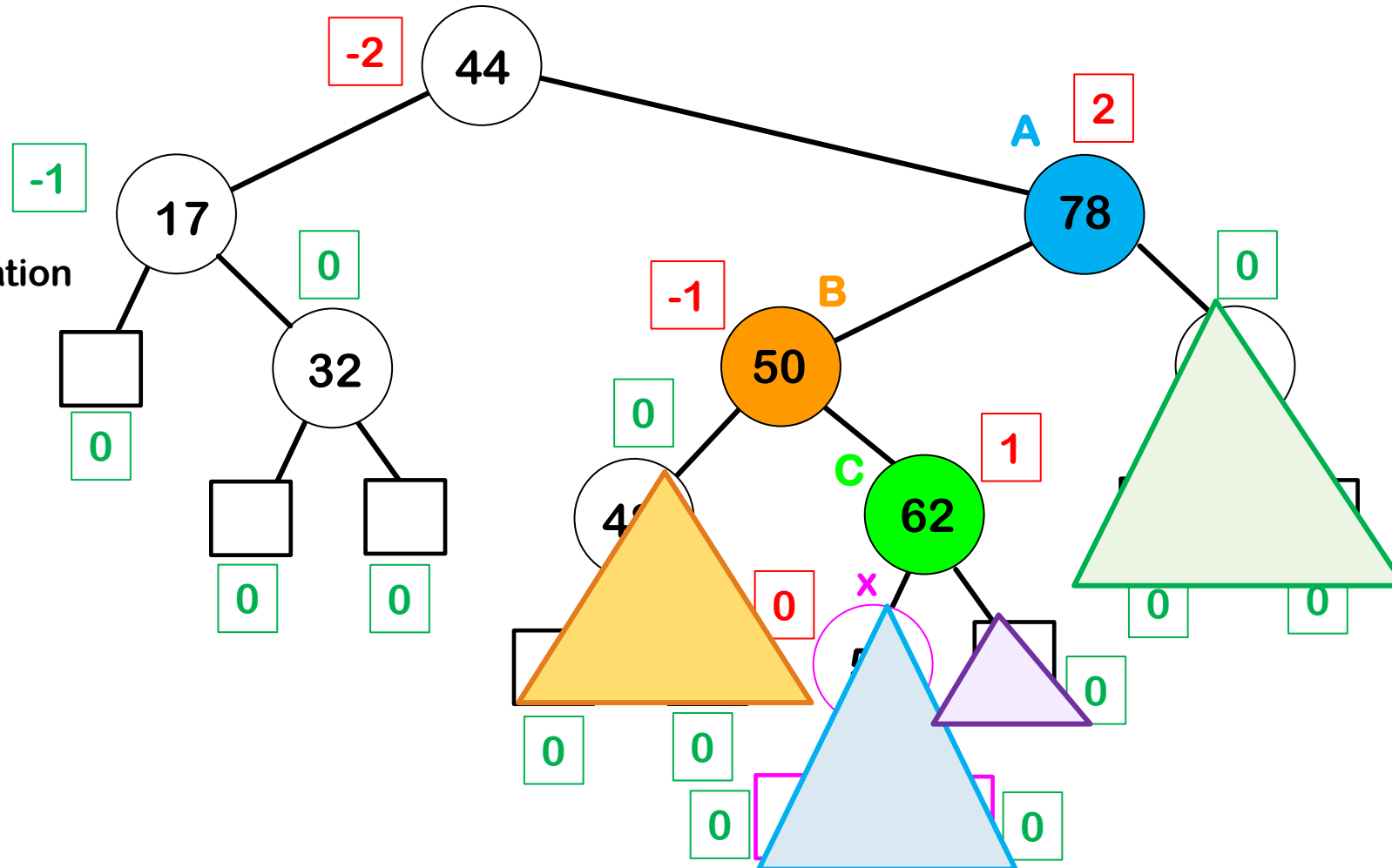


LEFT-RIGHT ROTATION



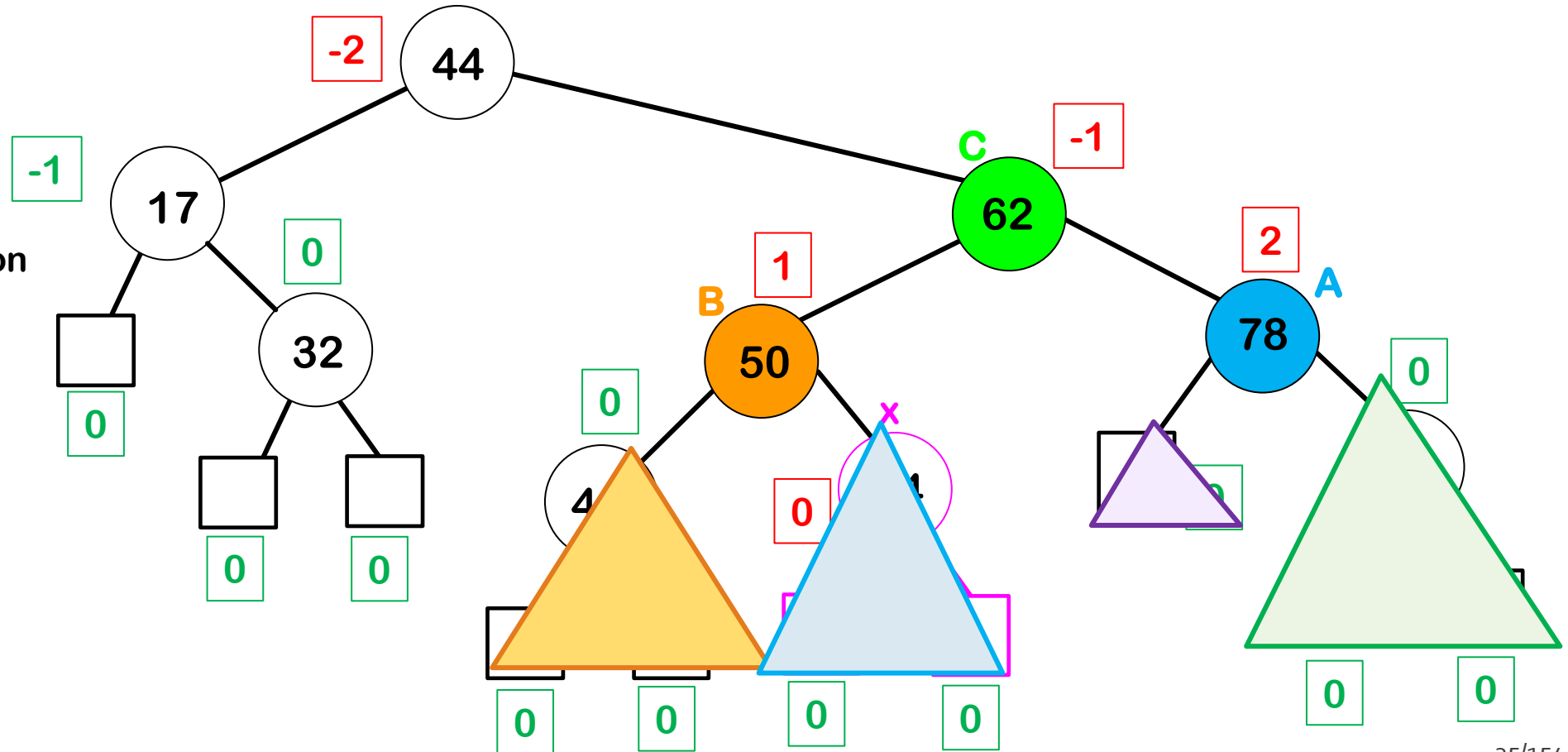
AVL: EXAMPLE

1. Insert(54)
2. **Update BFs**
3. **Find the first ancestor with BF of 2**
4. Determine the rotation



AVL: EXAMPLE

1. Insert(54)
2. **Update BFs**
3. Find the first ancestor with **BF of 2**
4. Perform the rotation

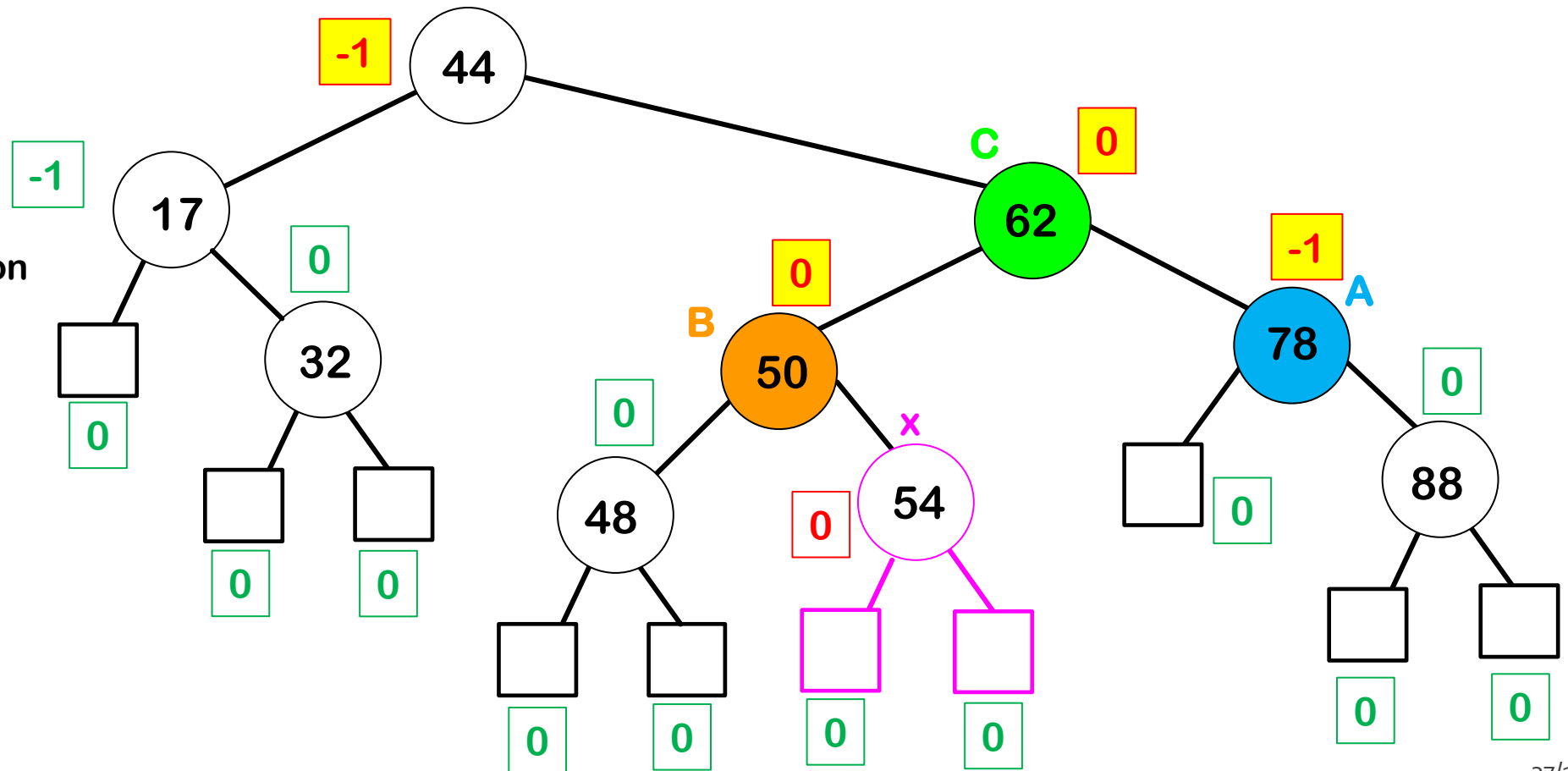


IS TREE STILL AN AVL?

1. On the path from **new node (x)** to the root, update the balance factors.
2. Find the first node with balance factor ± 2 . Denote this by A.
3. On the path from A to x, denote the child and grandchild of A by B and C respectively.
4. Perform the rotation
5. Update the heights and balance factors of A,B,C and all nodes from A to root.

AVL: EXAMPLE

1. Insert(54)
2. **Update BFs**
3. **Find the first ancestor with BF of 2**
4. Perform the rotation
5. Update heights and BFs

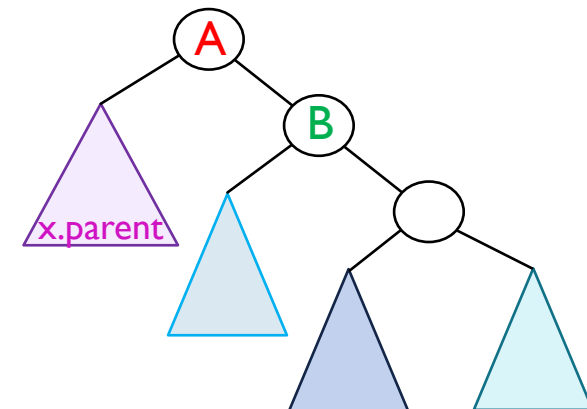


REMOVAL

- By changing the way a key is deleted from a tree, it would remain an AVL.
 1. Delete the key as usual
 2. If tree is not an AVL, Perform a **rotation**

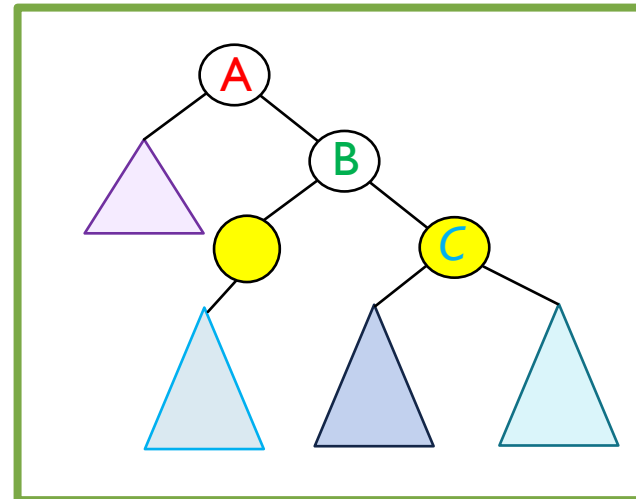
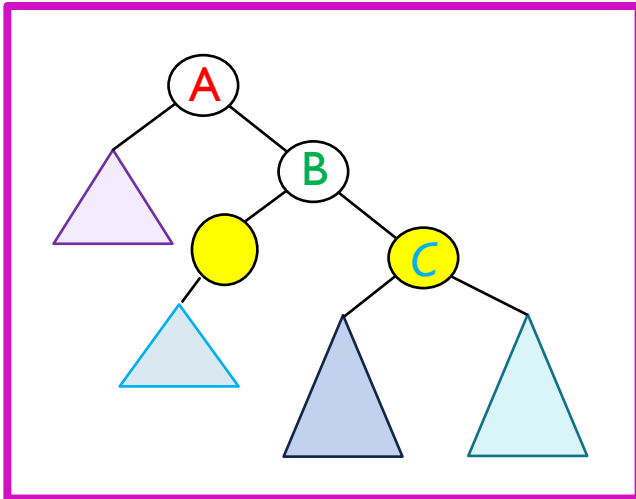
REMOVE

1. Delete the node as usual (BST deletion). Let x be the physically deleted node.
2. Set $\text{current_node} = x.\text{parent}$
3. While $\text{current_node} \neq \text{NULL}$, do the following:
 - Set $p = \text{current_node}.\text{parent}$
 - Update height and balance factor of current_node .
 - If current_node is unbalanced then
 - Let $A = \text{current_node}$ (first unbalanced node).
 - Let $B =$ taller child of A .
 - Let $C =$ taller child of B (or same-side child if equal height).
 - Perform rotation based on B and C 's positions relative to A .
 - Update heights/Balance factors after the rotation.
 - Move upward: $\text{current_node} = p$



REMOVE: HOW TO FIND C?

Let **C** = taller child of **B** (or same-side child if equal height).

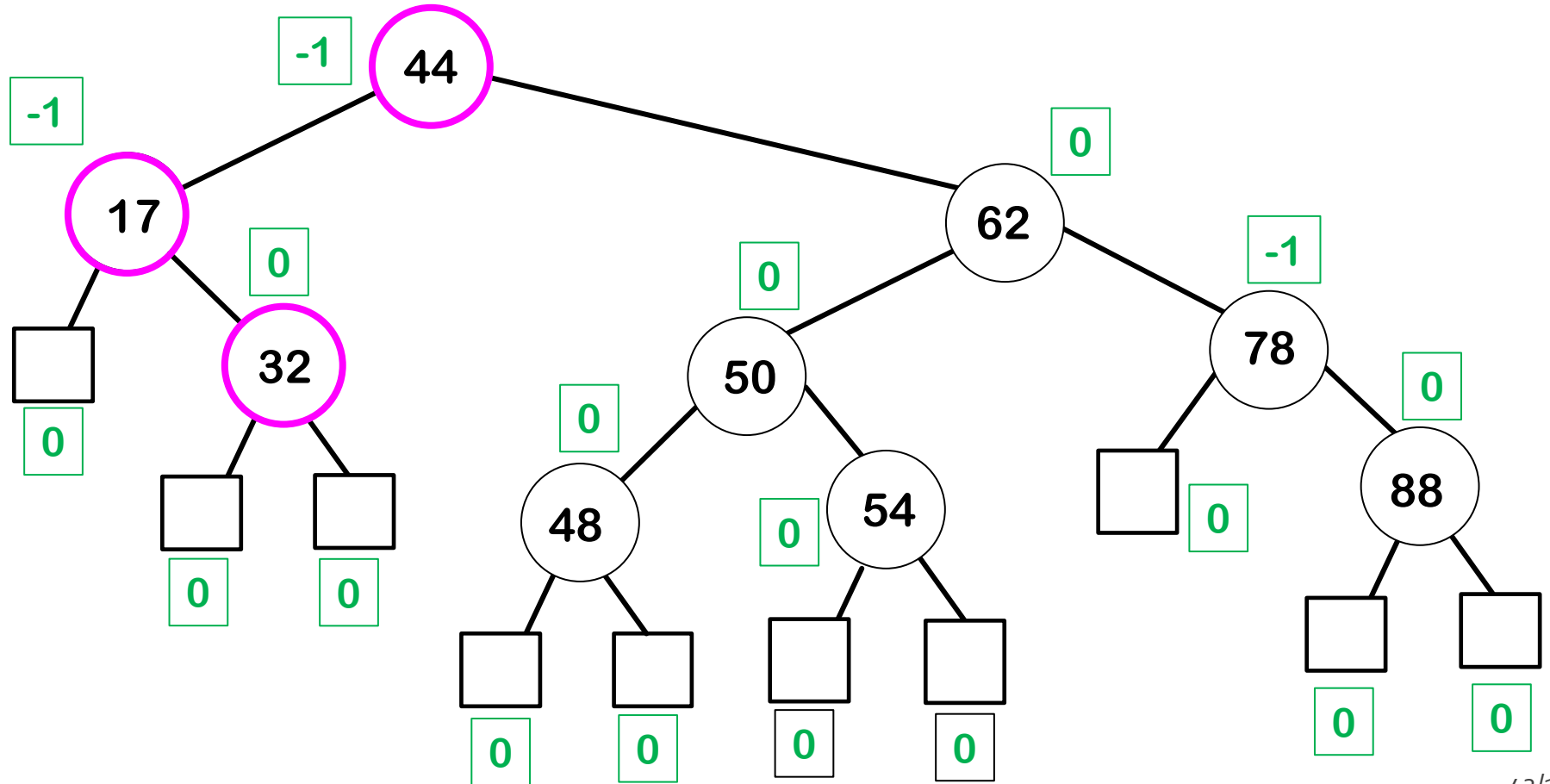


REMOVE: HOW TO ROTATE?

- **Perform rotation based on B and C's positions relative to A:**
 - B is the left child of A and C is the left child of B (LL Case) : Right Rotate(A)
 - B is the left child of A and C is the right child of B (LR Case) : Left Rotate(B) → Right Rotate(A)
 - B is the right child of A and C is the right child of B (RR Case): Left Rotate(A)
 - B is the right child of A and C is the left child of B (RL Case): Right Rotate(B) → Left Rotate(A)

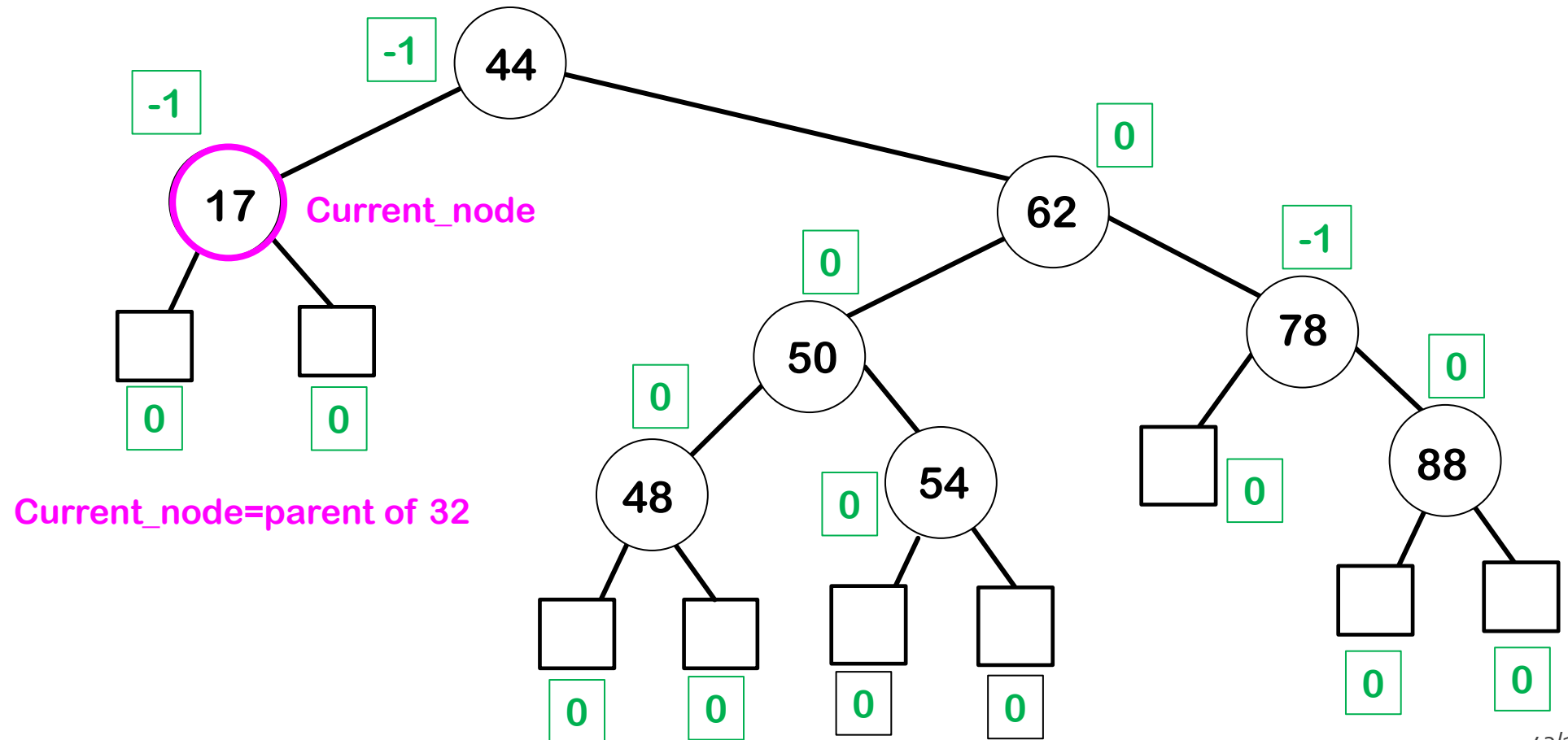
AVL: EXAMPLE

1. delete(32)



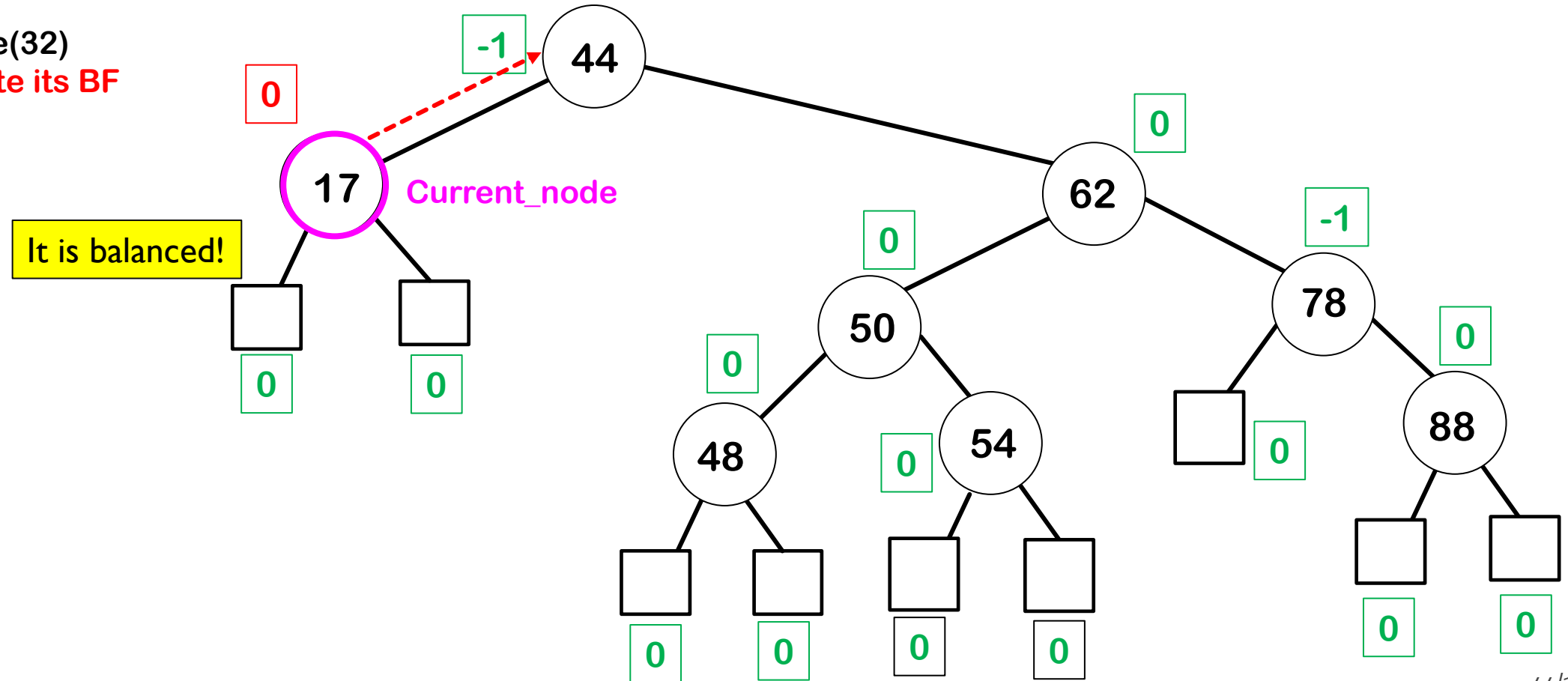
AVL: EXAMPLE

1. delete(32)



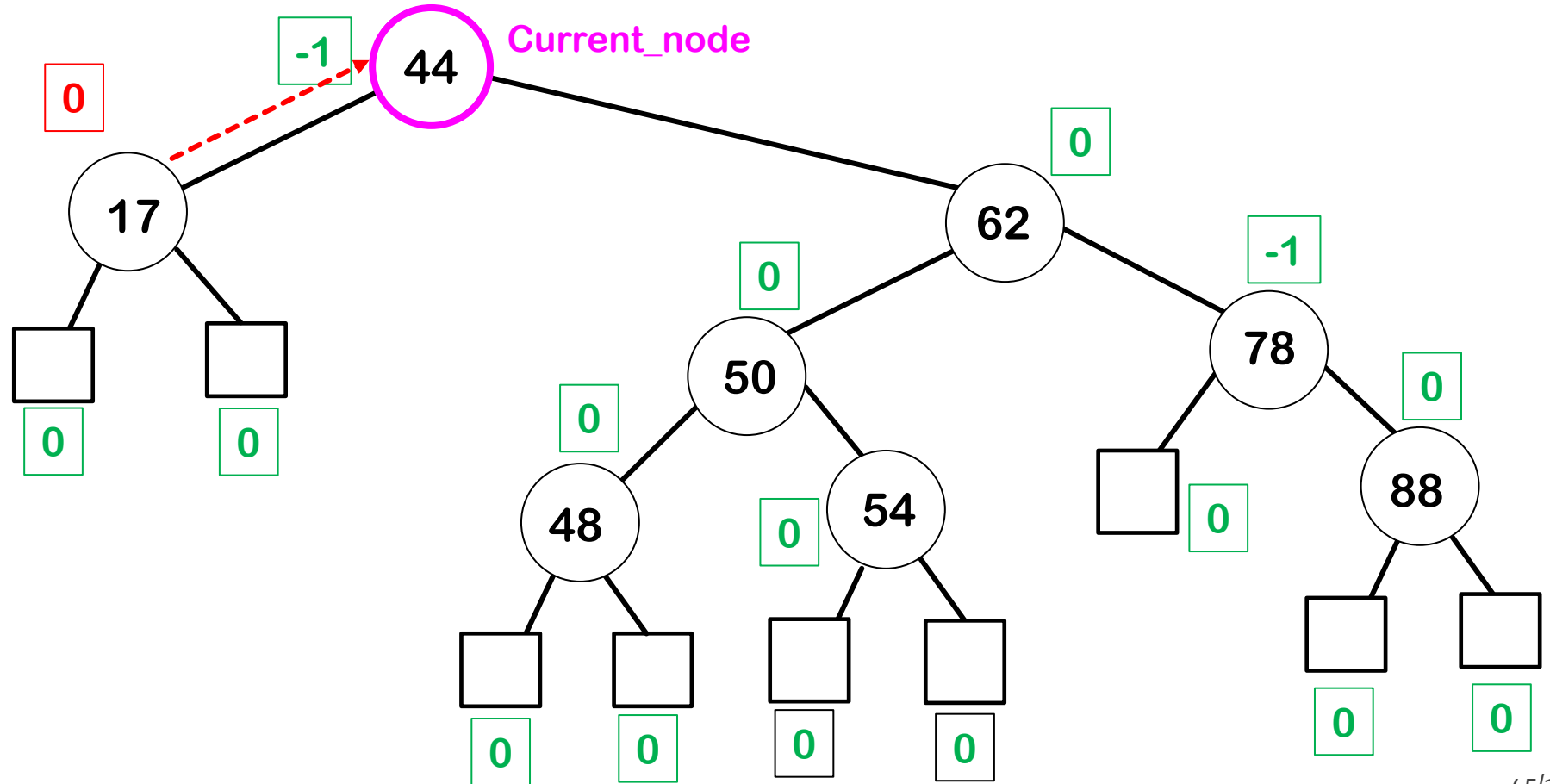
AVL: EXAMPLE

1. delete(32)
2. Update its BF



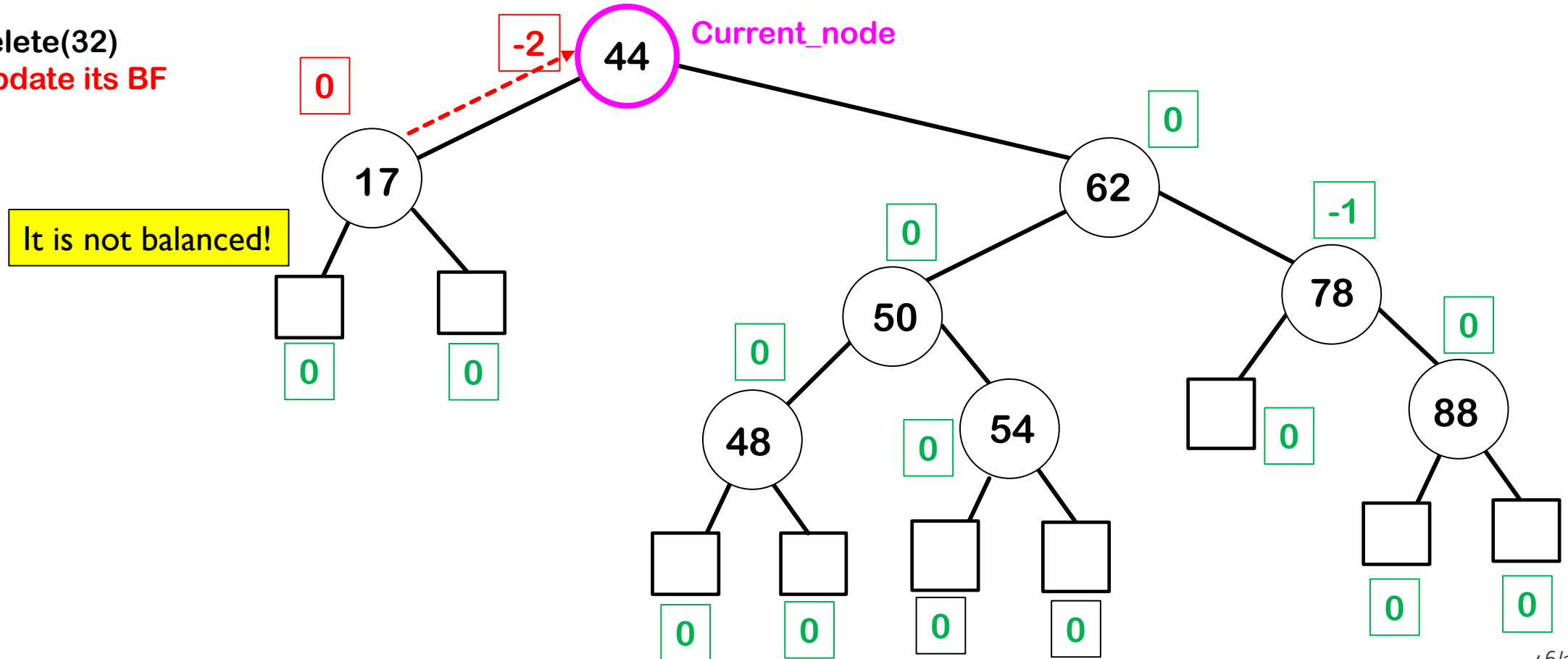
AVL: EXAMPLE

1. delete(32)
2. Update its BF



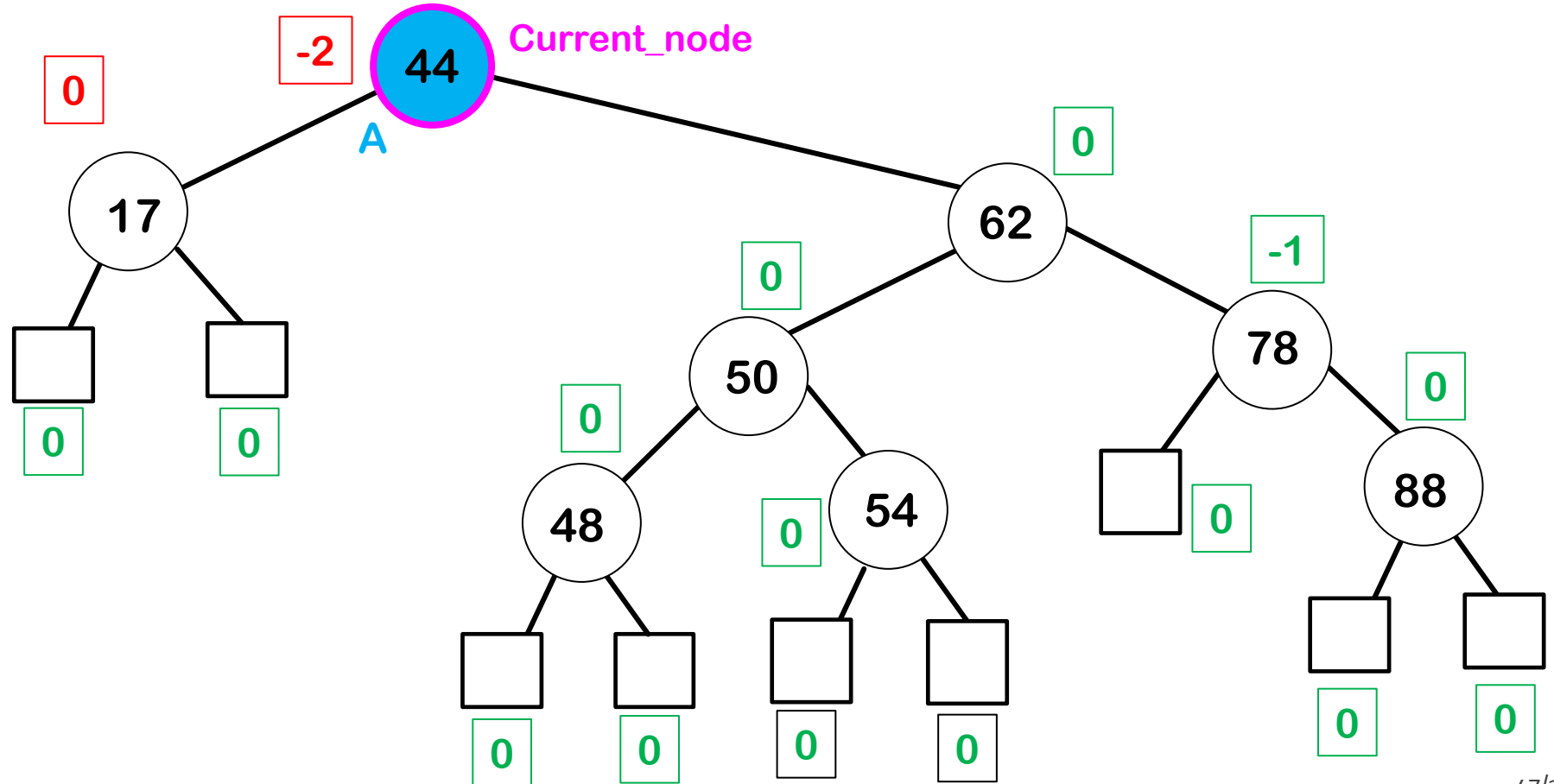
AVL: EXAMPLE

1. delete(32)
2. Update its BF



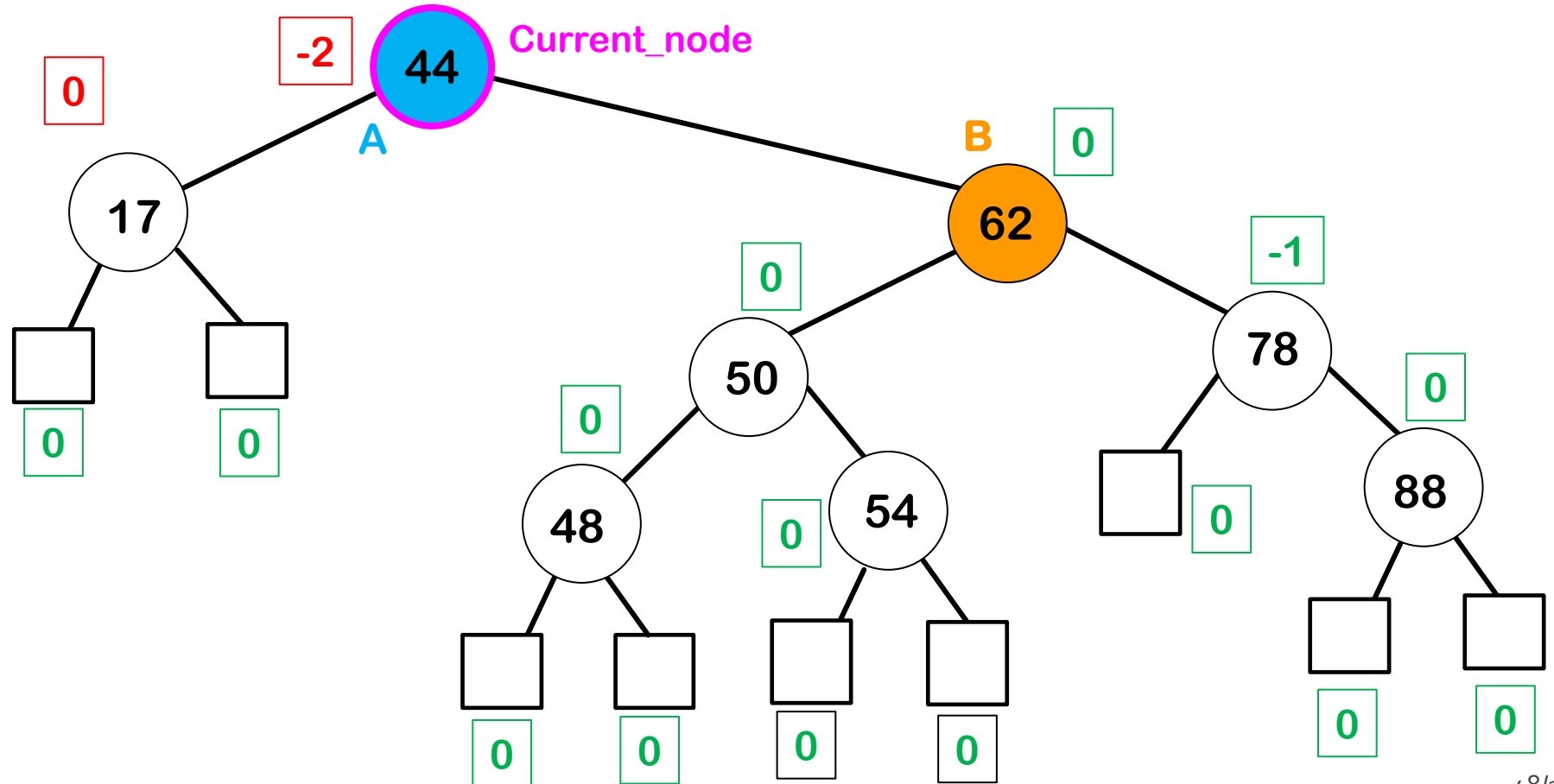
AVL: EXAMPLE

1. delete(32)
2. Update its BF
3. A= current_node



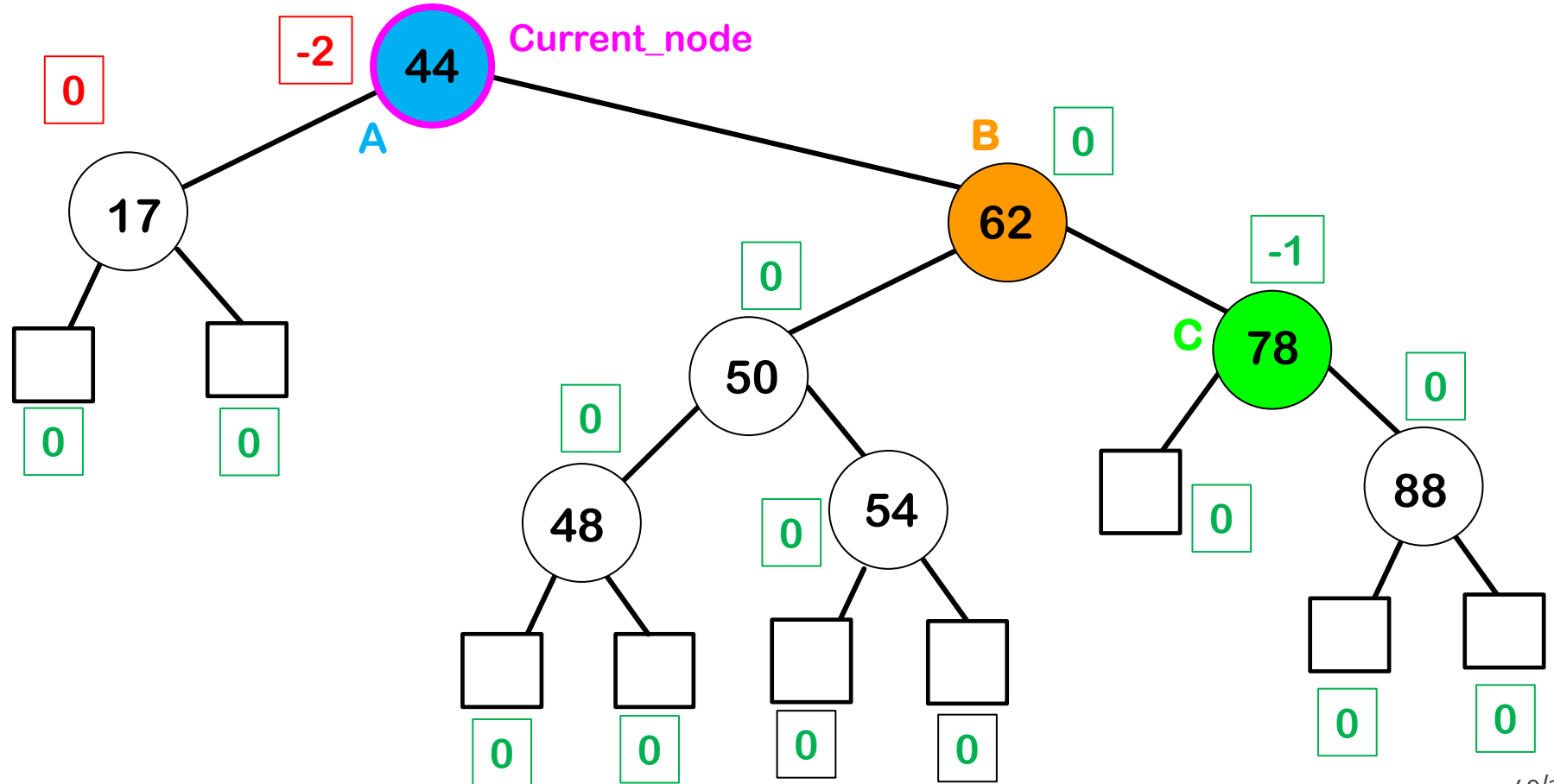
AVL: EXAMPLE

1. delete(32)
2. Update its BF
3. A= current_node
4. Determin B



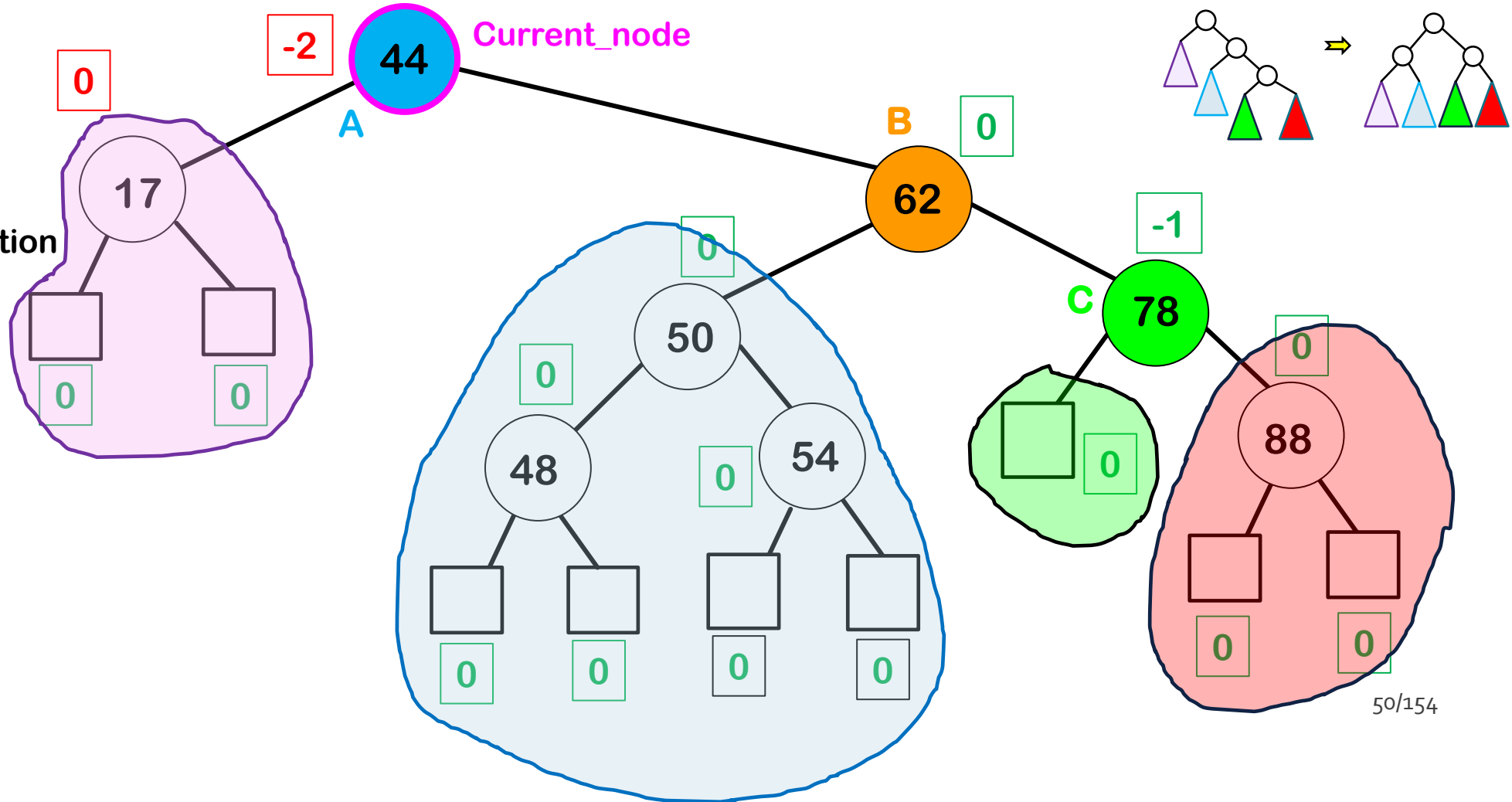
AVL: EXAMPLE

1. delete(32)
2. Update its BF
3. A= current_node
4. Determin B
5. Determine C



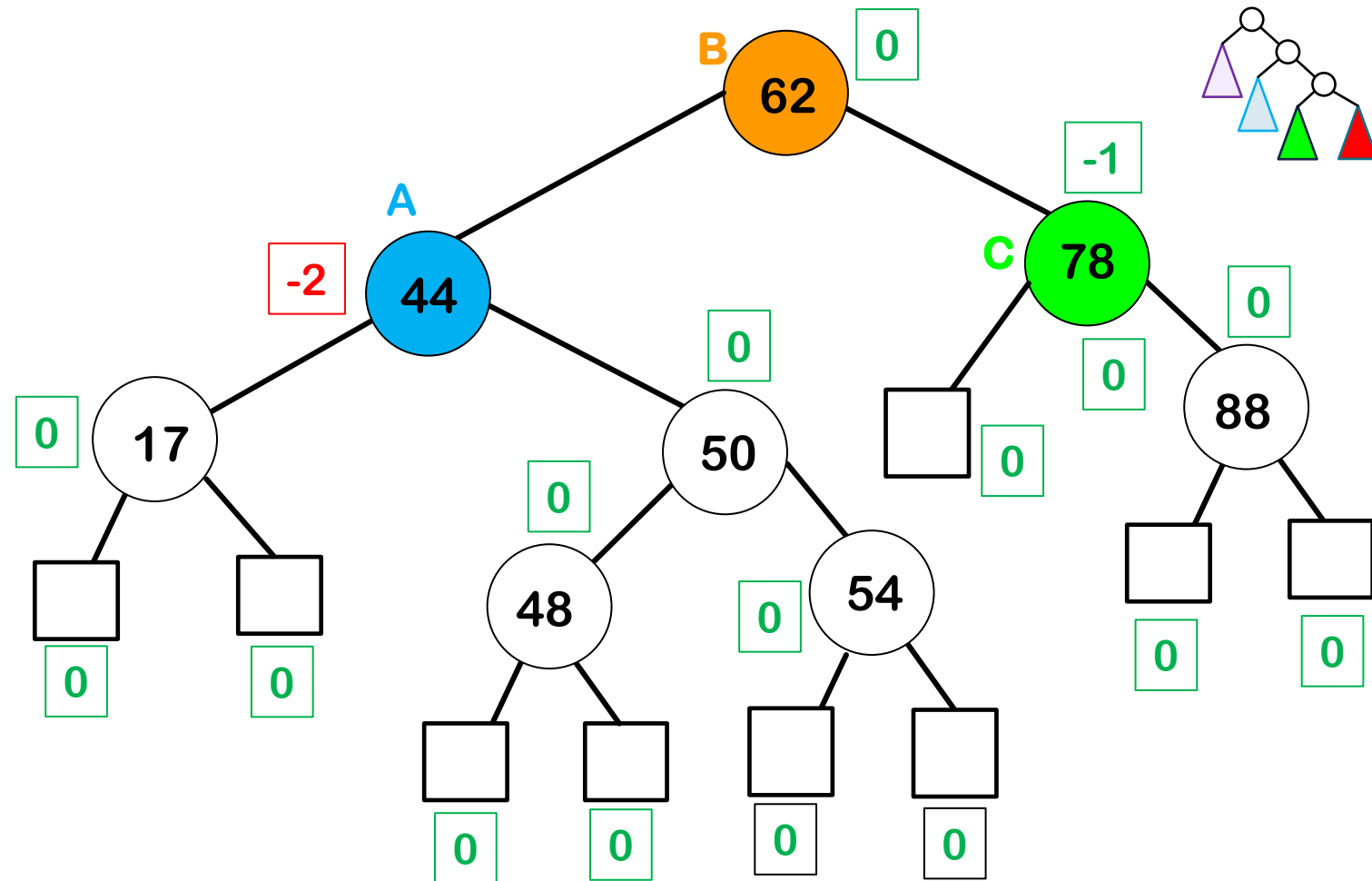
AVL: EXAMPLE

1. delete(32)
2. Update its BF
3. A= current_node
4. Determin B
5. Determine C
6. Determine the rotation



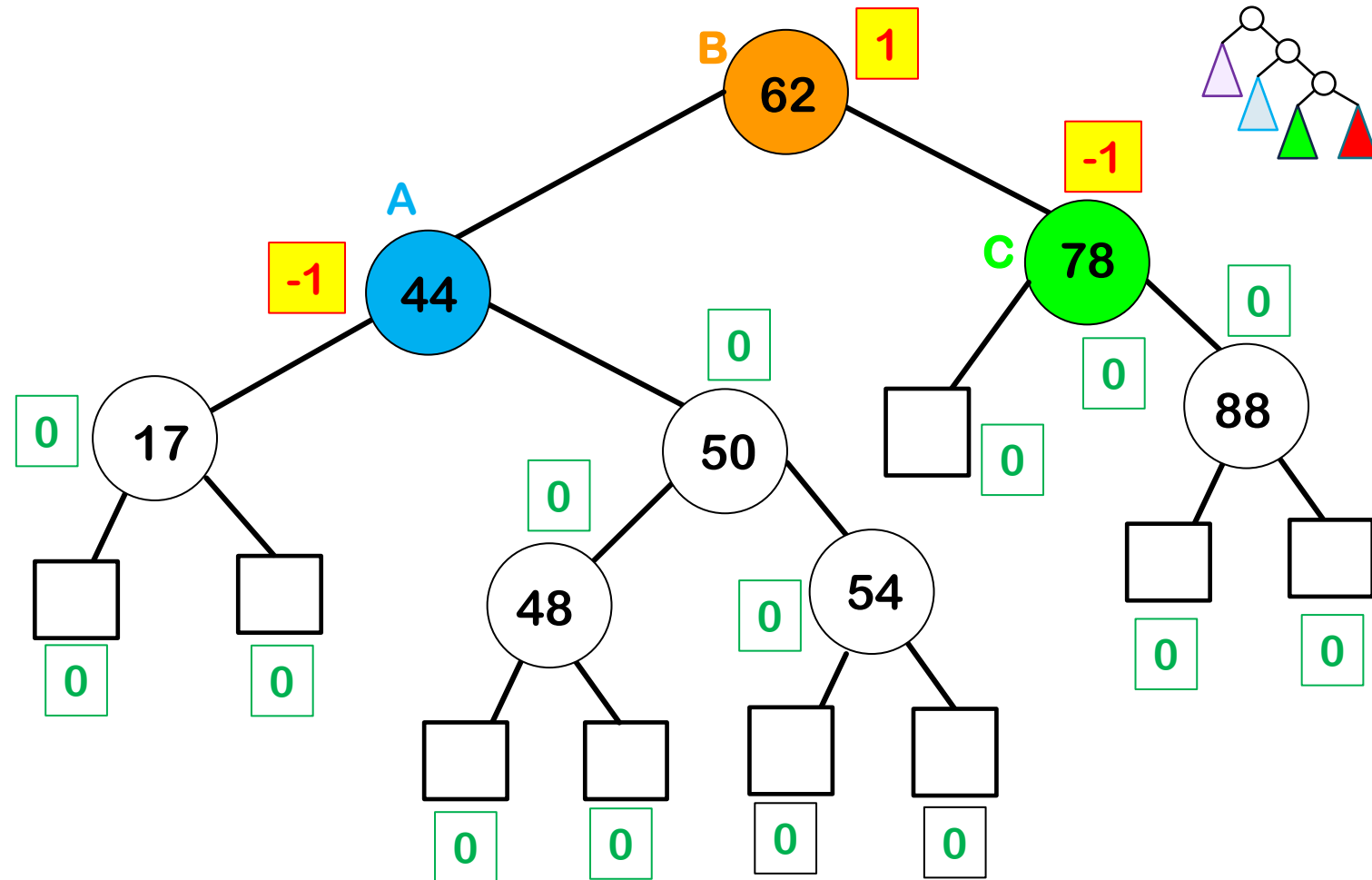
AVL: EXAMPLE

1. delete(32)
2. Update its BF
3. A= current_node
4. Determin B
5. Determine C
6. Perform the rotation



AVL: EXAMPLE

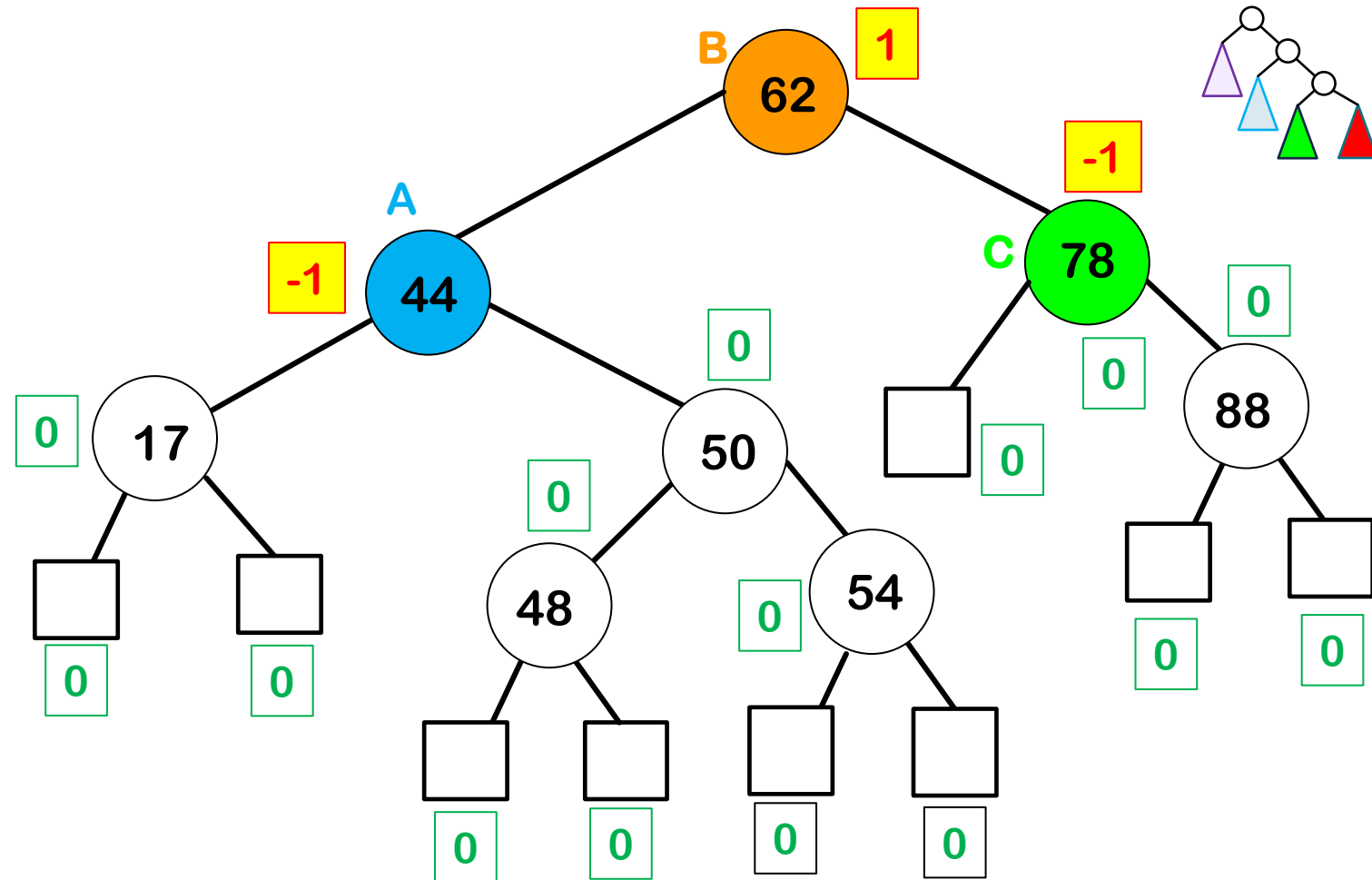
1. delete(32)
2. Update its BF
3. A= current_node
4. Determin B
5. Determine C
6. Perform the rotation
7. Update BF's of A,B,C



AVL: EXAMPLE

1. delete(32)
2. Update its BF
3. A= current_node
4. Determin B
5. Determine C
6. Perform the rotation
7. Update BF's of A,B,C
8. Repeat from parent of current_node until we reach NULL!

44 was root and its parent was Null, so we are done!



AVL: INSERT/ DELETE

AVL.Insert

1. Insert node x as usual.
2. On the path from x to the root, update the balance factors to find the first unbalance node (A).
3. Determine B and C and perform rotation.
4. Update the heights and balance factors of nodes A, B, and C. Continue updating the heights and balance factors on the path from node A up to the root.

AVL.Delete

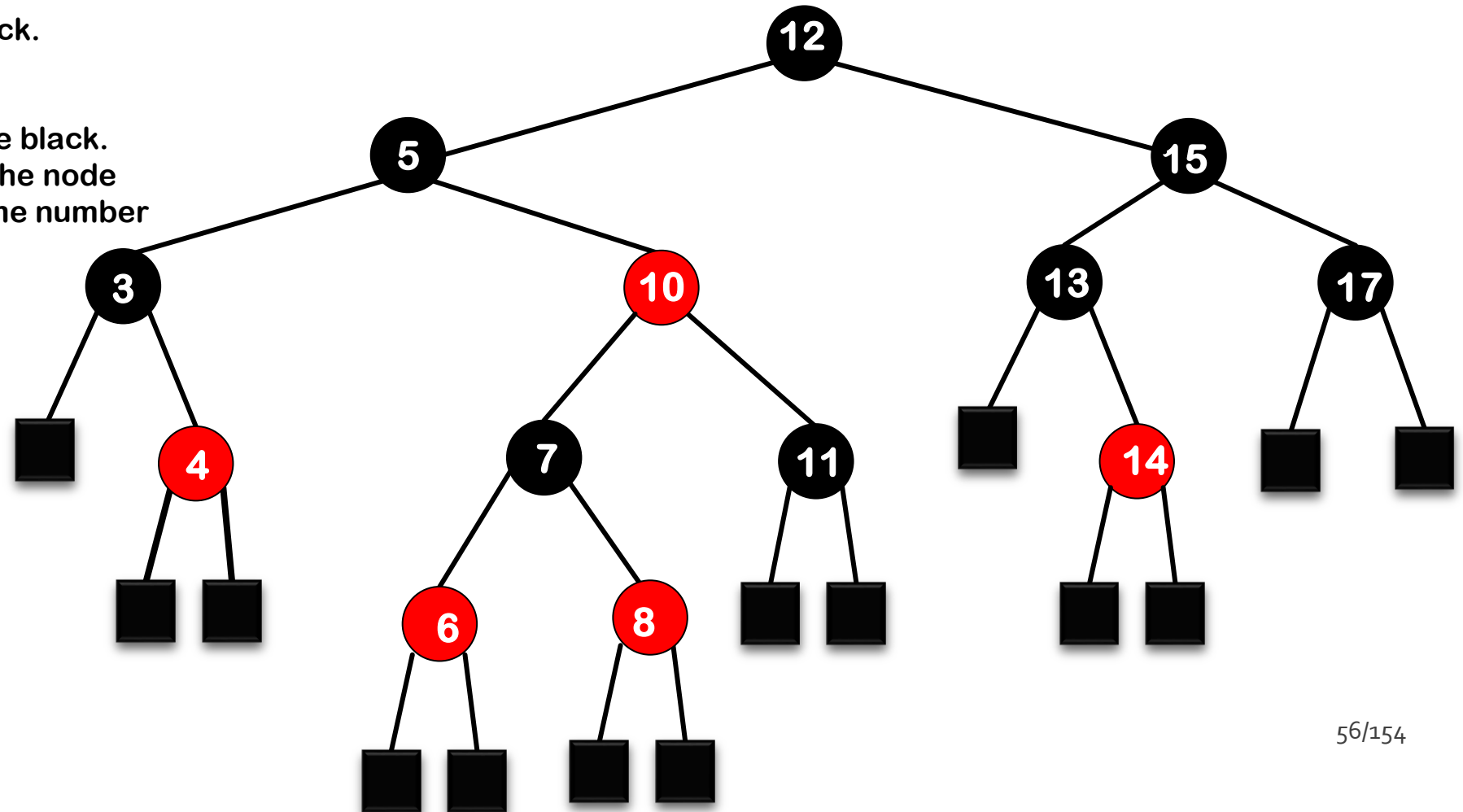
1. Delete node x as usual.
2. Set current_node = x.parent.
3. While current_node $\neq$ NULL:
 - a. Update height and balance factor of current_node.
 - b. If current_node is unbalanced:
Determine A,B,C and perform rotation
Update heights and balance factors after rotation.
Set current_node = current_node.parent.

RED-BLACK TREE

- Red-black tree is a **binary search tree** that satisfies the following properties:
 1. Every node is either red or black.
 2. The root is black
 3. Every external node is black.
 4. Both children of a red node are black.
 5. For each node, all paths from the node to its descendant leaves (nil nodes) contain the same number of black nodes.

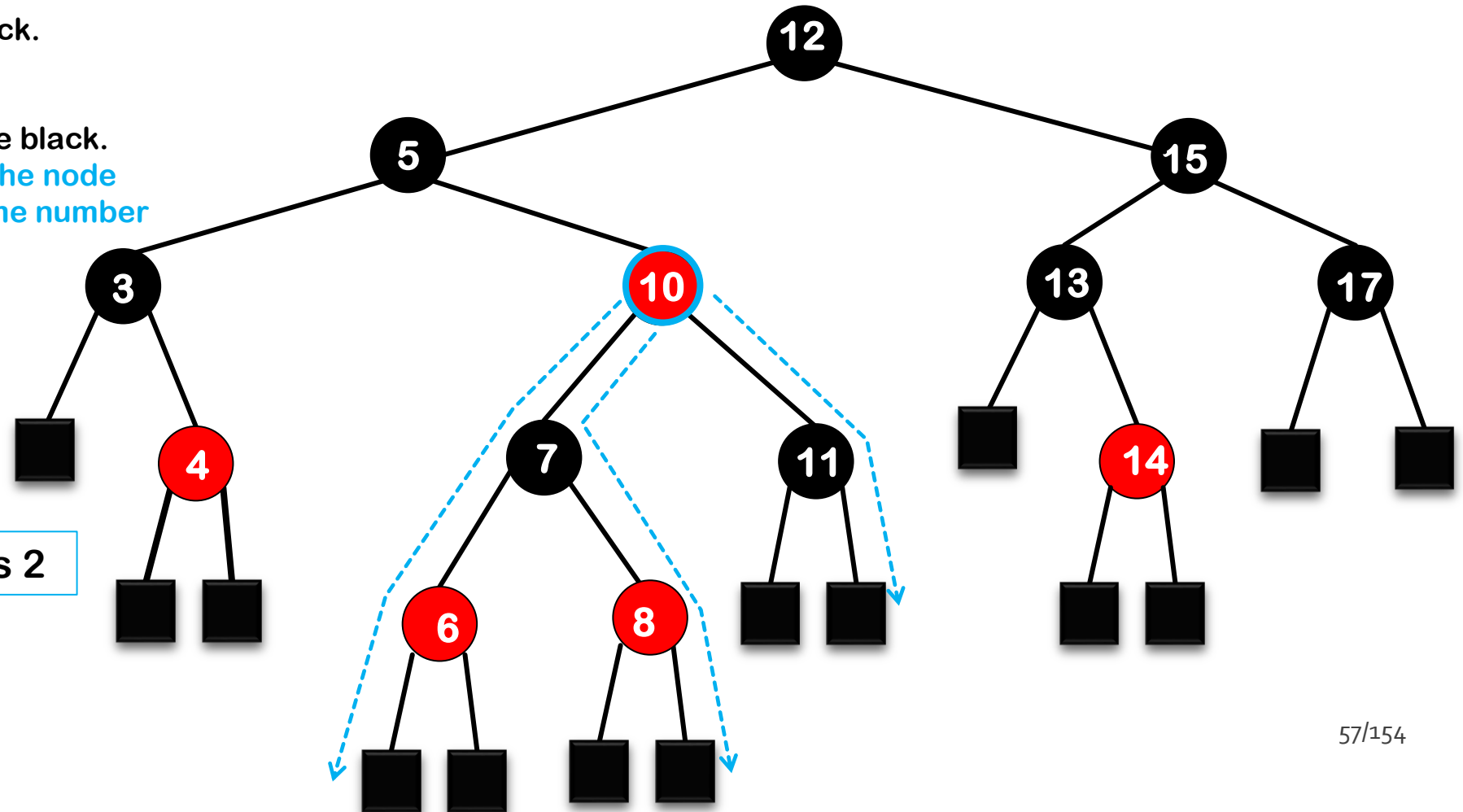
RED-BLACK TREE

1. Every node is either red or black.
2. The root is black
3. Every external node is black.
4. Both children of a red node are black.
5. For each node, all paths from the node to its nil nodes contain the same number of black nodes.



RED-BLACK TREE

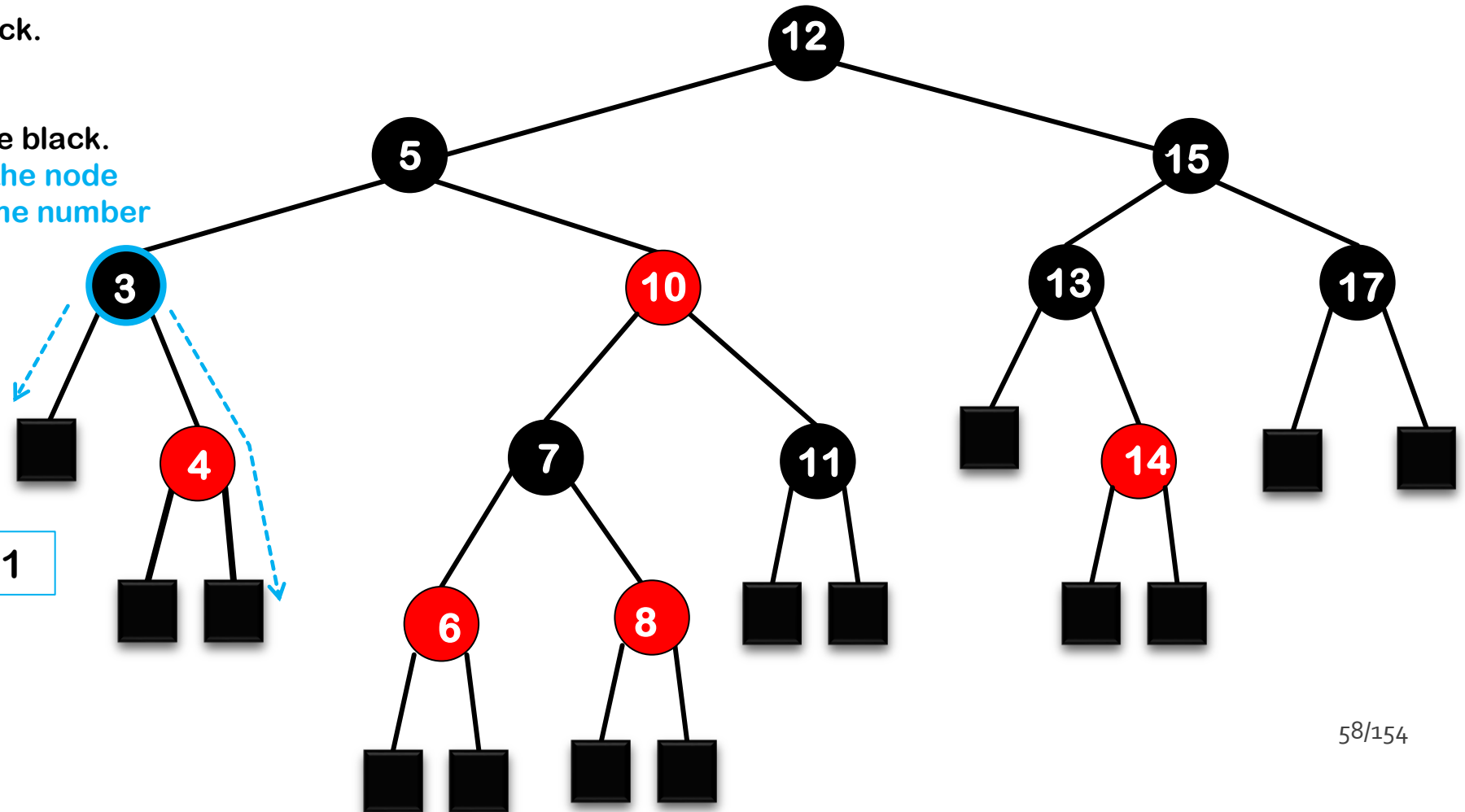
1. Every node is either red or black.
2. The root is black
3. Every external node is black.
4. Both children of a red node are black.
5. For each node, all paths from the node to its nil nodes contain the same number of black nodes.



Black height of node 10 is 2

RED-BLACK TREE

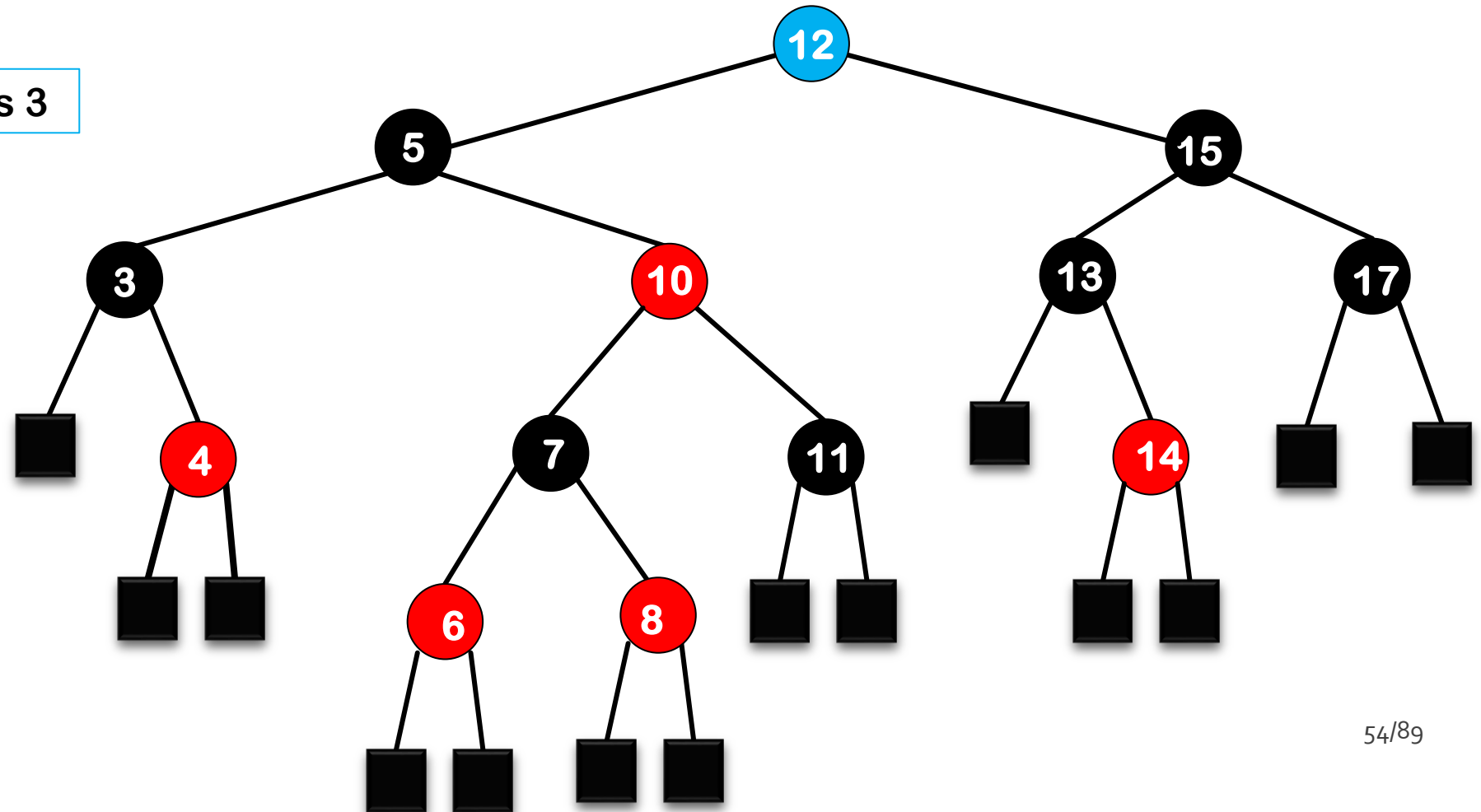
1. Every node is either red or black.
2. The root is black
3. Every external node is black.
4. Both children of a red node are black.
5. For each node, all paths from the node to its nil nodes contain the same number of black nodes.



Black height of node 3 is 1

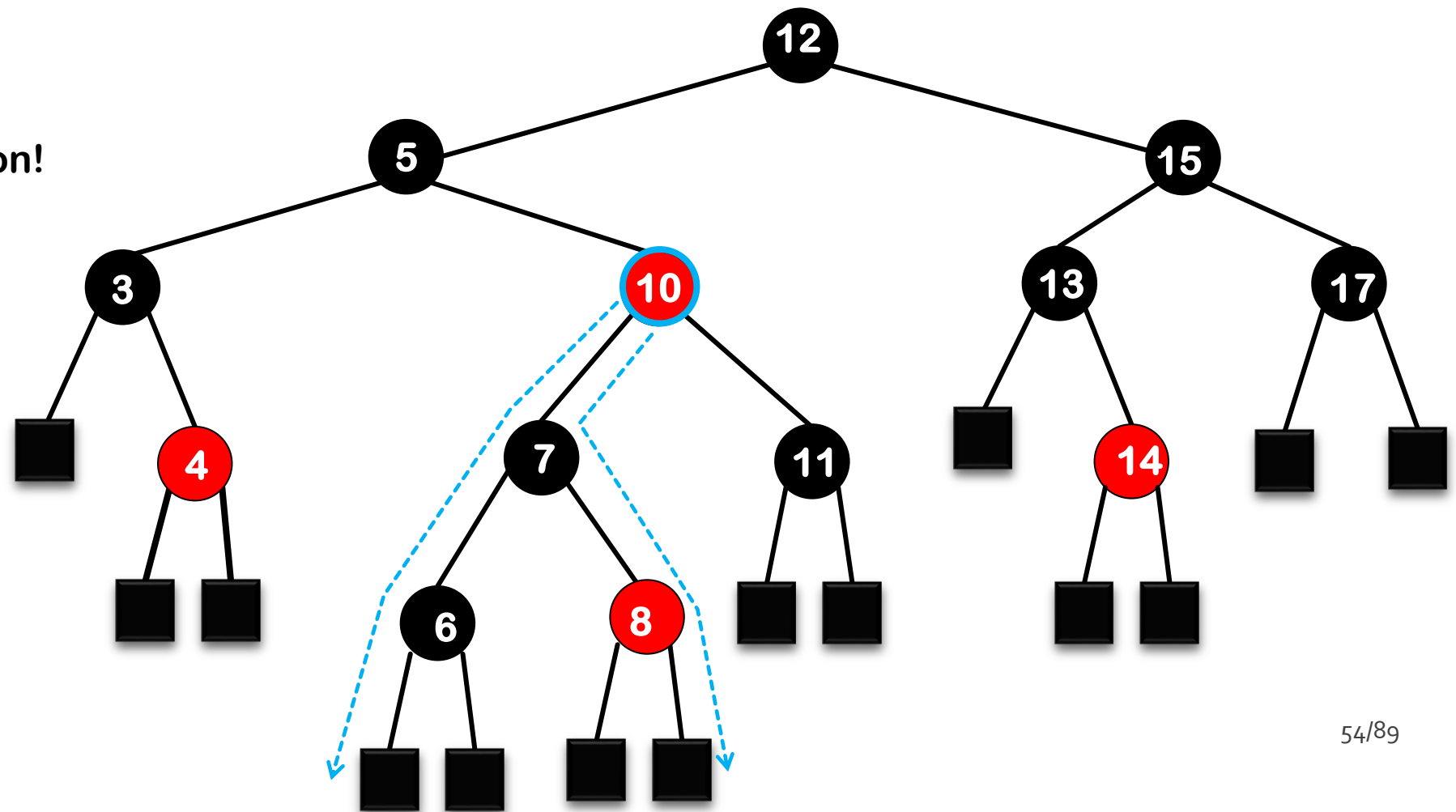
BLACK HEIGHT OF RBT

Black height of root is 3



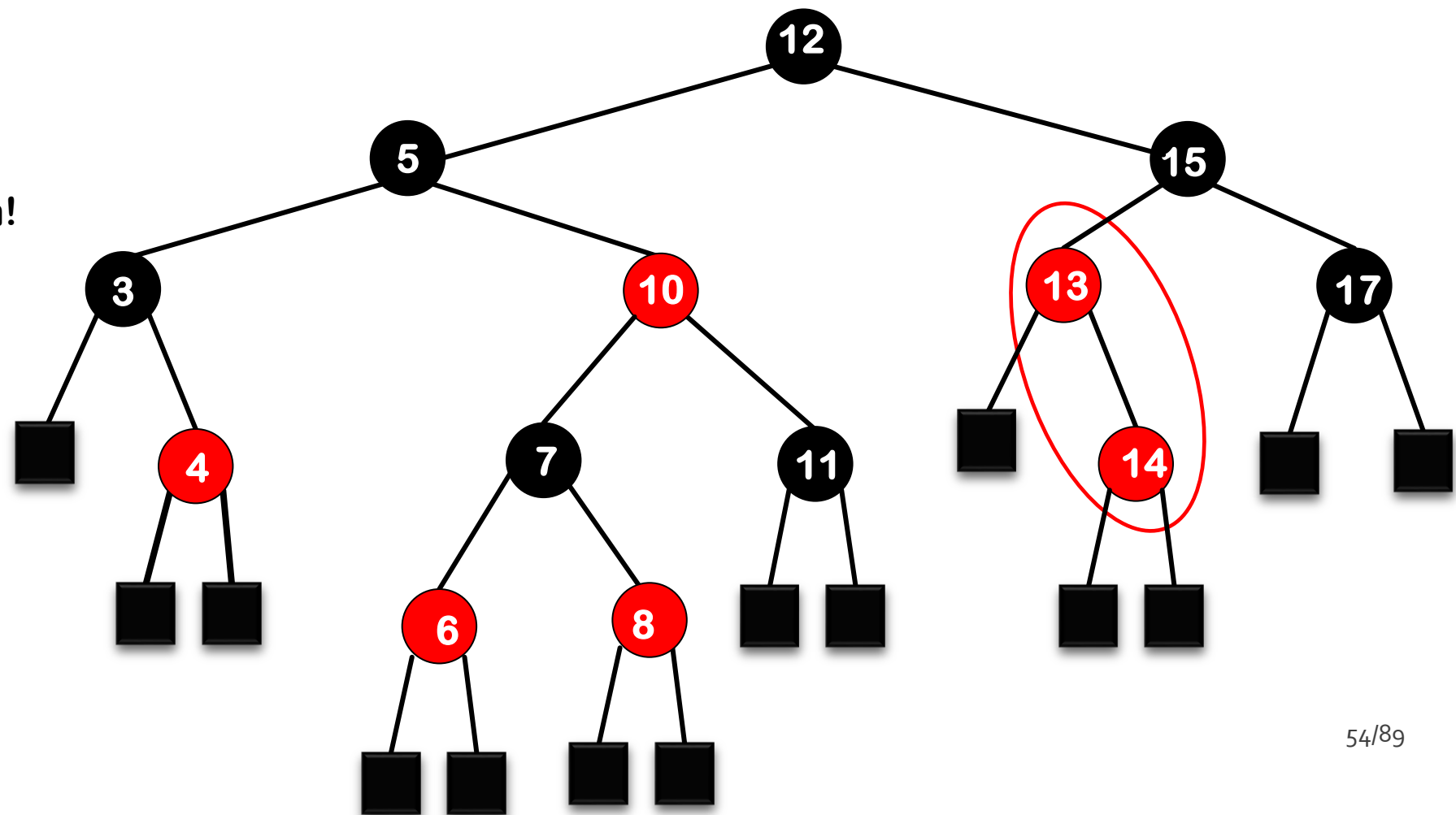
VIOLATIONS!

“Black height” violation!



VIOLATIONS!

“Double red” violation!



RED-BLACK TREE

The height of a red-black tree storing n items is $O(\log n)$.

INSERTION

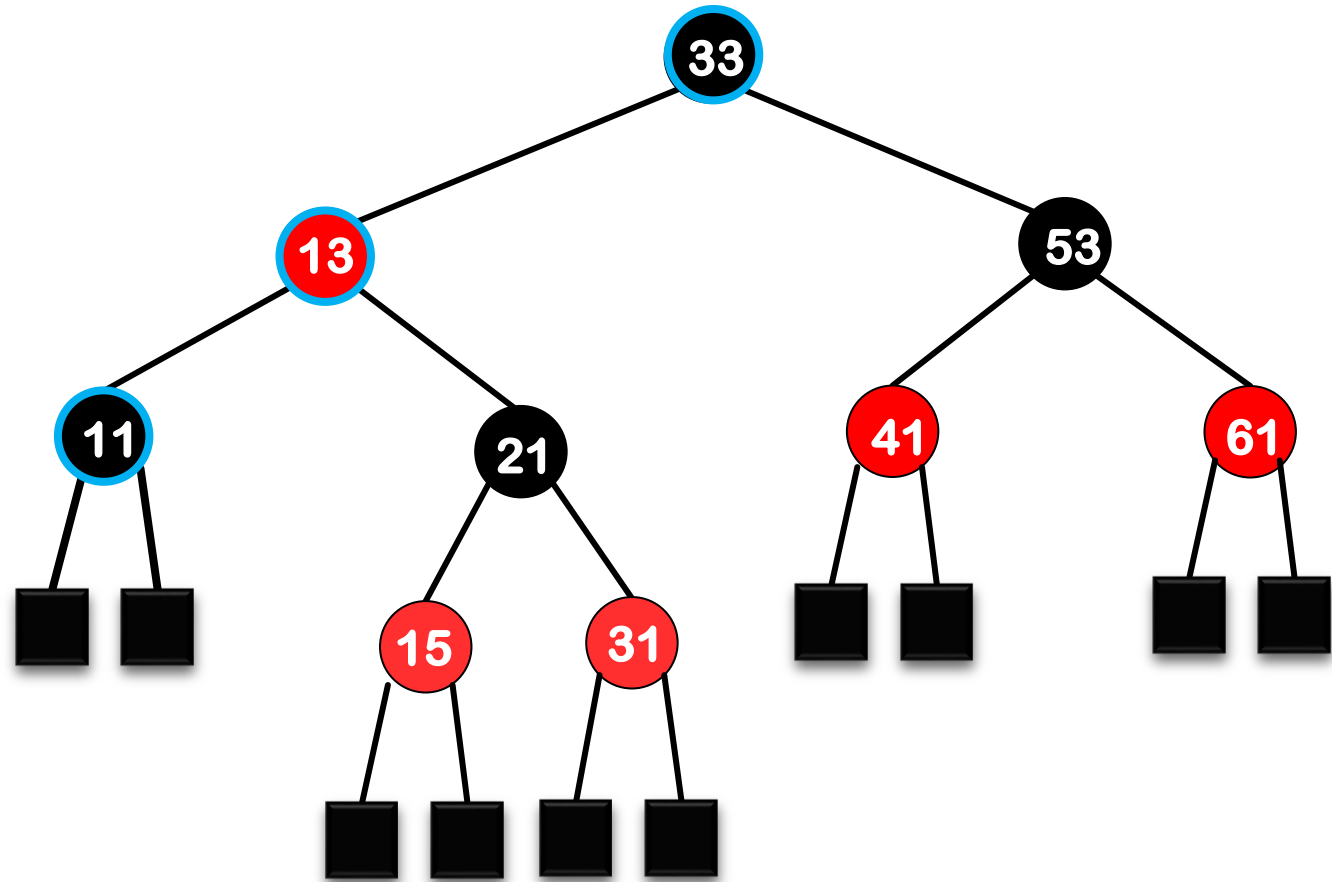
1. Insert the new key as usual
2. Paint the new node.
3. If tree is not a RBT, perform **rotations and/or recolorings!**

INSERTION

- Insert the new key as usual
- If the new node is the root, color it **black** and we are done! Otherwise color it **red**.
- If its parent is **black**, nothing else needs to be done, we are done!
- Otherwise, that means a “double red” violation of red-black tree rules, which needs to be fixed.

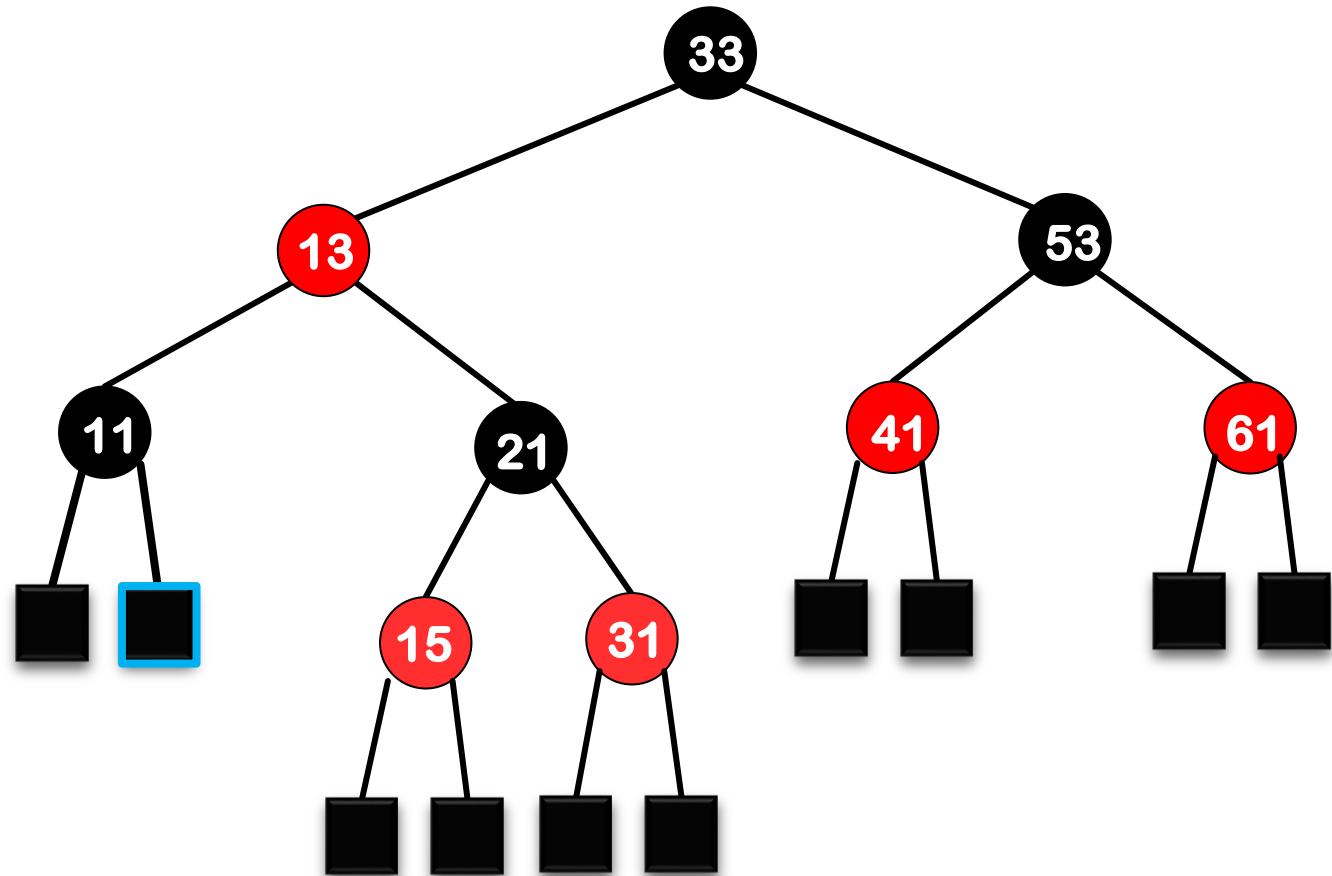
INSERTION: EXAMPLE

Insert 12



INSERTION: EXAMPLE

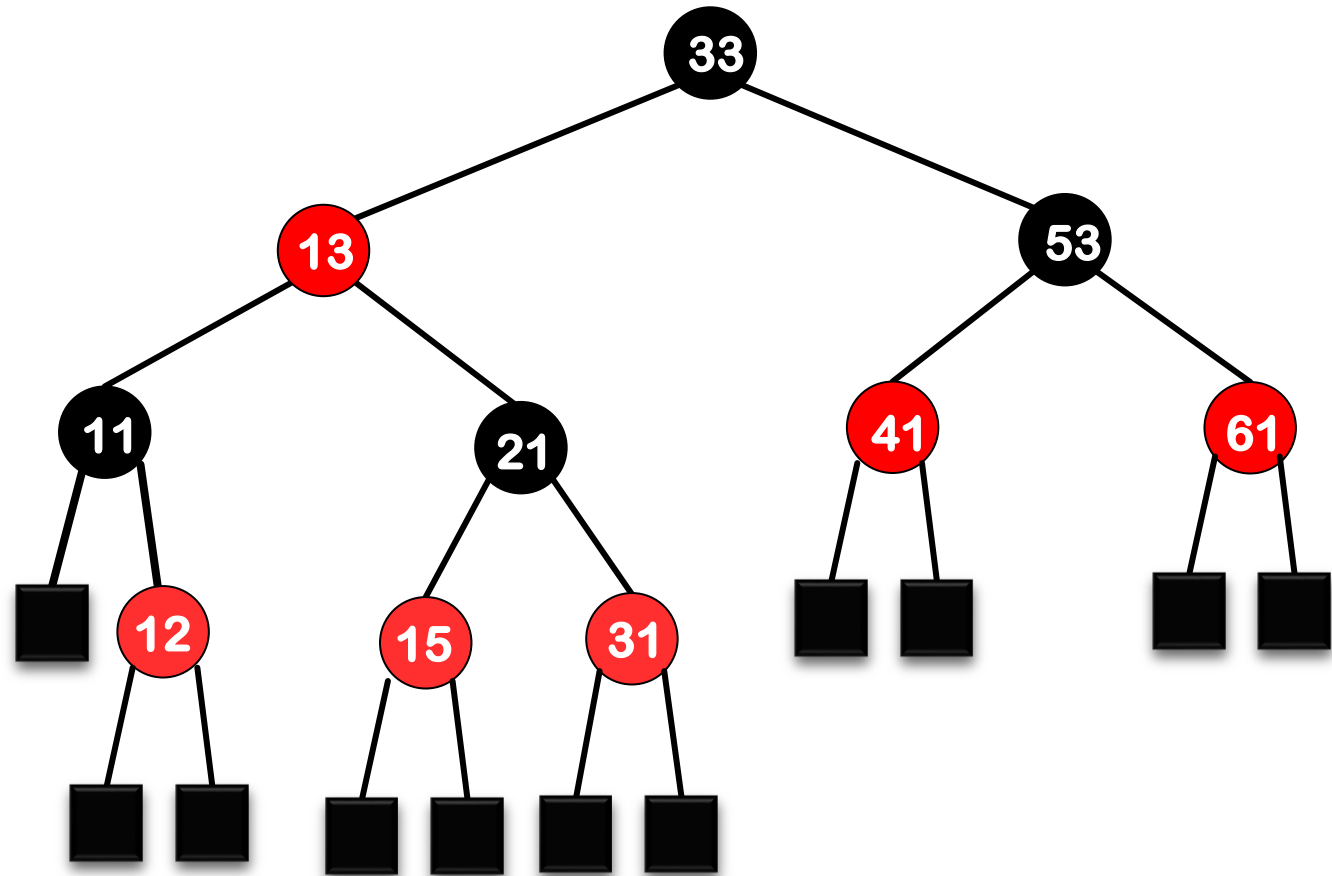
Insert 12



INSERTION: EXAMPLE

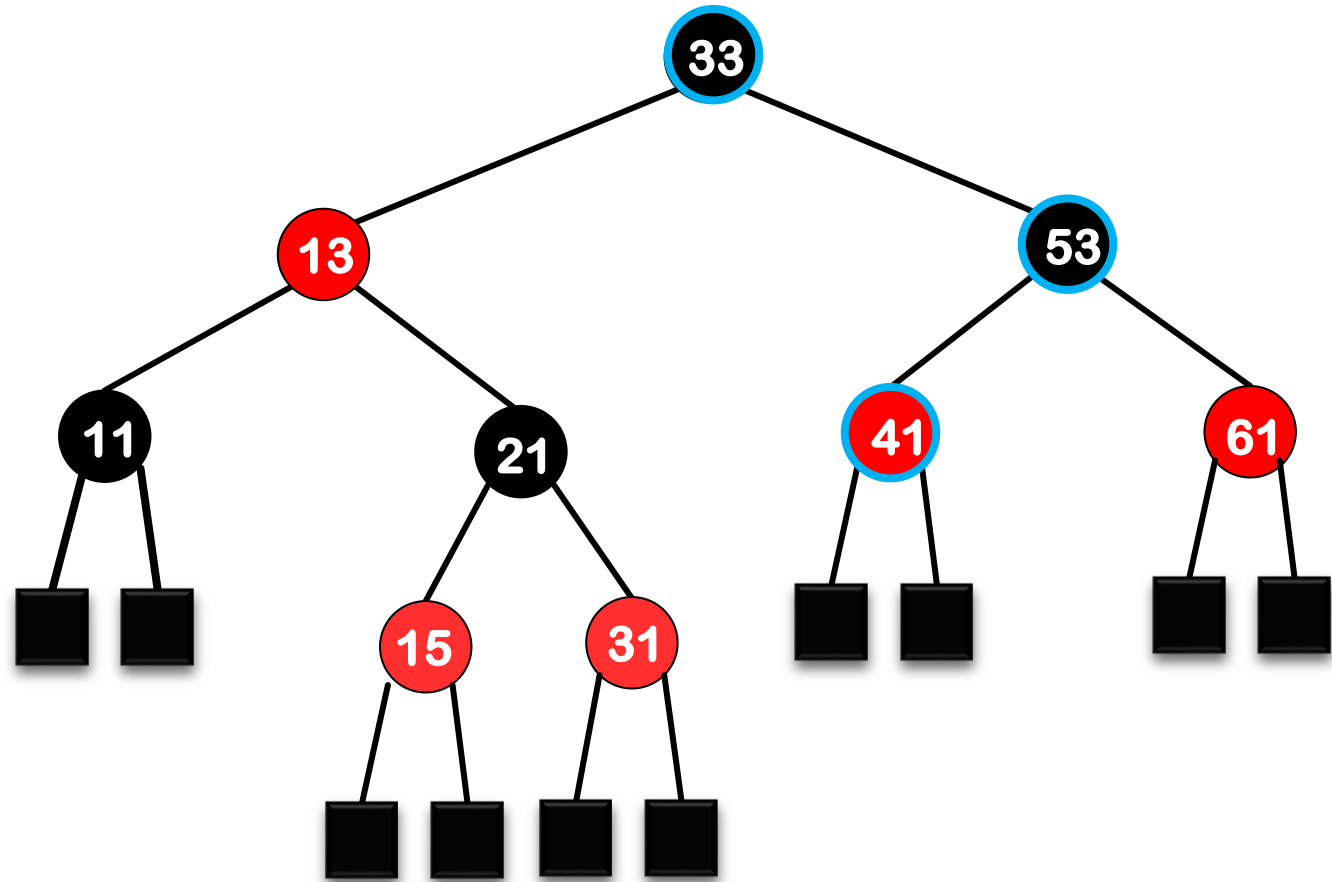
Insert 12

If its parent is black, we are done.



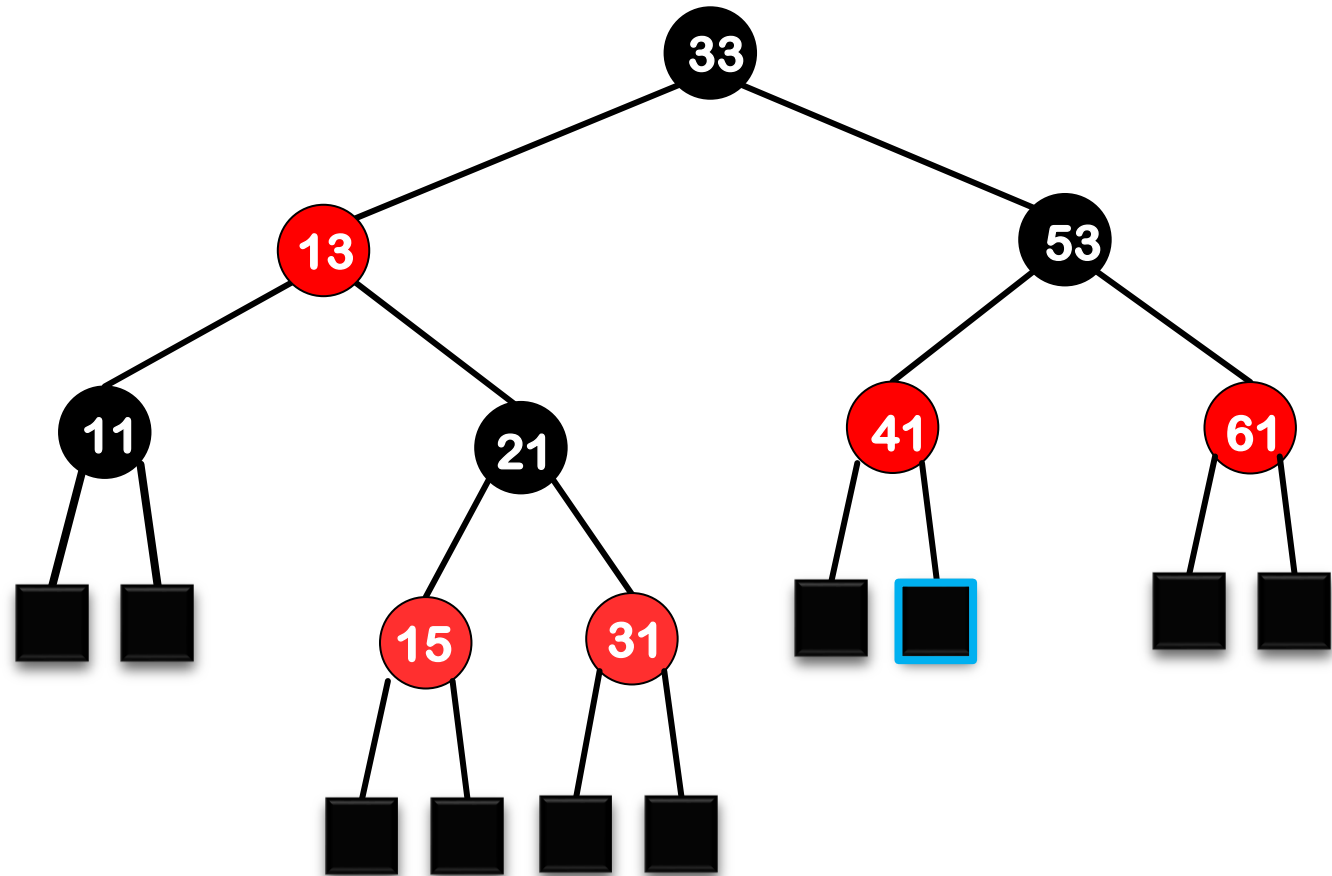
INSERTION: EXAMPLE

Insert 50



INSERTION: EXAMPLE

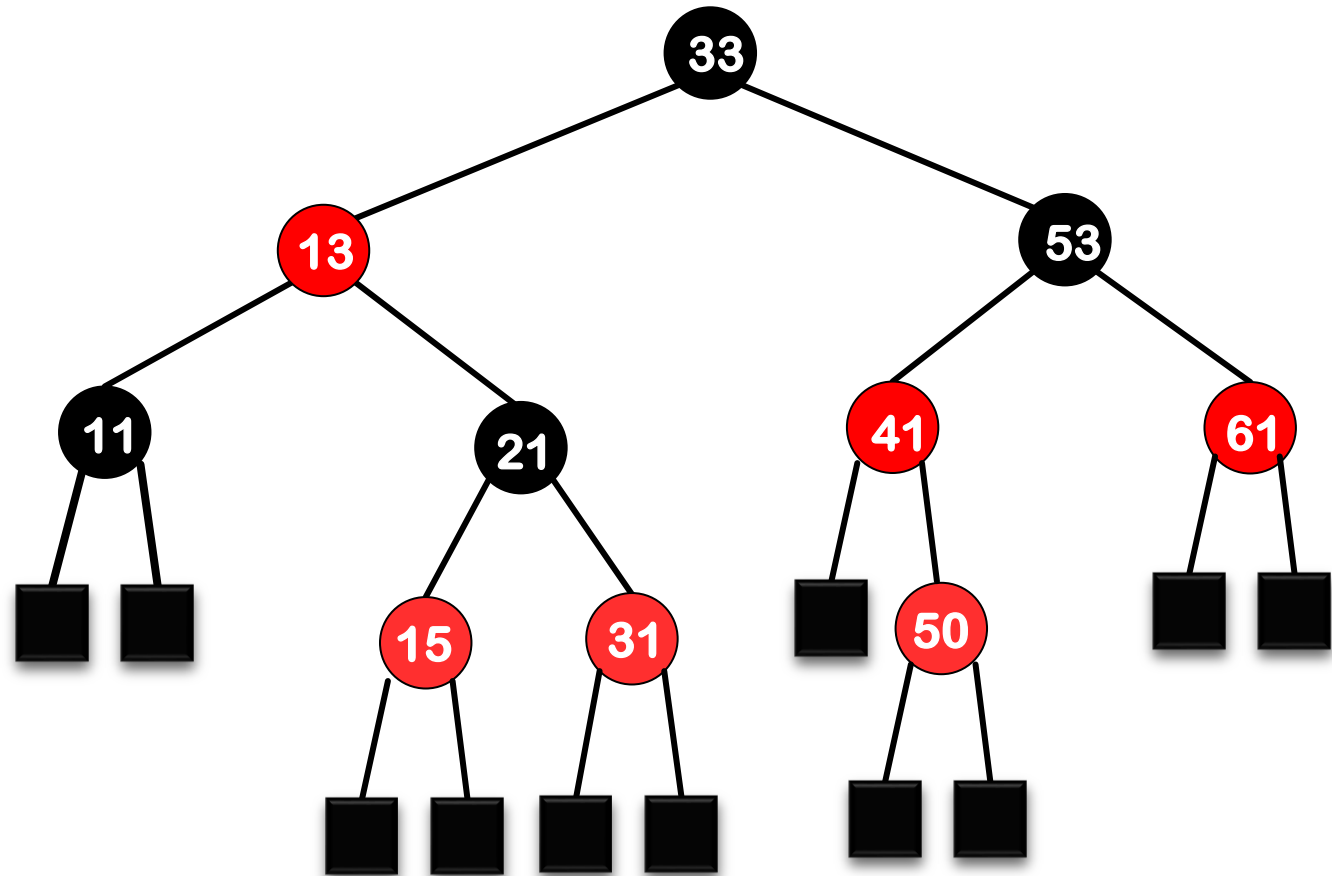
Insert 50



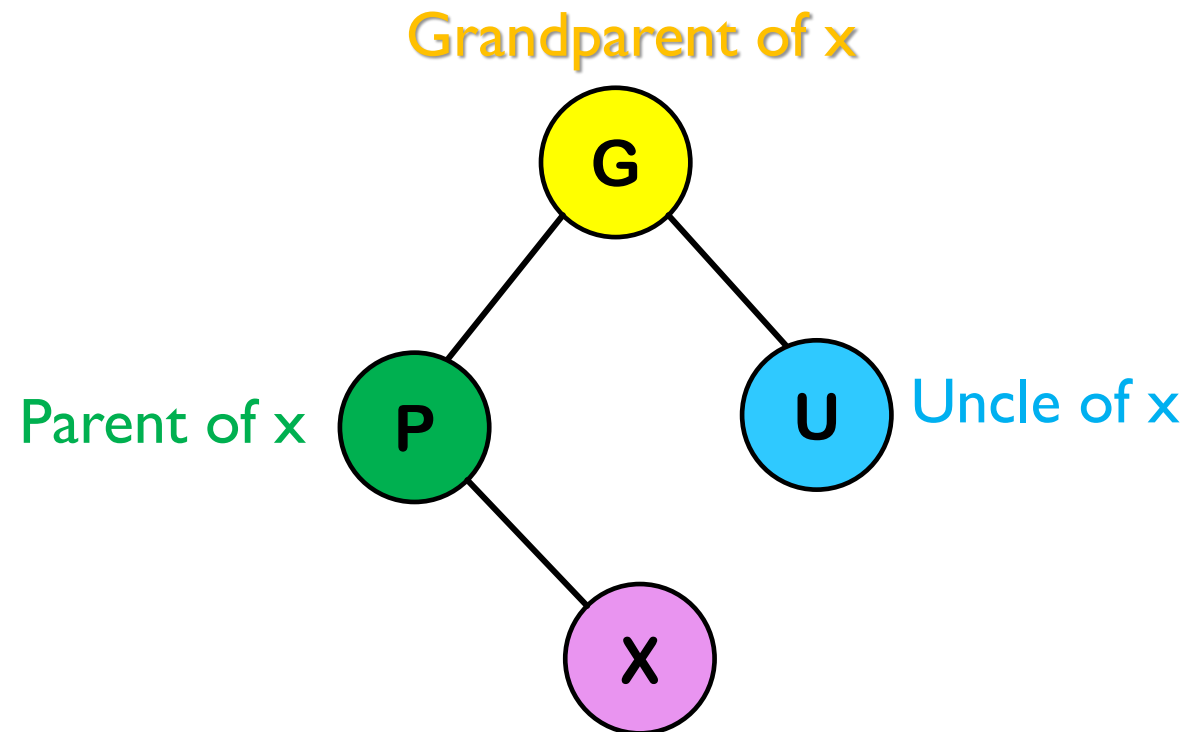
INSERTION: EXAMPLE

Insert 50

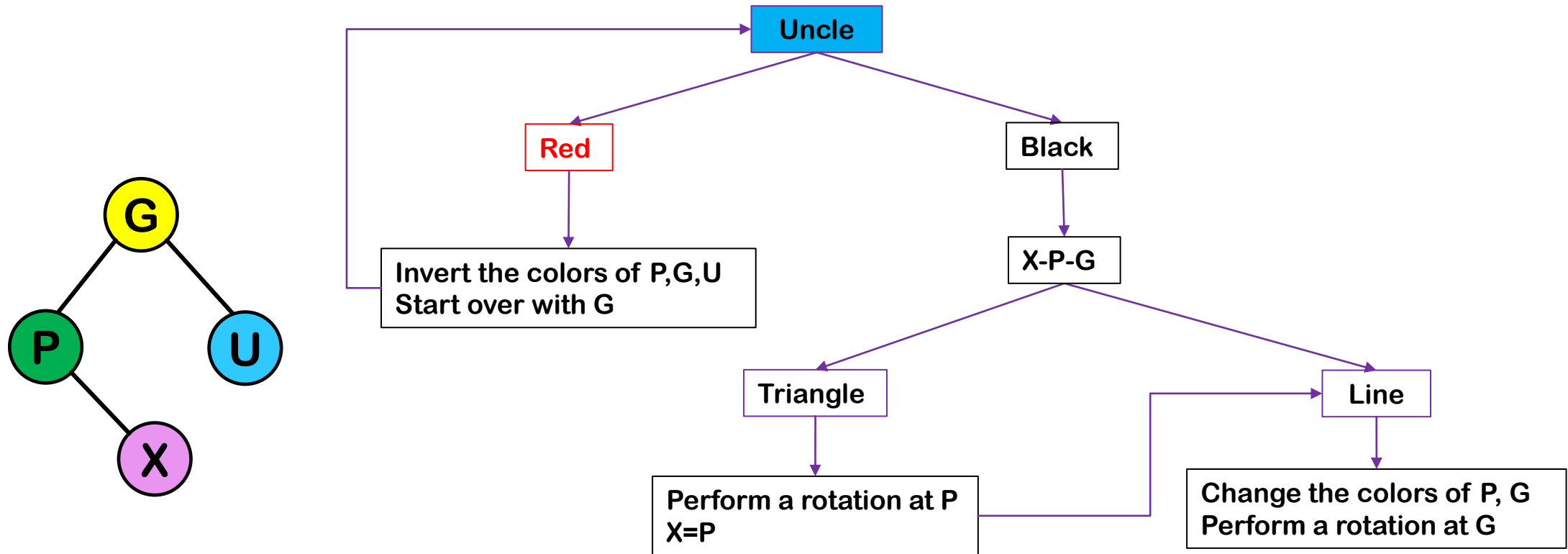
If its parent is red, rebalance!



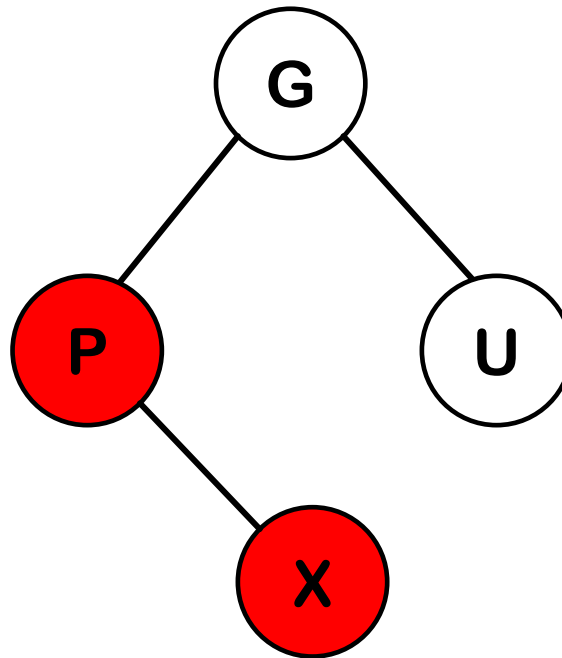
REBALANCING RBT AFTER INSERTION



INSERT

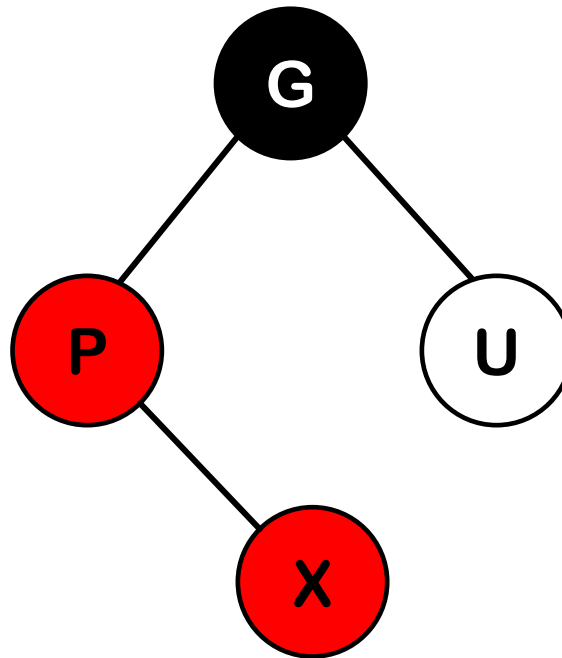


REBALANCING RBT AFTER INSERTION

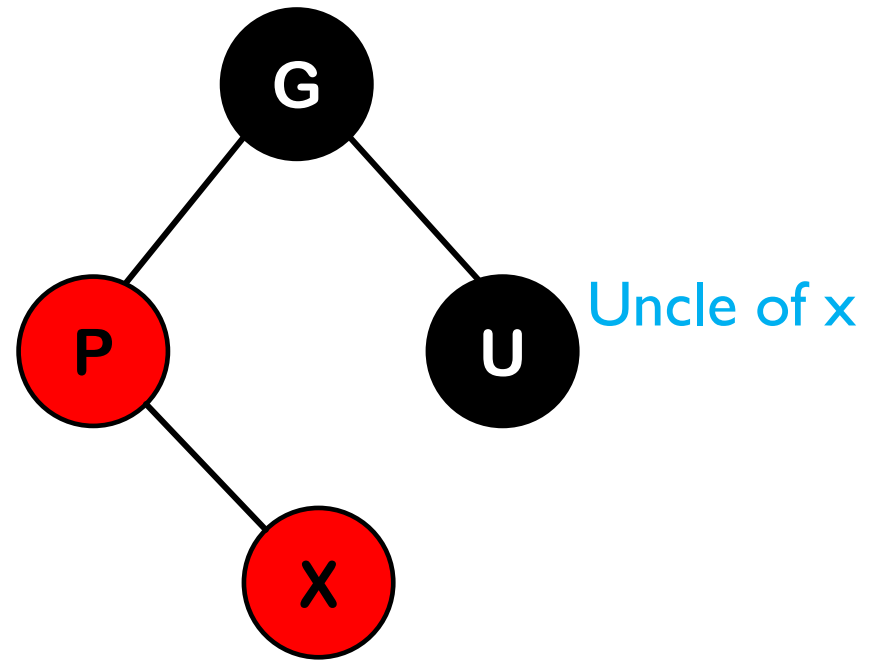
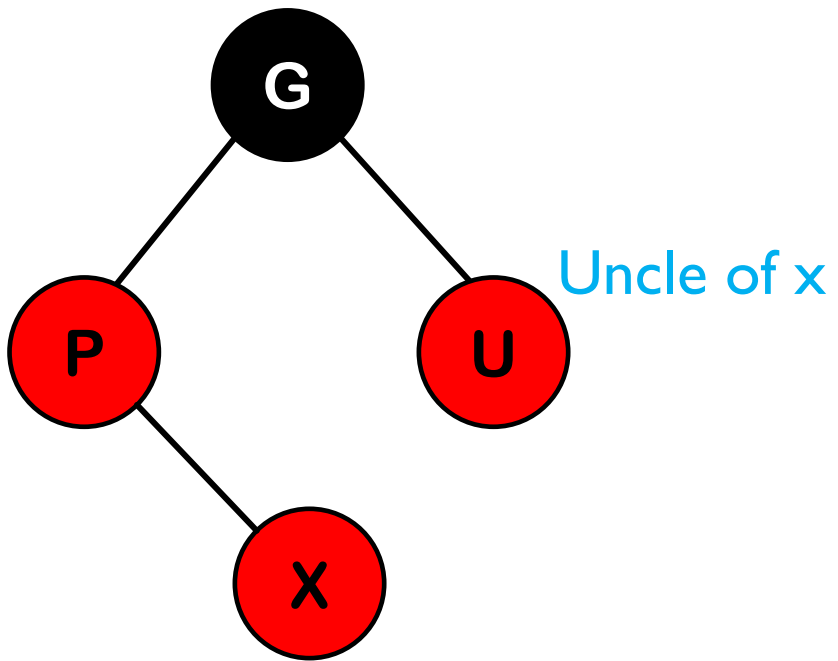


What color is the grandparent?

REBALANCING RBT AFTER INSERTION

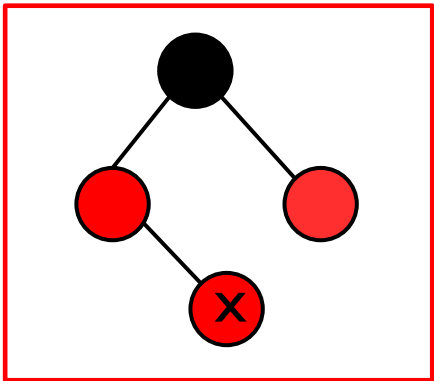


REBALANCING RBT AFTER INSERTION:TWO CASES

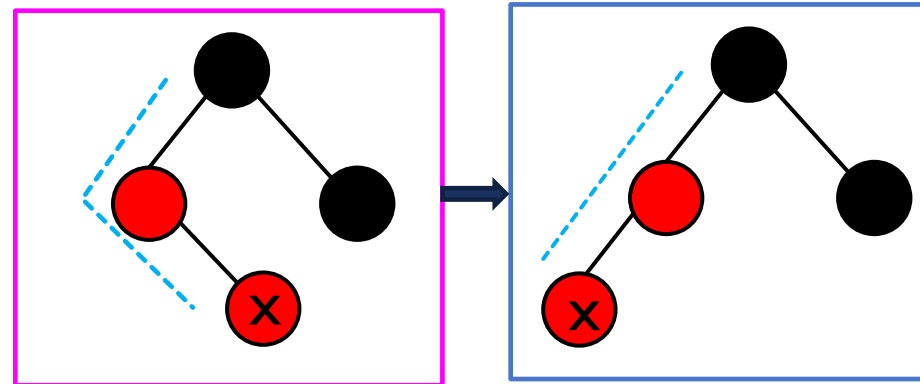


REBALANCING

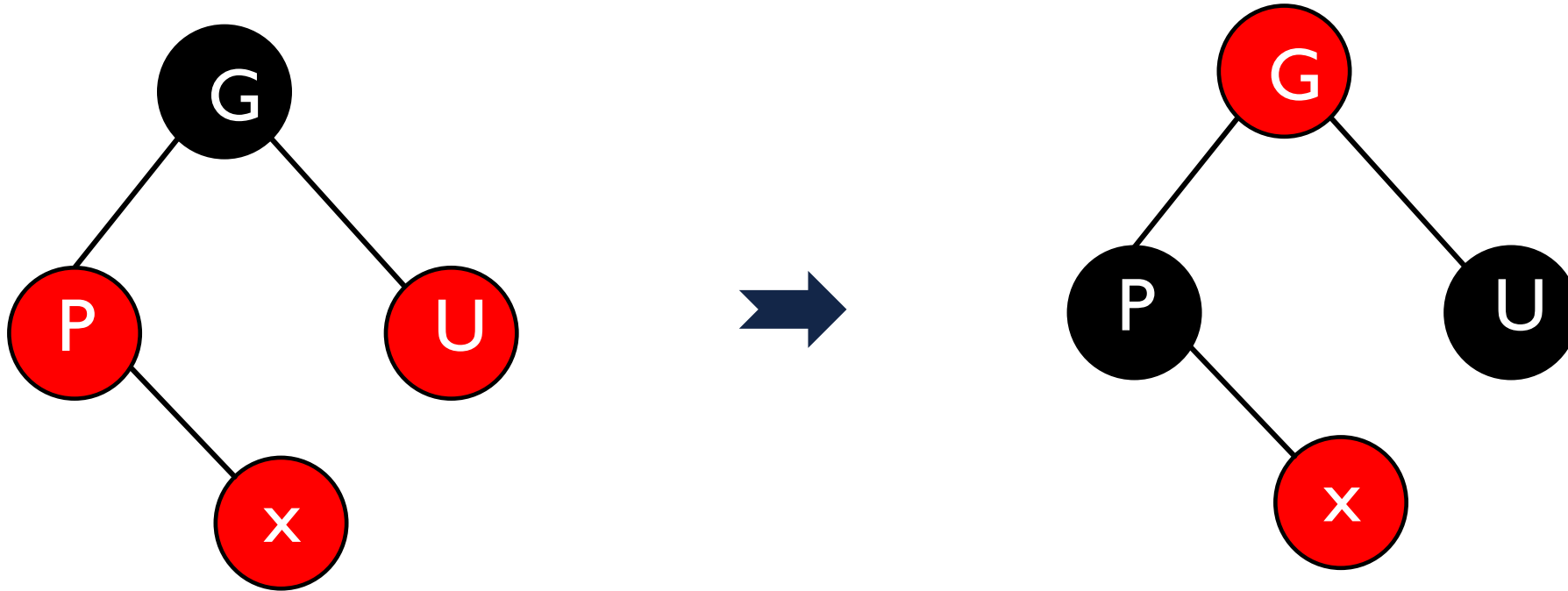
1. Uncle of x is red



2. Uncle of x is black

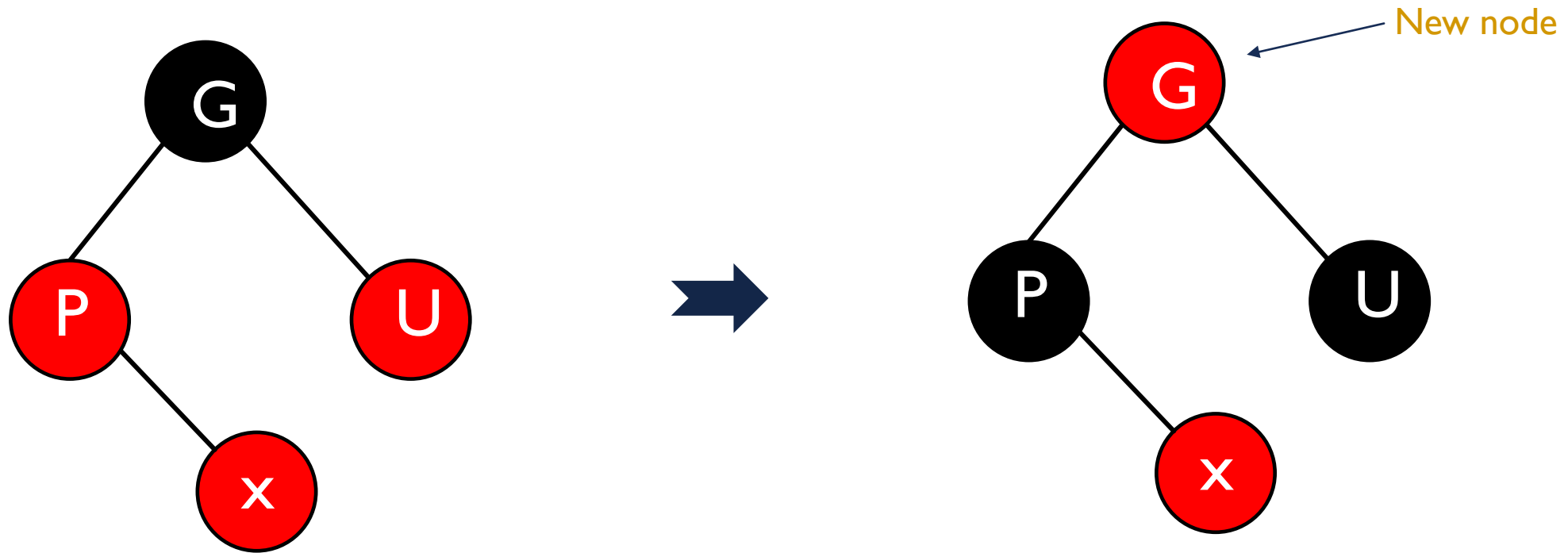


UNCLE IS RED



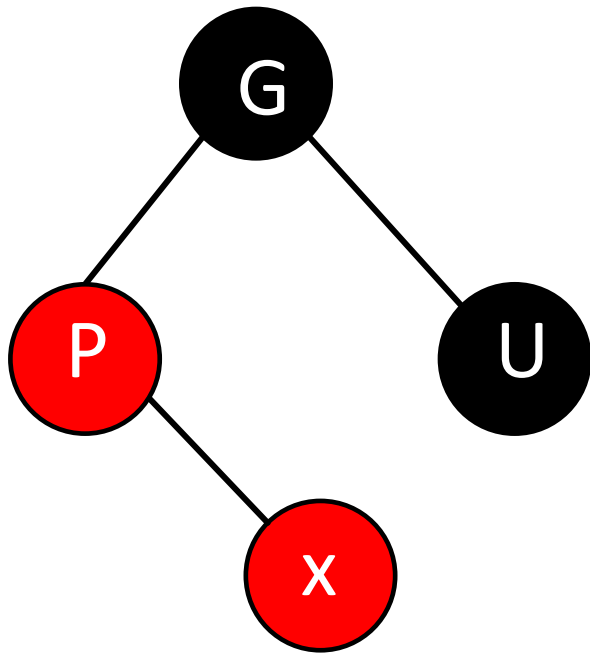
1. Change the colors of P,G,U: Black height does not change.
2. Step 1 may form double-red if parent of G is red. So, we start over with G as a newly-red node.

UNCLE IS RED



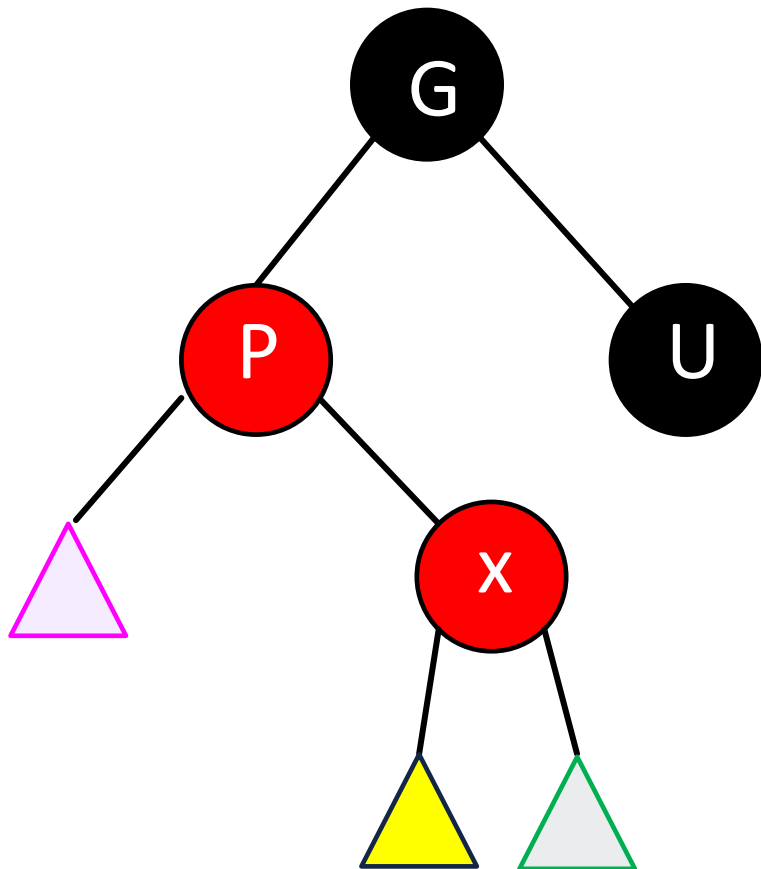
1. Change the colors of P,G,U: Black height does not change.
2. Step 1 may form double-red if parent of G is red. So, we start over with G as a newly-red node.

UNCLE IS BLACK : TRIANGLE



Transform triangle case to the line case!

UNCLE IS BLACK : TRIANGLE

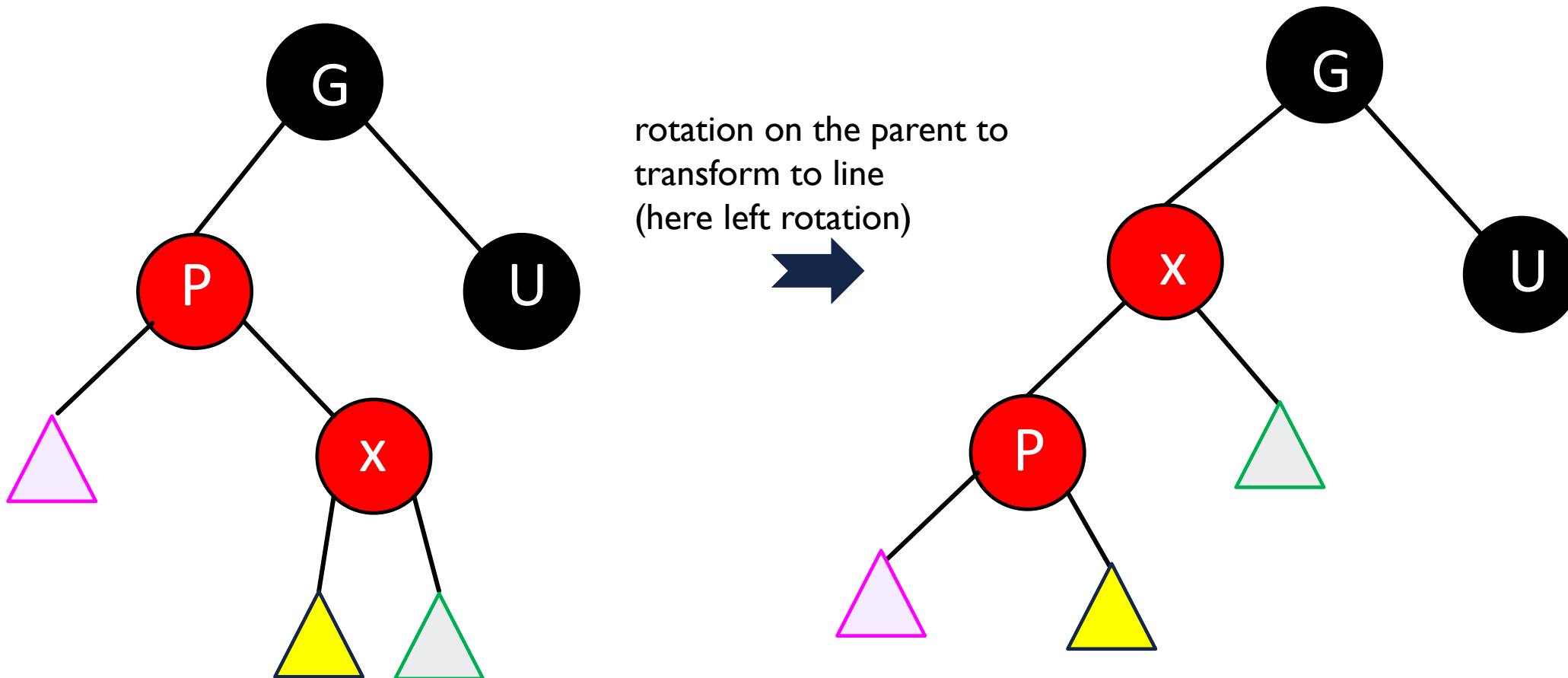


Perform a rotation on the parent of x:

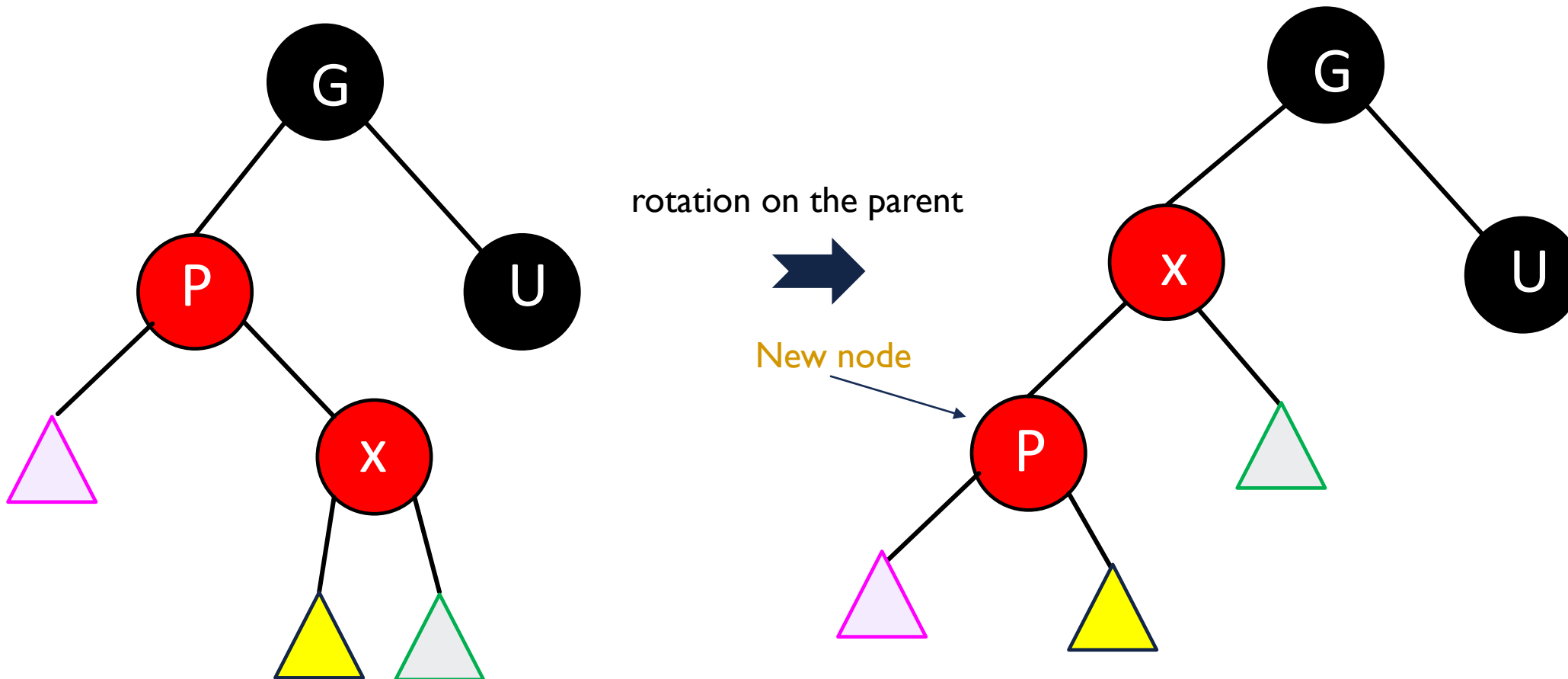
If parent is a left child: Left rotation is needed, otherwise, right rotation

Since both x and its parent are red, so rotation won't affect the black height.

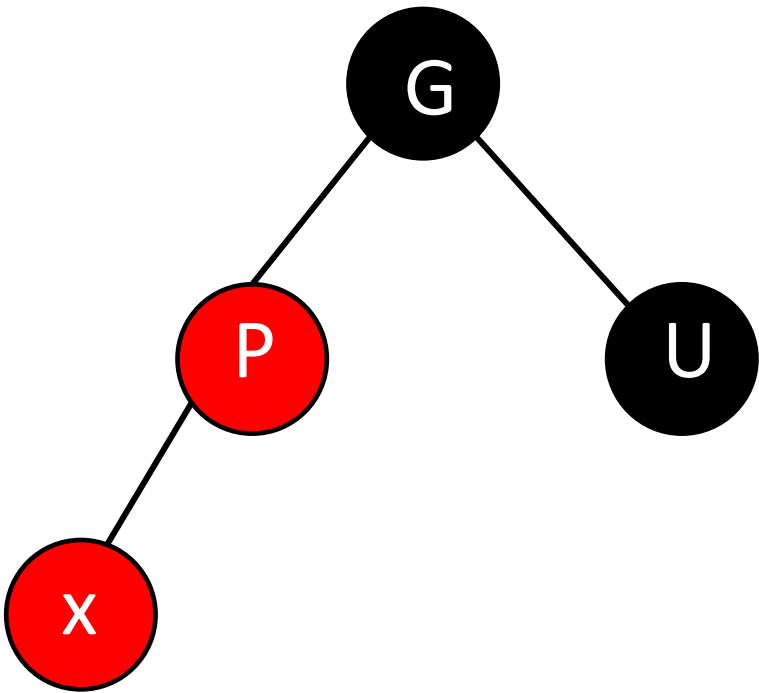
UNCLE IS BLACK : TRIANGLE



UNCLE IS BLACK : TRIANGLE

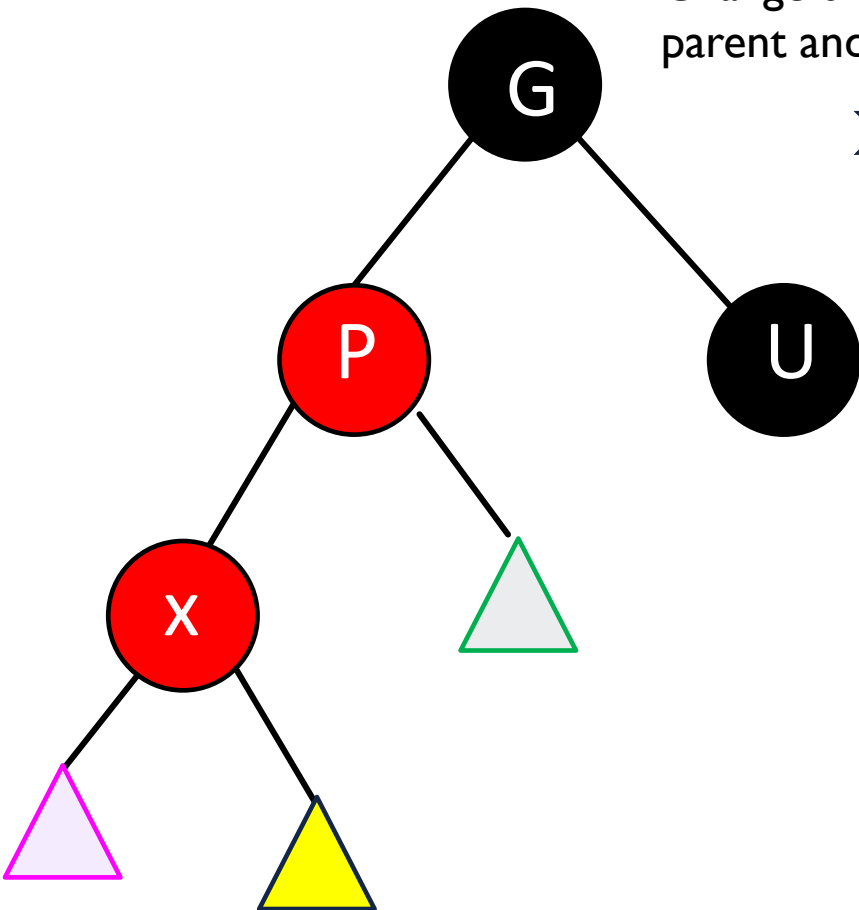


UNCLE IS BLACK : LINE

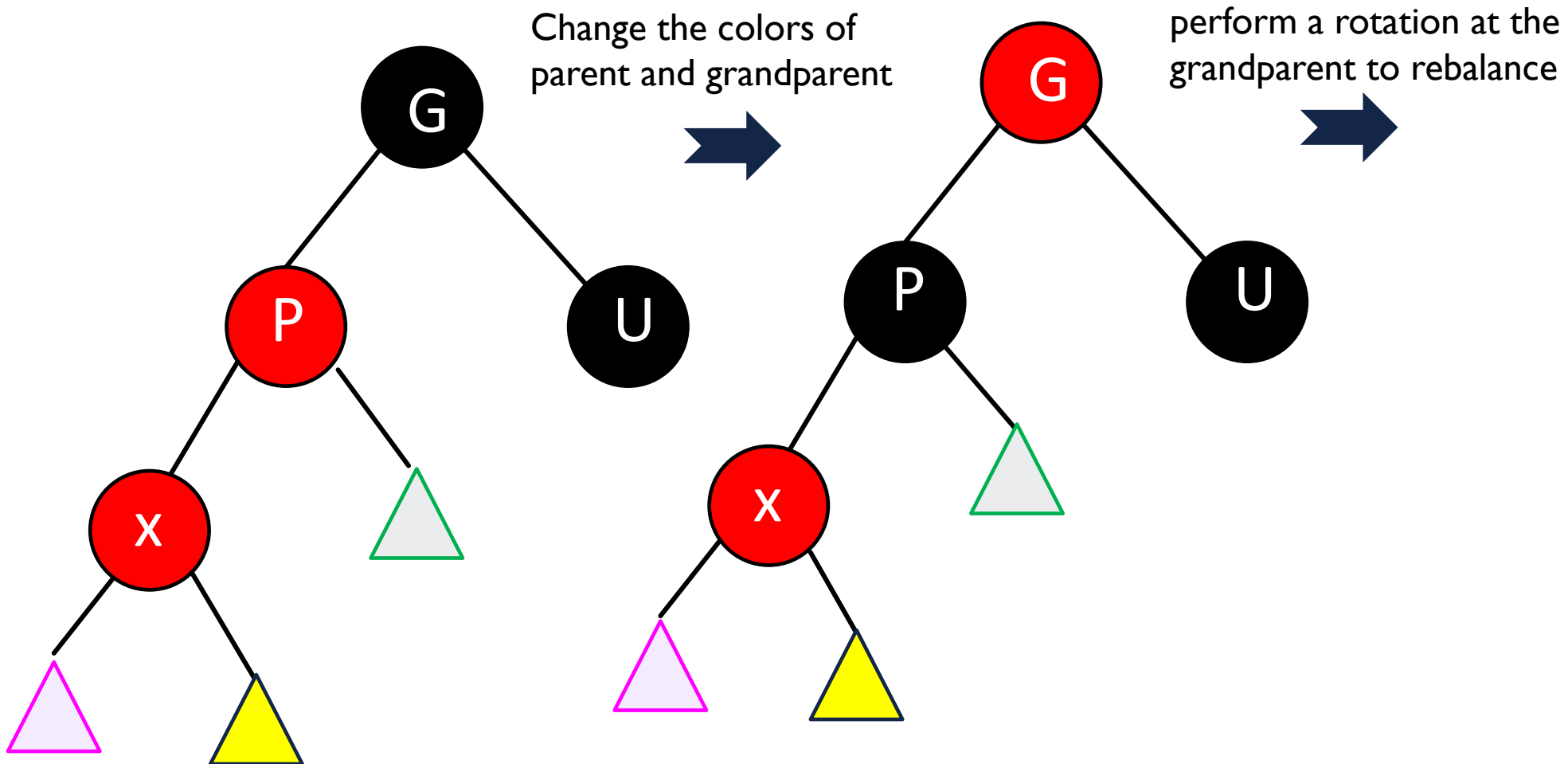


UNCLE IS BLACK : LINE

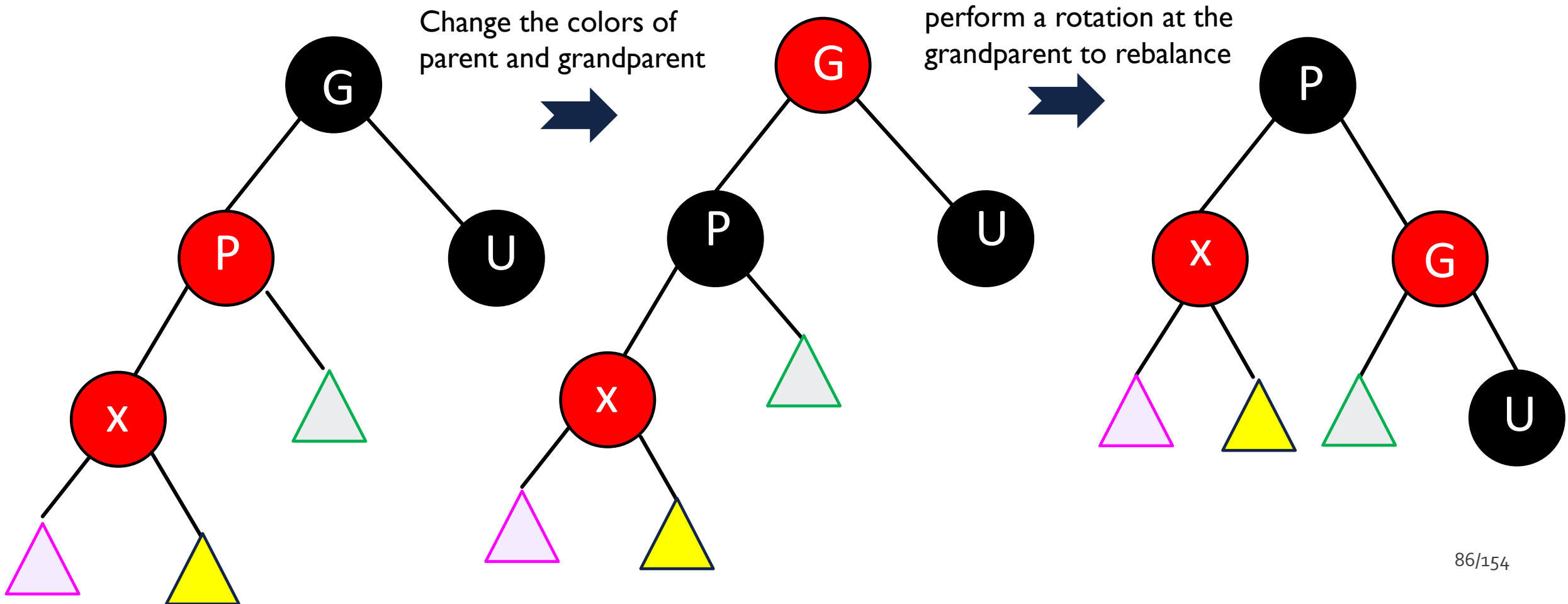
Change the colors of
parent and grandparent



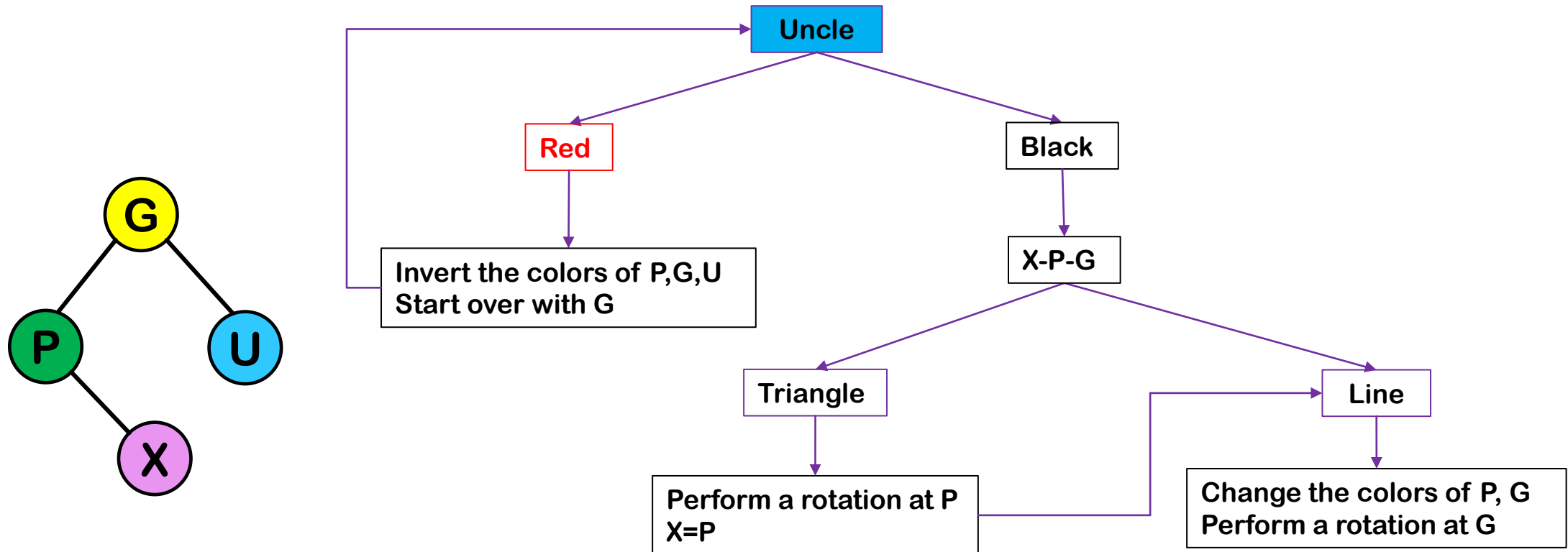
UNCLE IS BLACK : LINE



UNCLE IS BLACK : LINE

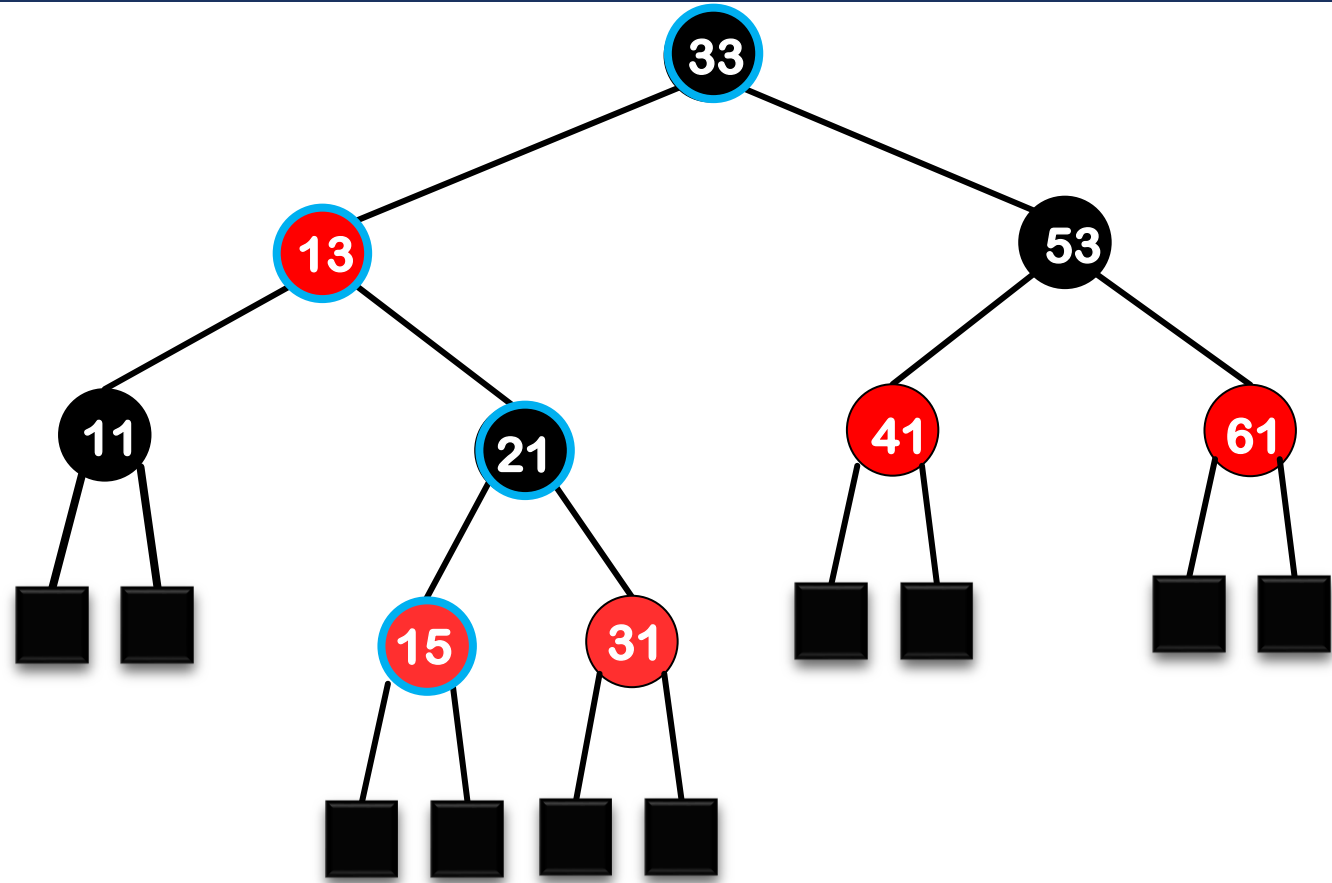


INSERT



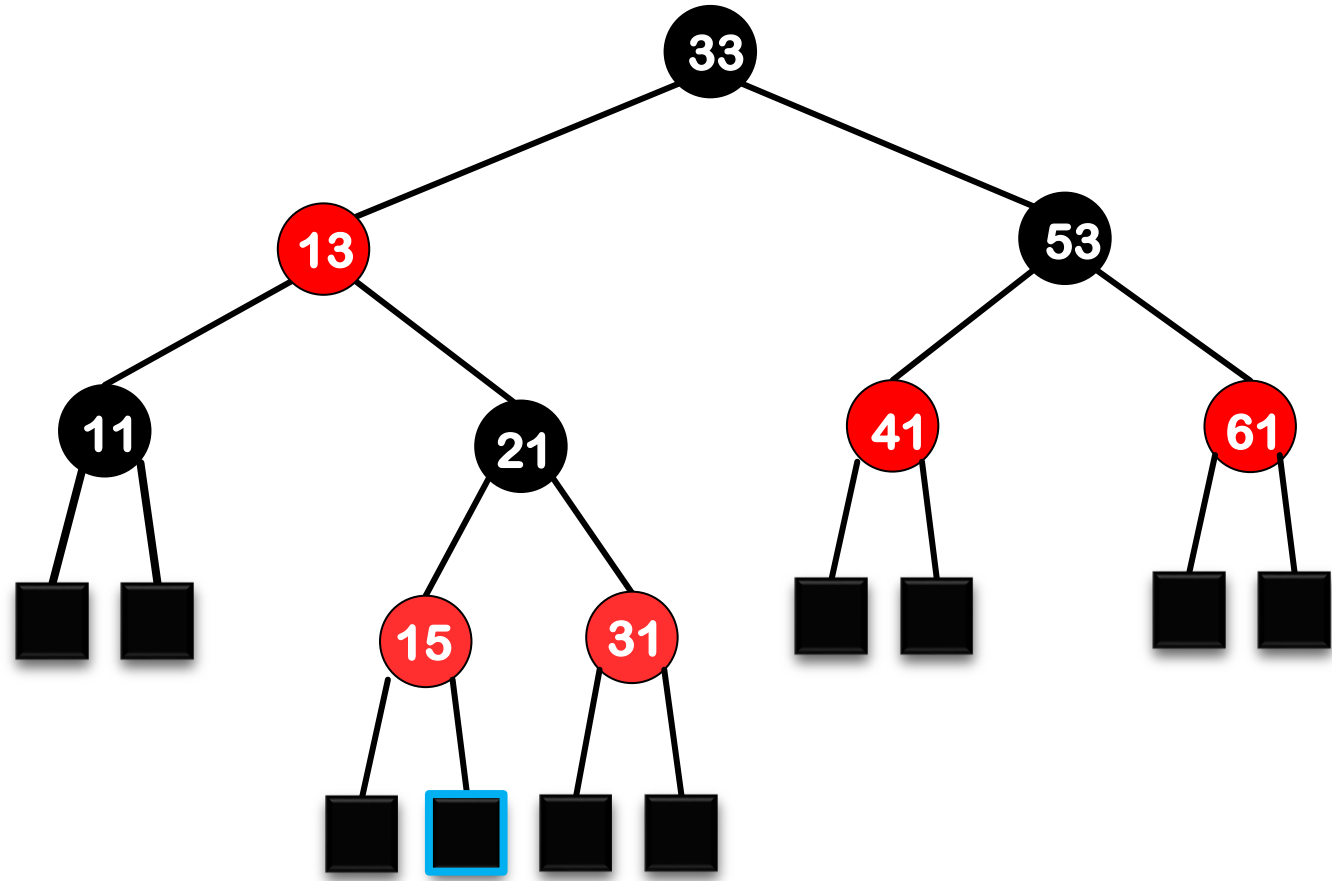
INSERTION: EXAMPLE

Insert 20



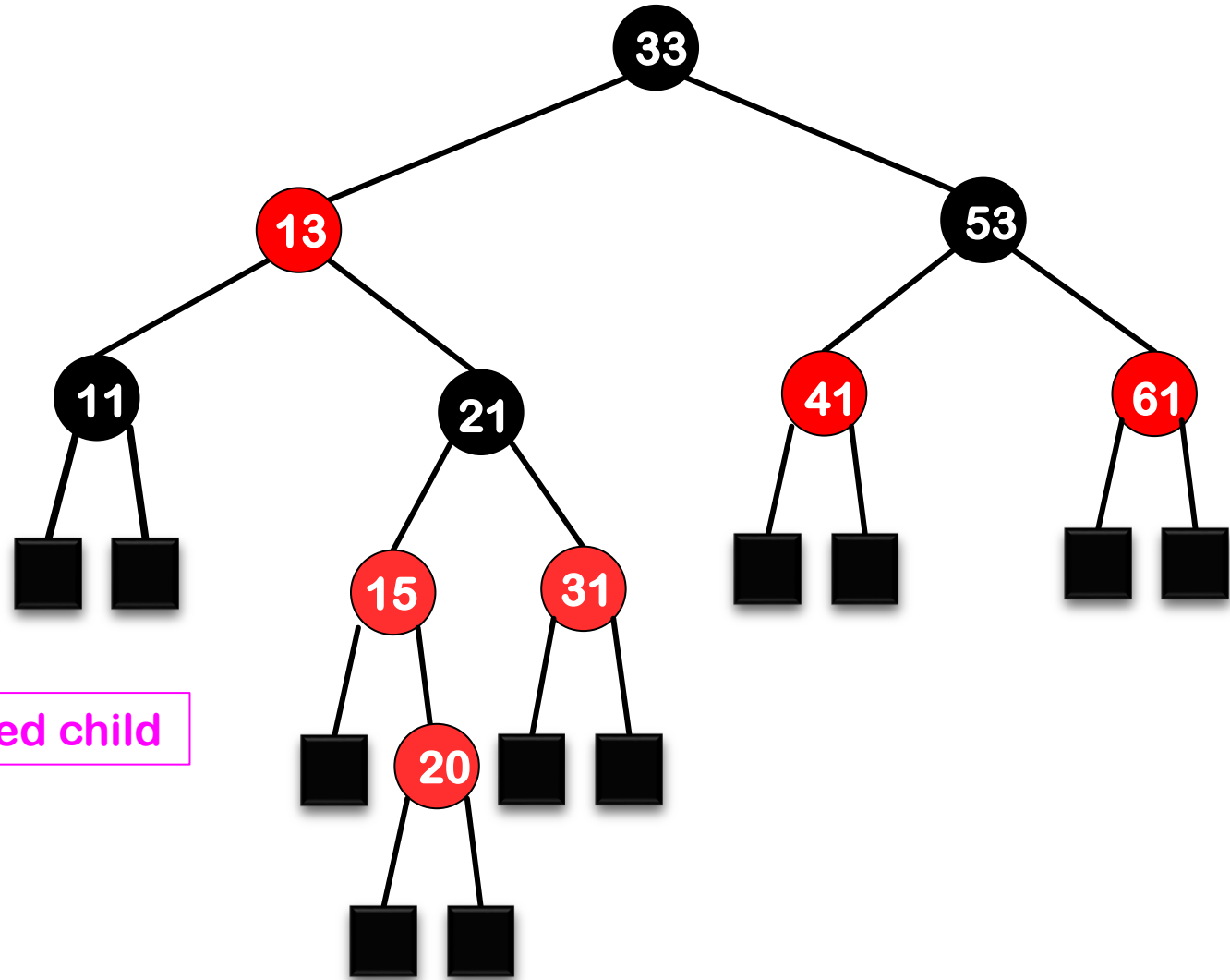
INSERTION: EXAMPLE

Insert 20



INSERTION: EXAMPLE

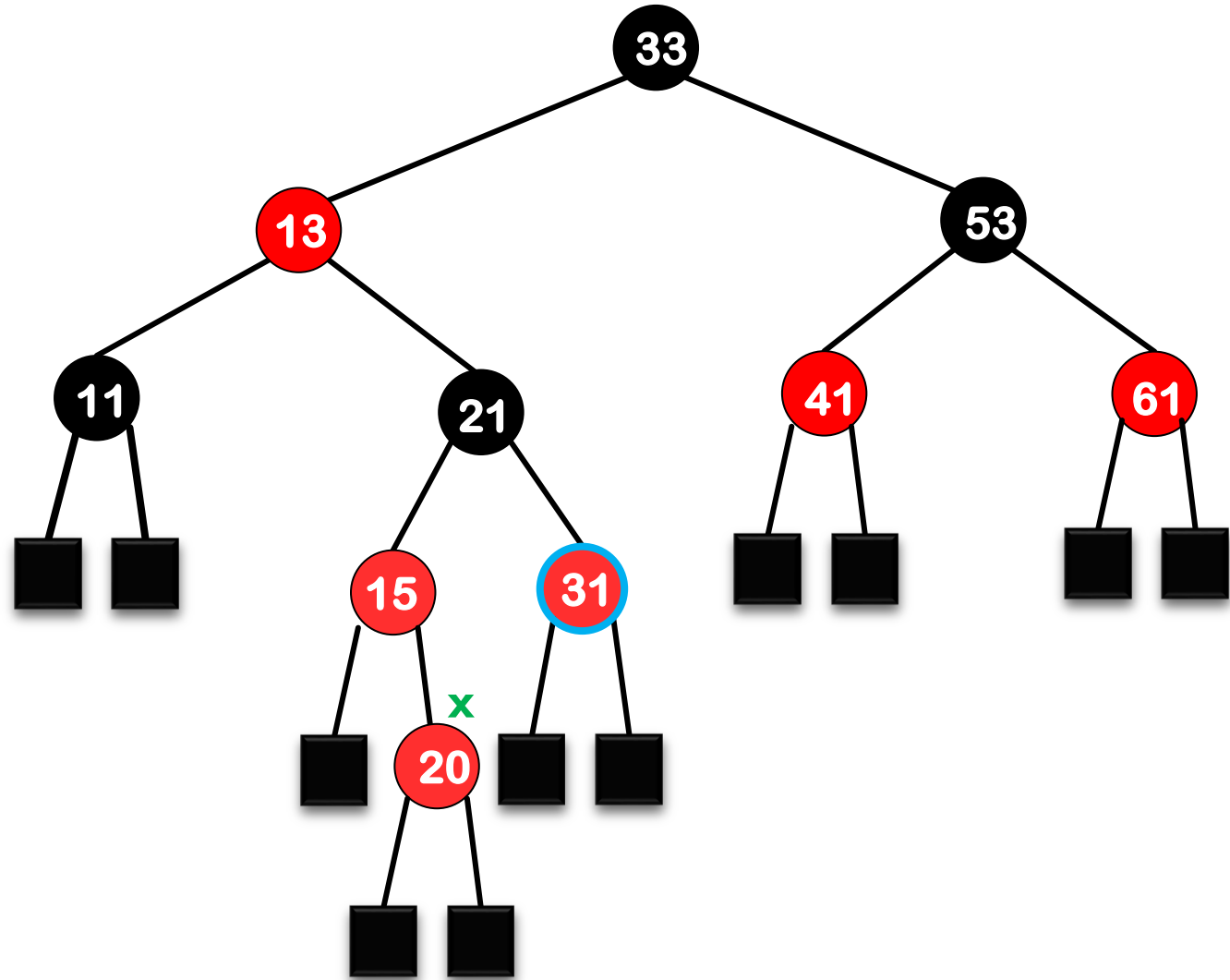
Insert 20



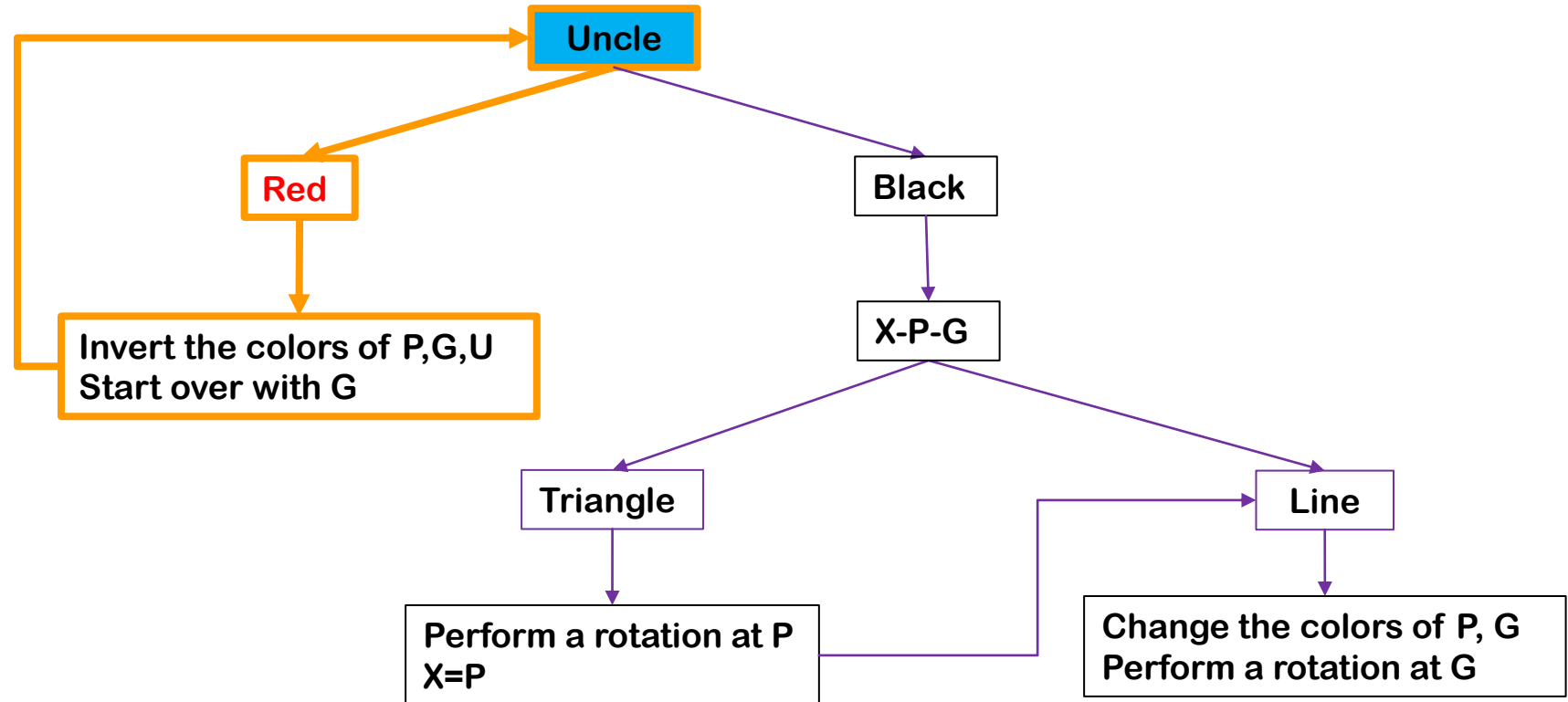
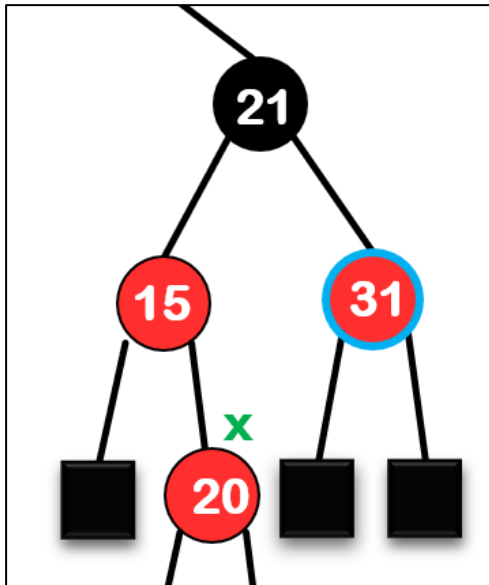
it is not a RB tree because 15 has a red child

INSERTION: EXAMPLE

Insert 20

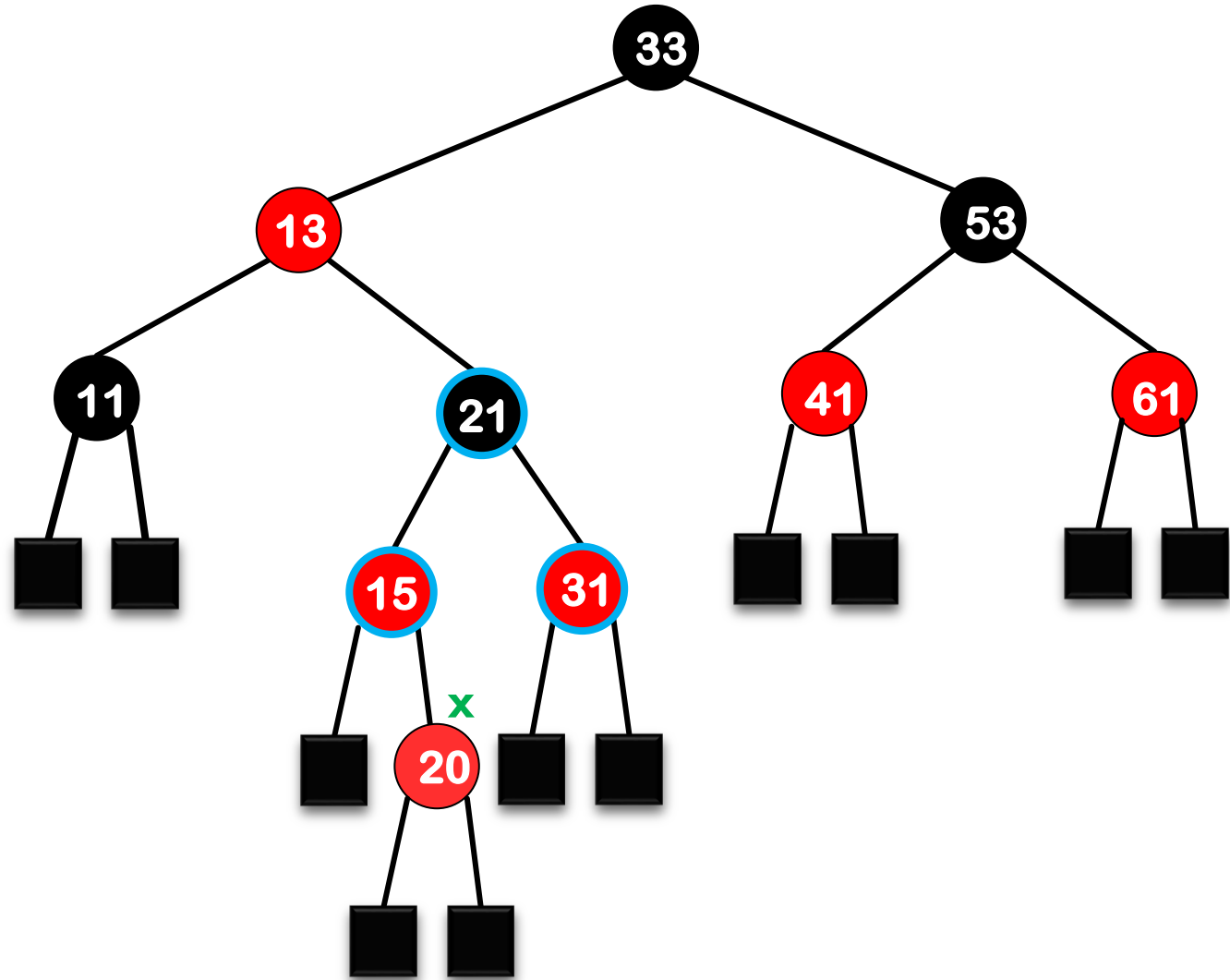


INSERT



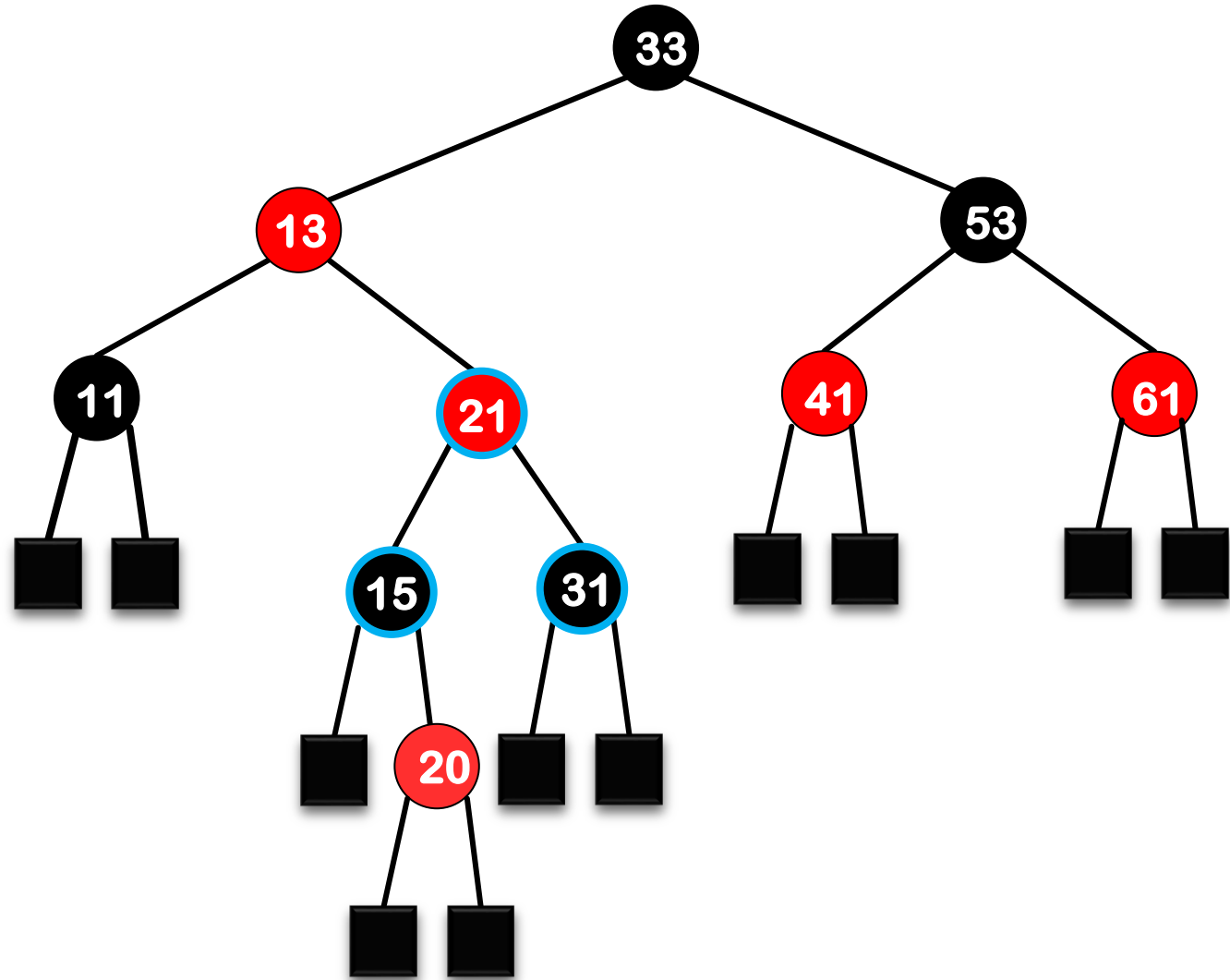
INSERTION: EXAMPLE

Insert 20



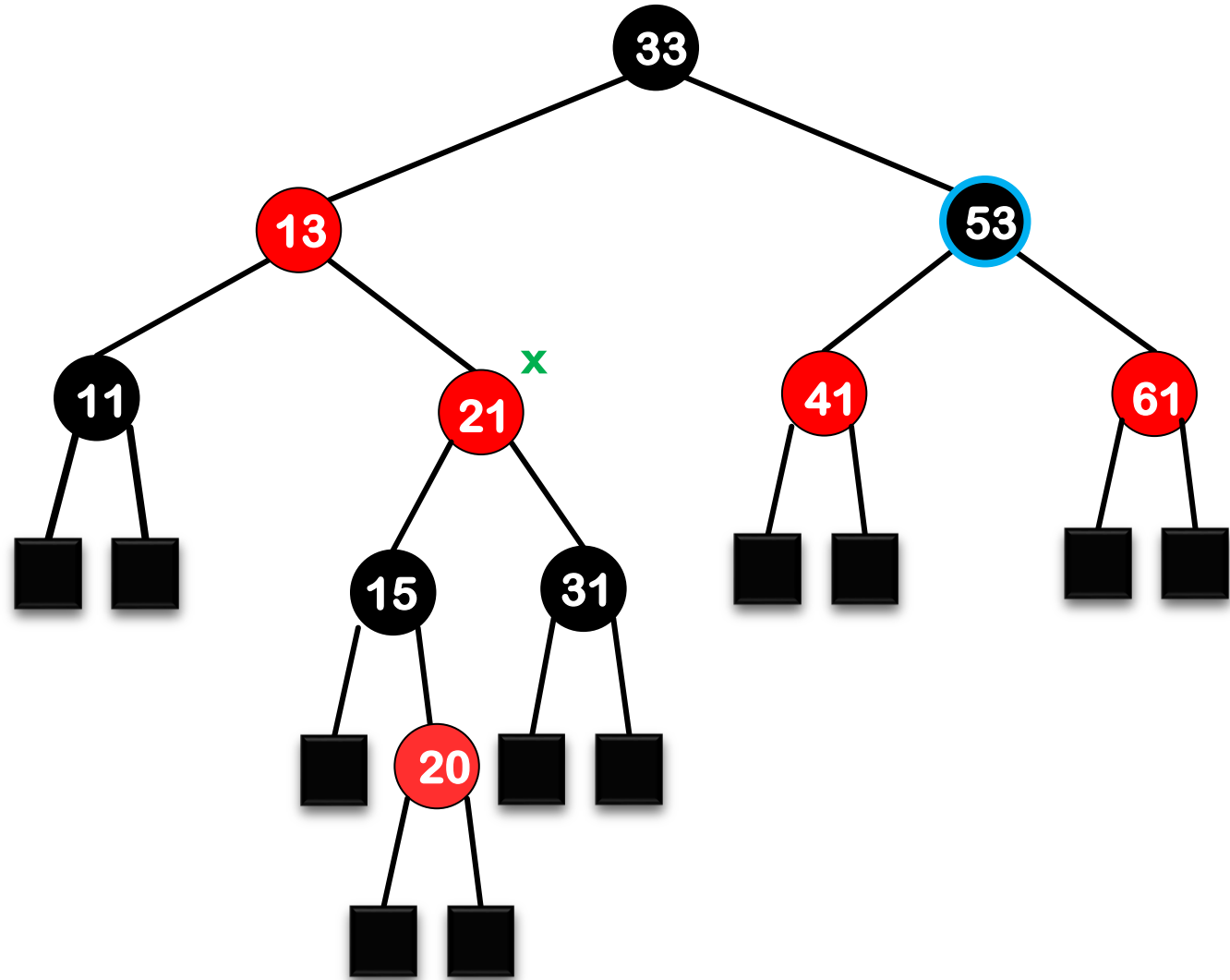
INSERTION: EXAMPLE

Insert 20

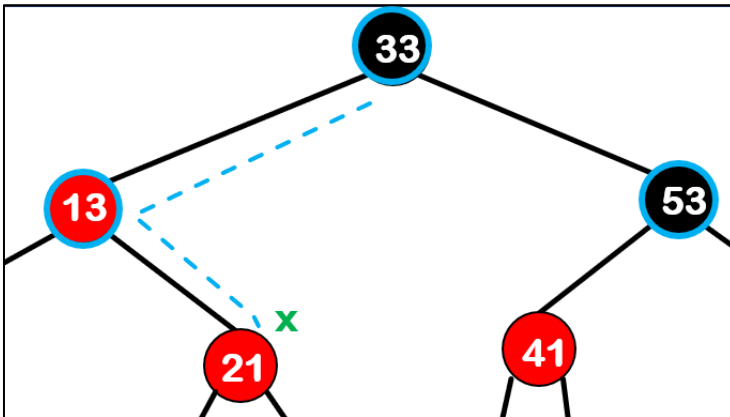
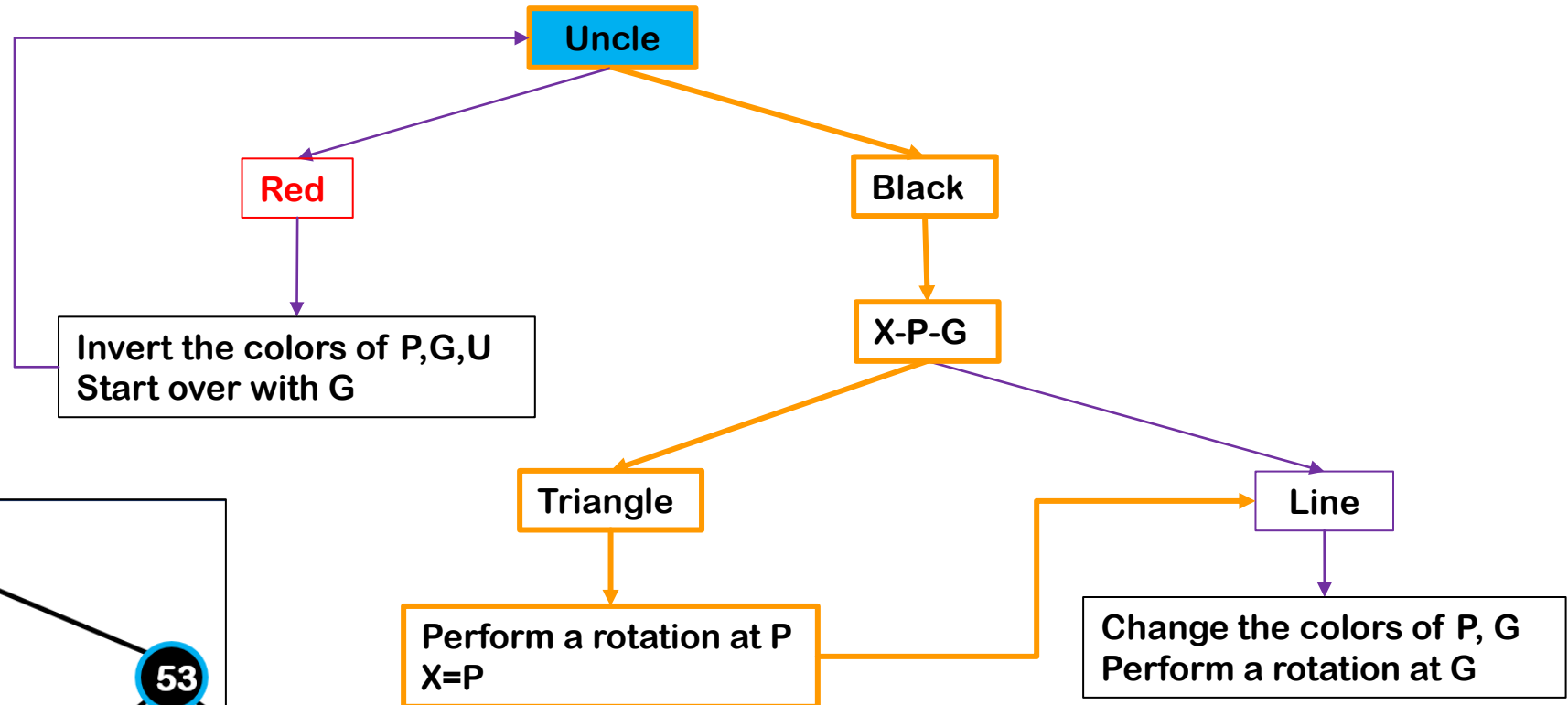


INSERTION: EXAMPLE

Insert 20



INSERT



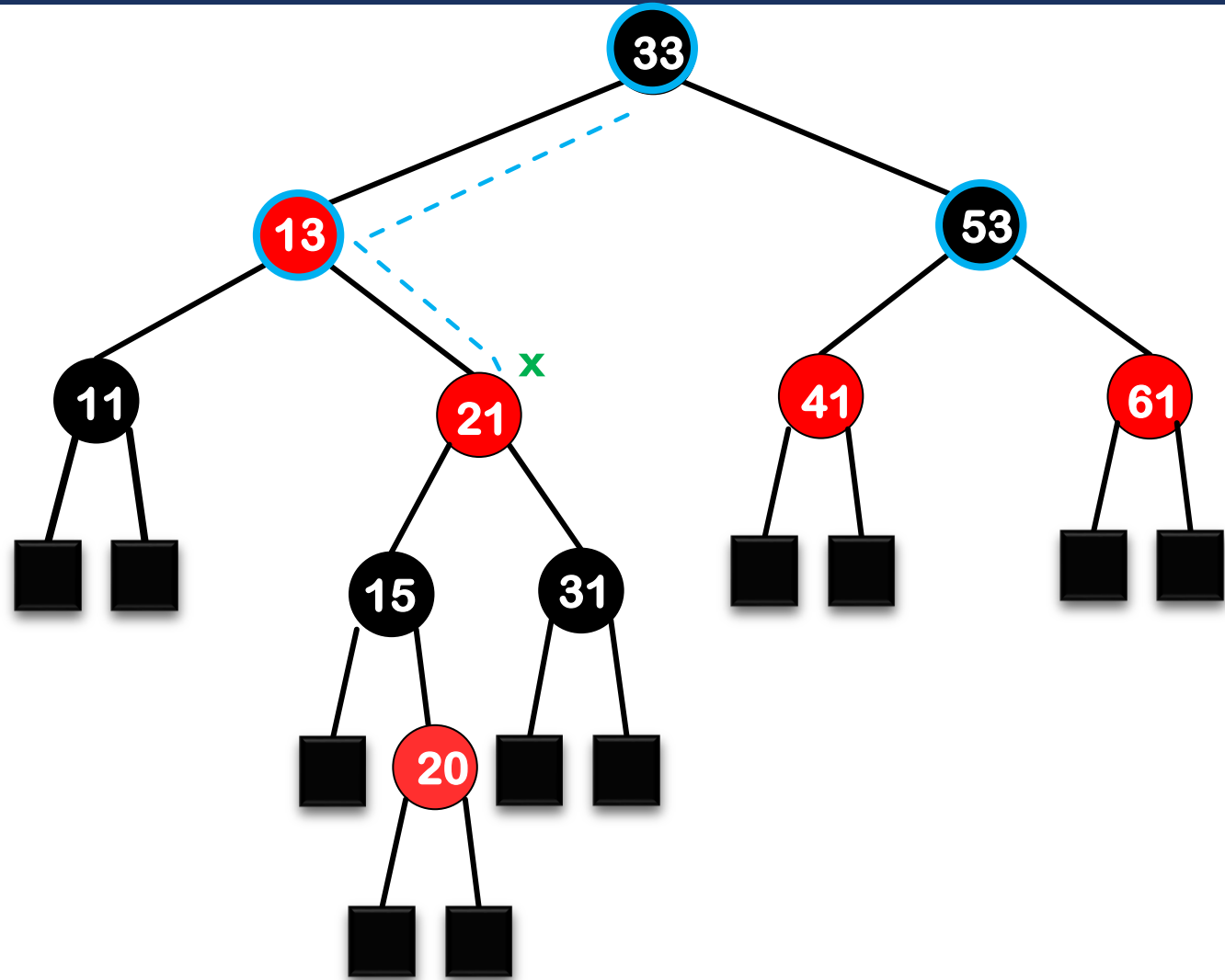
INSERTION: EXAMPLE

Insert 20

Uncle is black

Triangle: Transform to line

Left rotation at parent of x
Start over with p



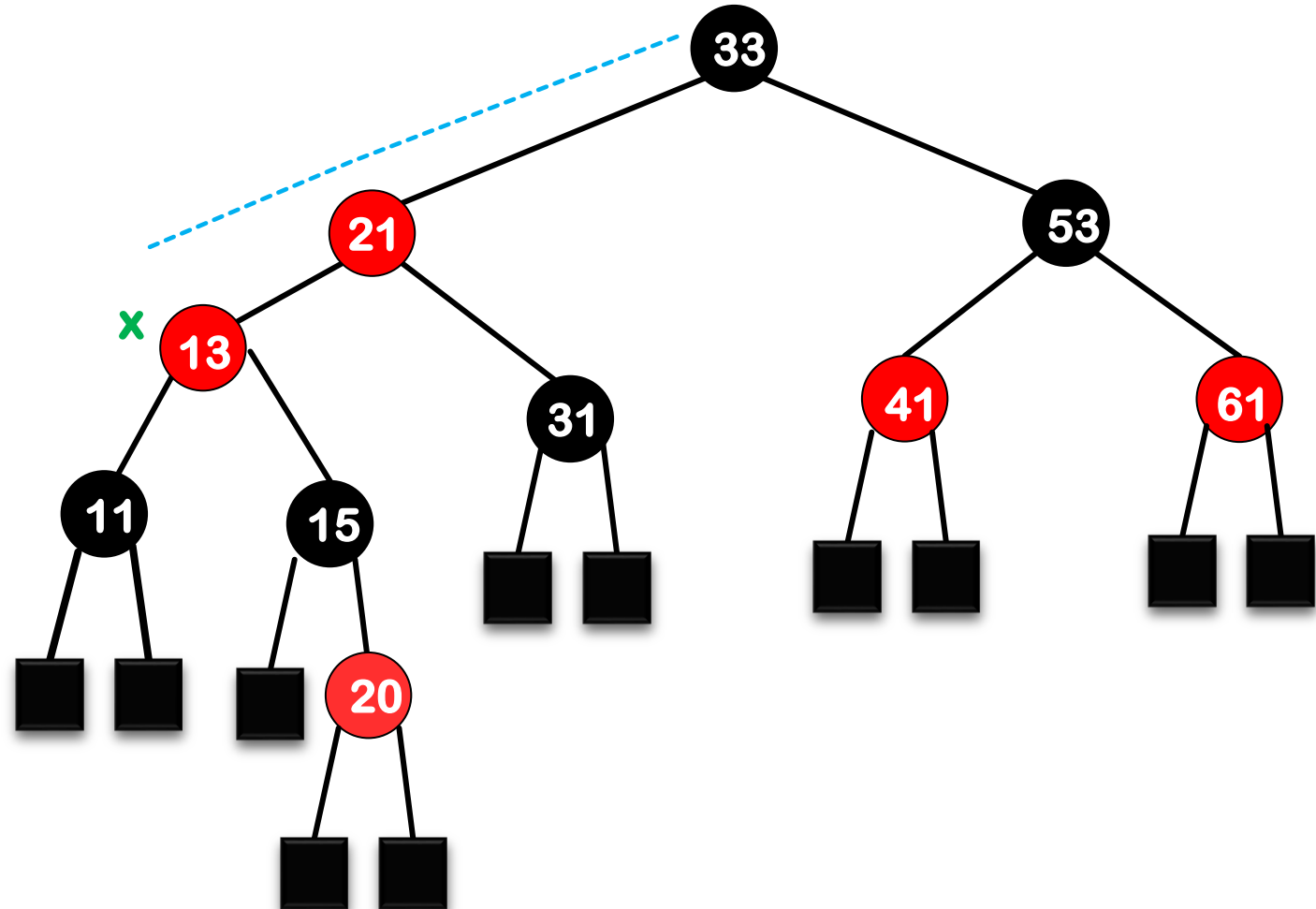
INSERTION: EXAMPLE

Insert 20

Uncle is black

Triangle: Transform to line

Left rotation at parent of x
Start over with p

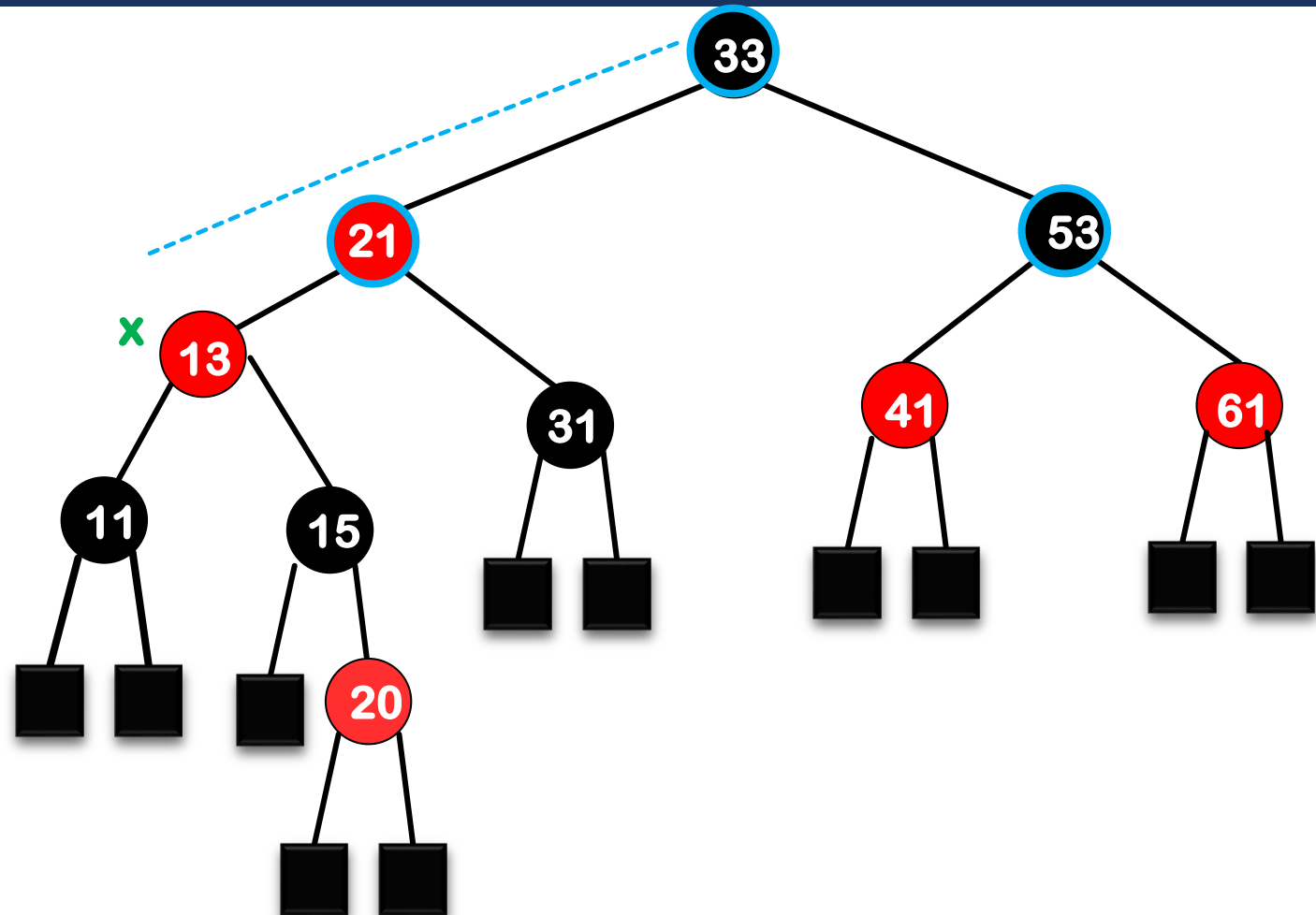


INSERTION: EXAMPLE

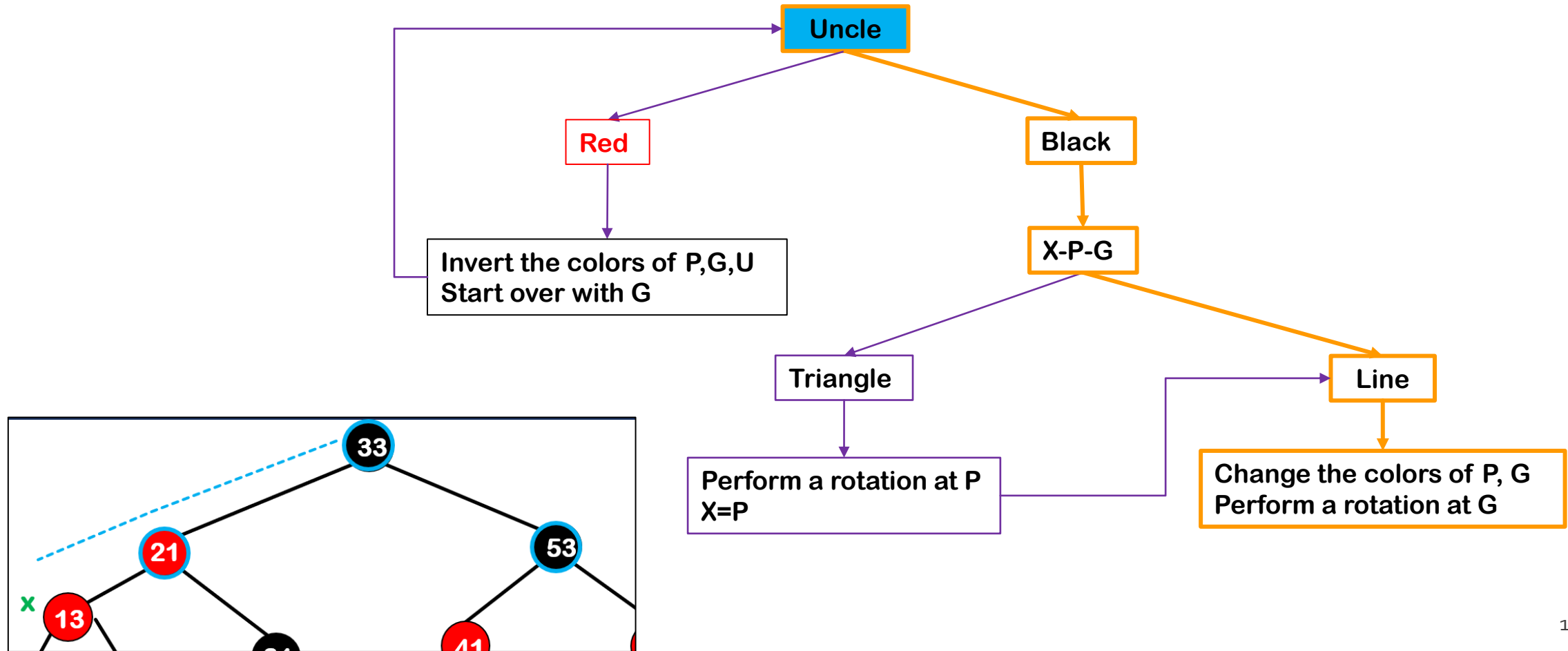
Insert 20

Uncle is black

Line



INSERT



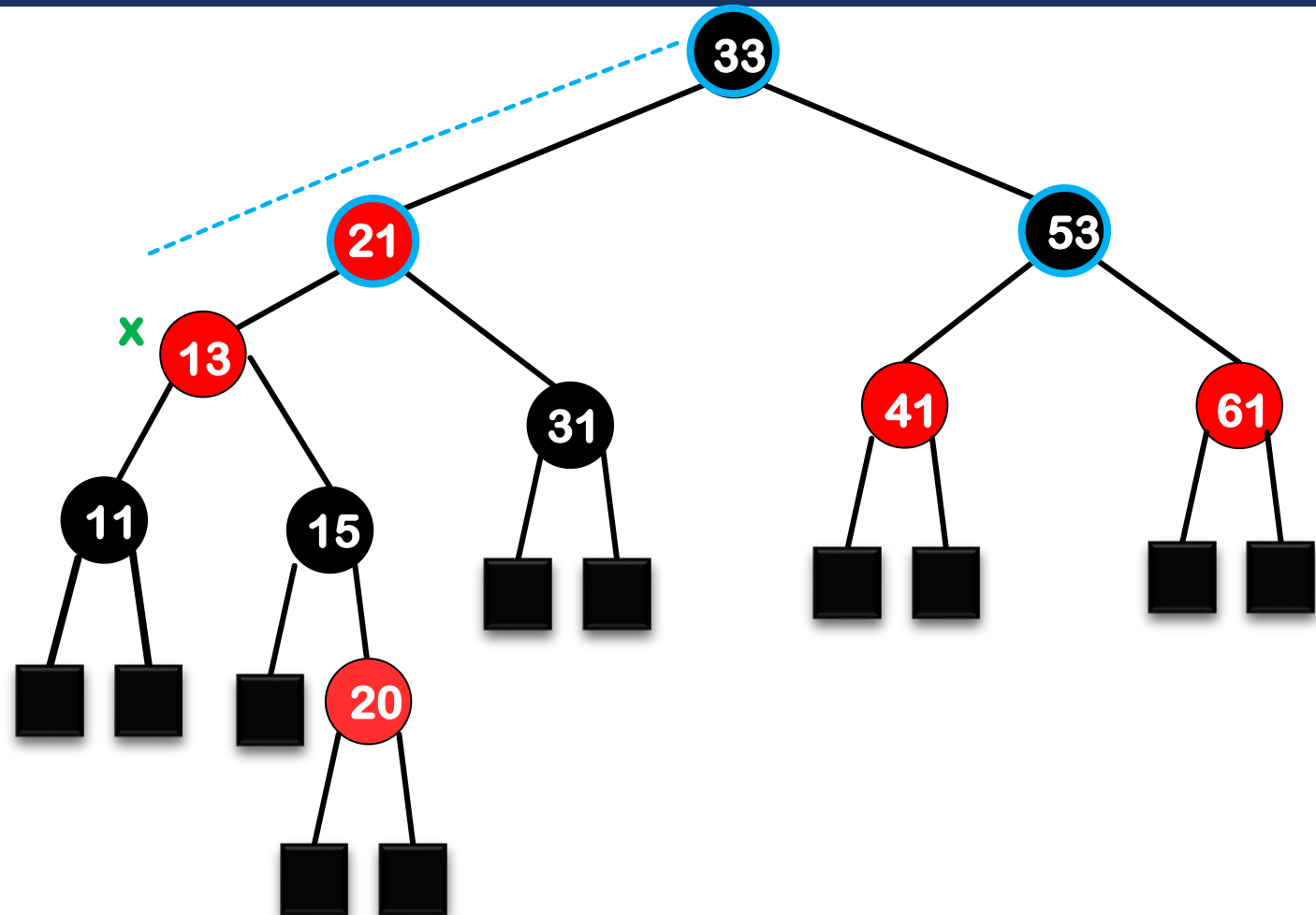
INSERTION: EXAMPLE

Insert 20

Uncle is black

Line

Change the colors of G and P



INSERTION: EXAMPLE

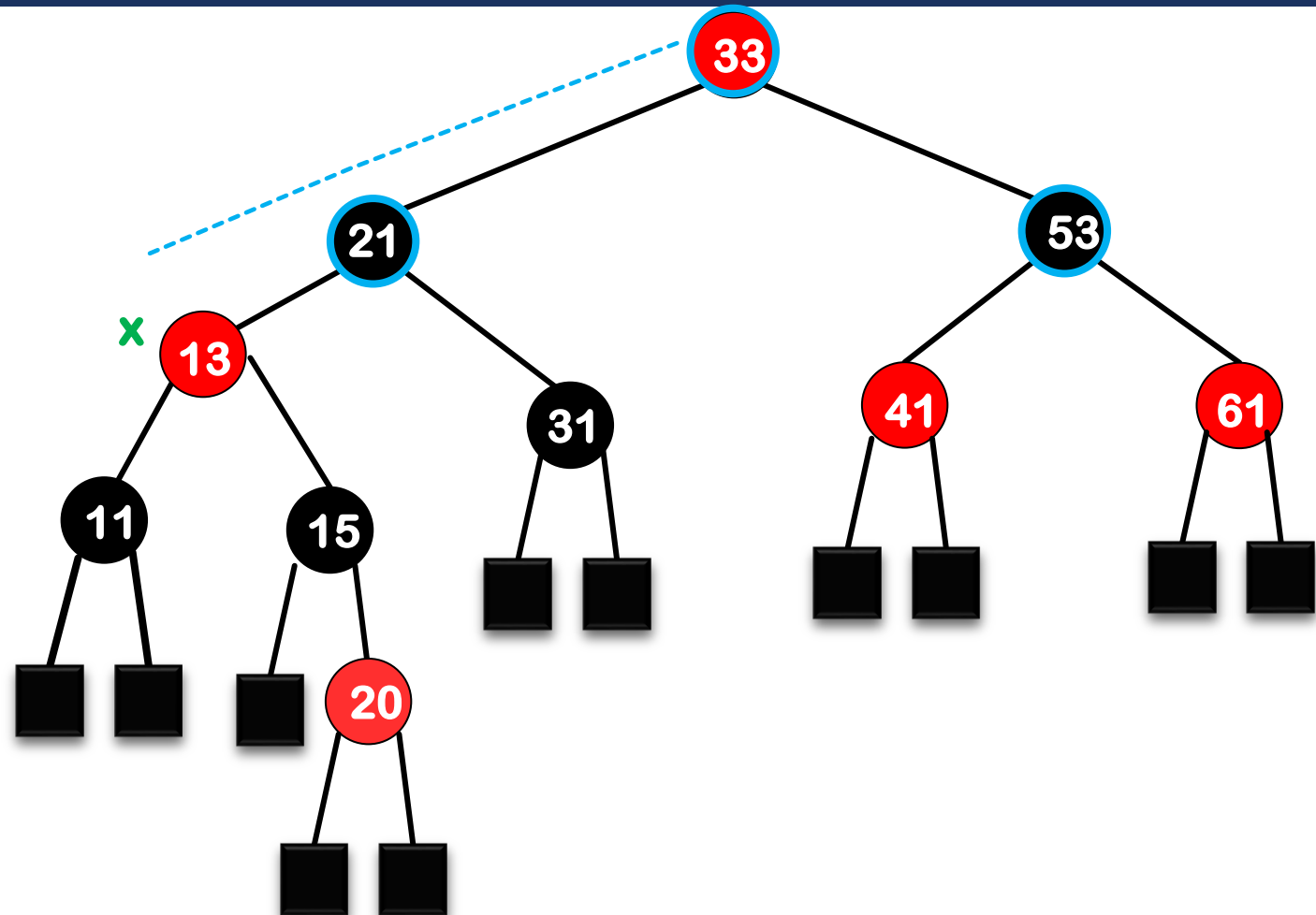
Insert 20

Uncle is black

Line

Change the colors of G and P

Do a right rotation at G: 33



INSERTION: EXAMPLE

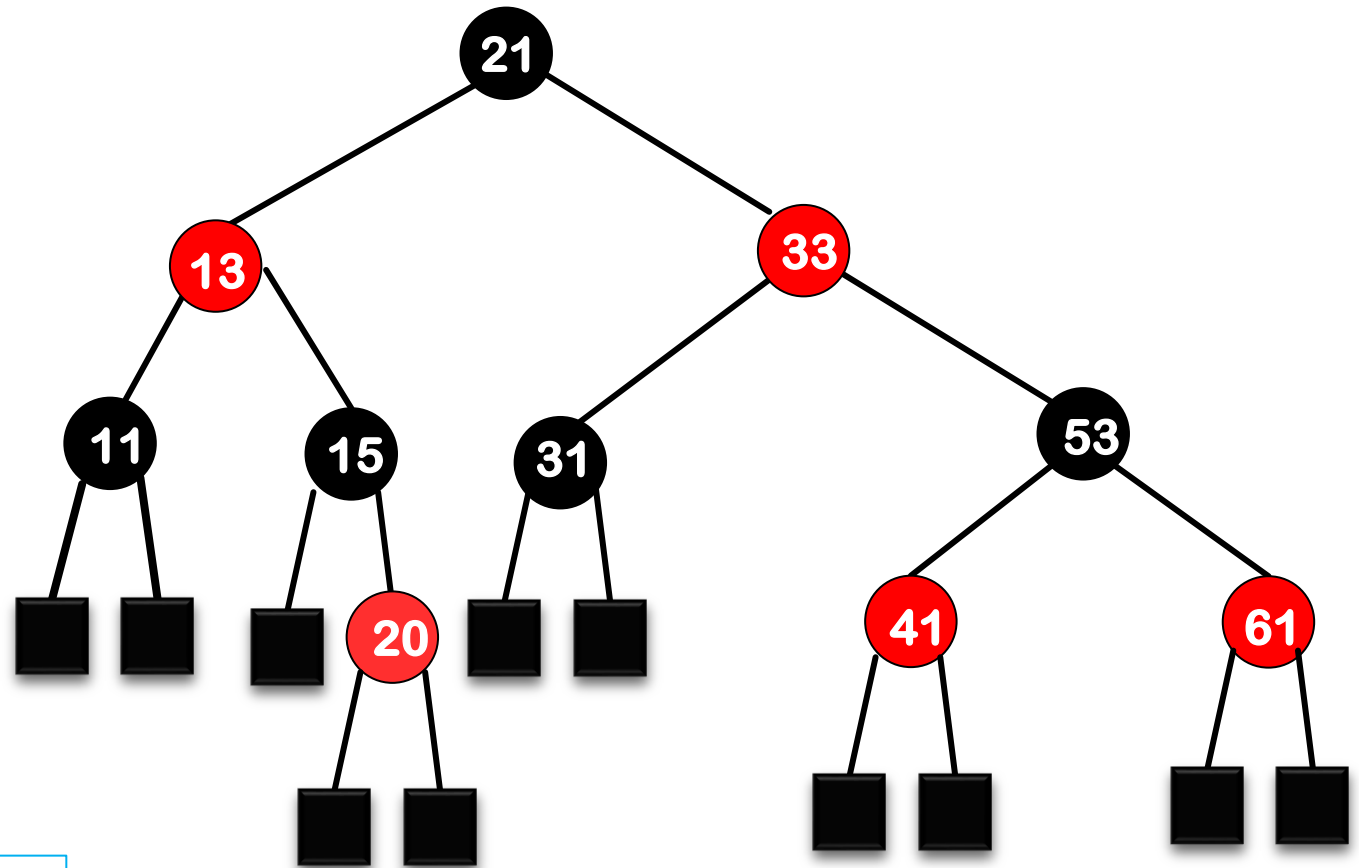
Insert 20

Uncle is black

Line

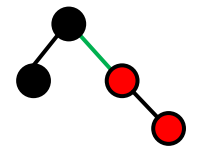
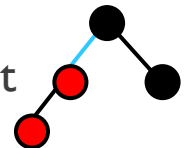
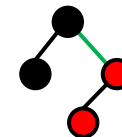
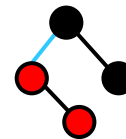
color the parent black and
the grandparent red

Do a right rotation at the grandparent: 33



PSEUDOCODE

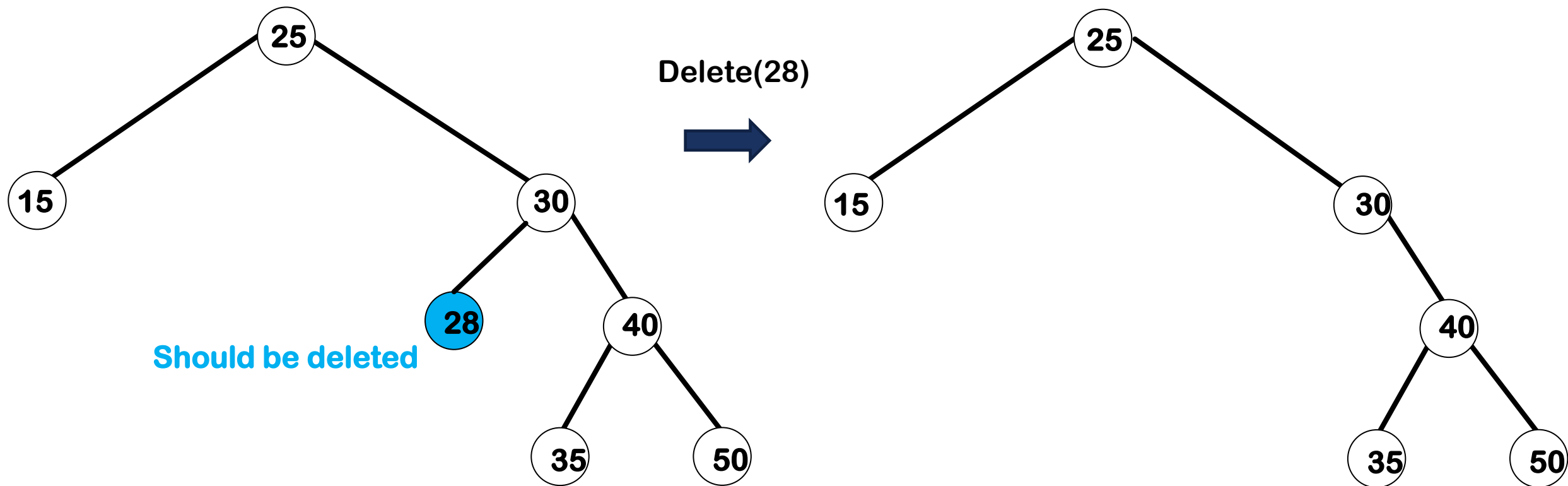
1. Perform standard BST **insertion** and set the color of the newly inserted node (denoted by x) to RED.
2. If x is the root, change the color of x to **BLACK**.
3. while **x.parent.color == Red**:
 - a) if **x.uncle.color == RED**:
 - (i) x.parent.color=BLACK , x.uncle.color=BLACK, x.grandparent.color=RED.
 - (ii) x = x.grandparent.
 - b) elif **x.uncle.color==BLACK**, there are four possible cases:
 - i. Line: Swap the colors of x.parent and x.grandparent, perform a right rotation at the grandparent
 - ii. Triangle: Perform a left rotation at x.parent, x=x.parent
 - iii. Line: Swap the colors of x.parent and x.grandparent, perform a left rotation at the grandparent
 - iv. Triangle: Perform a right rotation at x.parent, x=x.parent
4. root.color=BLACK



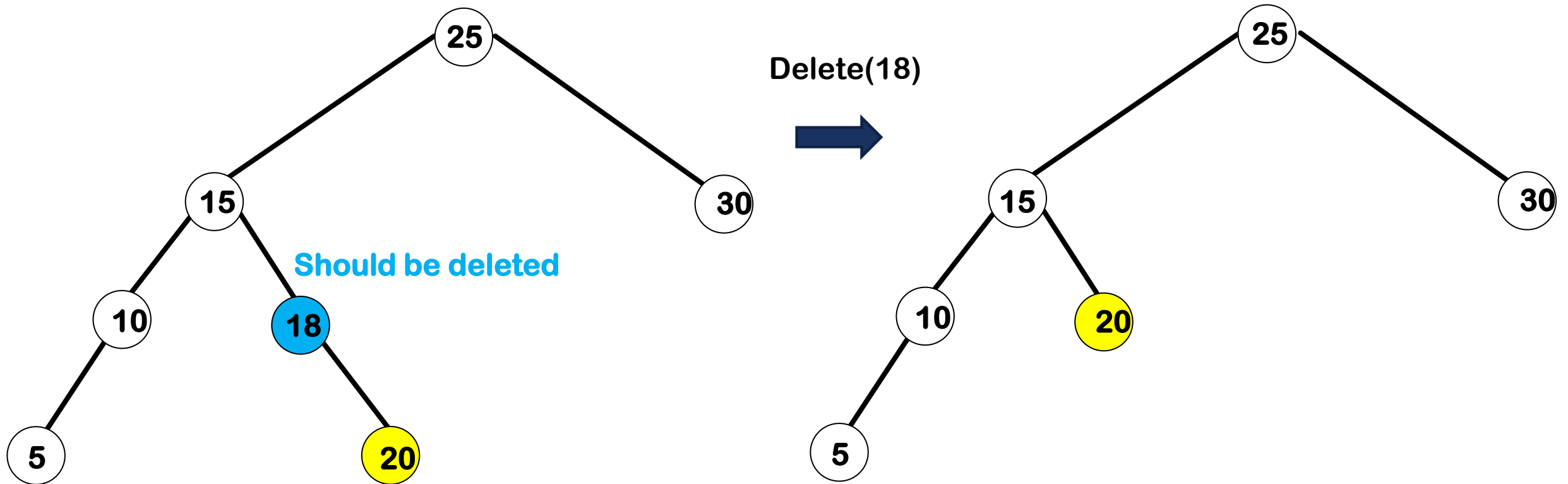
REMOVAL

- Note that when we perform standard delete operation in BST, there are three cases for the node that should be deleted :
 1. It has no children
 2. It has only one child
 3. It has two children
- In each case, we always end up deleting a node that is **either a leaf or has only one child**. This is clear for cases 1 and 2. In case 3, we copy the **data** (not the color!) **of the smallest successor on the right** to **the node that should be deleted**, and then we delete that successor, which is either a leaf or has only one child (see figure).

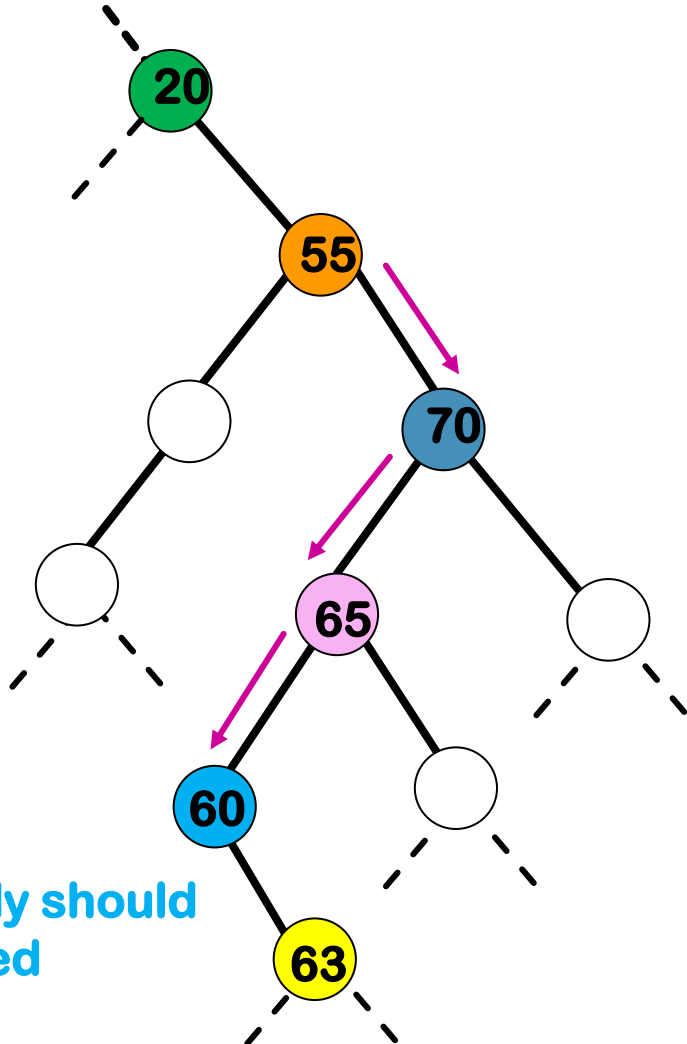
I. NO CHILDREN



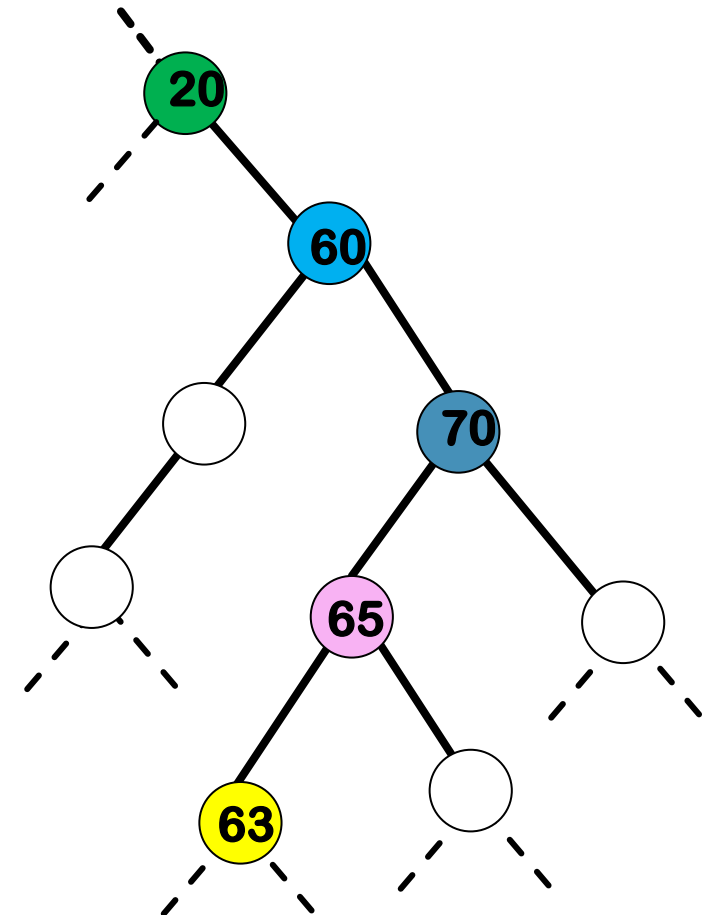
2. ONE CHILD



3.TWO CHILDREN



Delete(55)



REMOVAL

- ❑ Let x be the node that is to be physically deleted.
- ❑ If x is **red**, then nothing else needs to be done.
- ❑ If x is **black**

REMOVAL

- ❑ Let x be the node that is to be physically deleted.
- ❑ If x is **red**, then nothing else needs to be done.
- ❑ If x is **black** and has a single **red** child

REMOVAL

- ❑ Let x be the node that is to be physically deleted.
- ❑ If x is **red**, then nothing else needs to be done.
- ❑ If x is **black** and has a single **red** child, then we turn the single child to black and we are done.

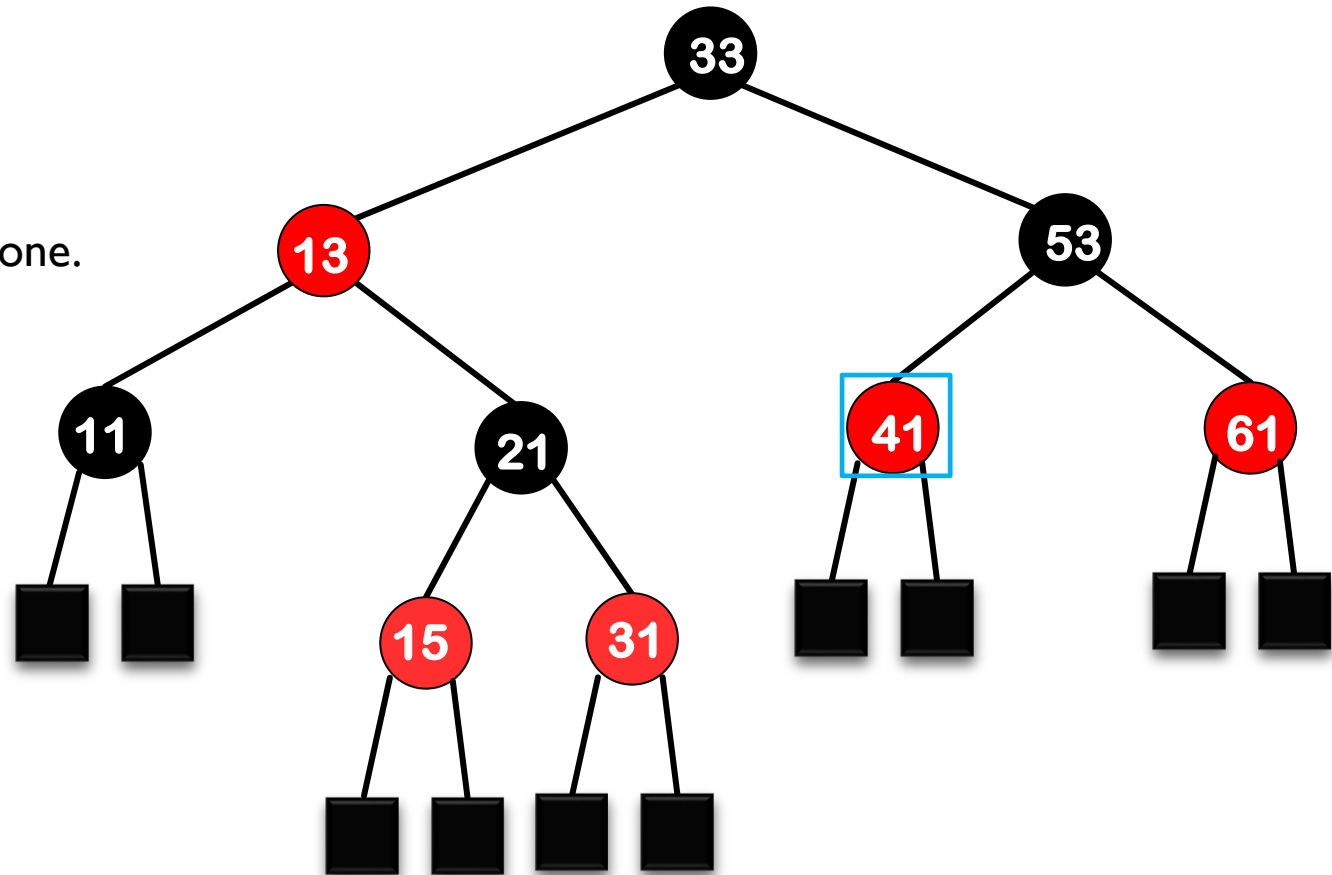
REMOVAL

- ❑ Let **x** be the node that is to be physically deleted.
- ❑ If **x** is **red**, then nothing else needs to be done.
- ❑ If **x** is **black** and has a single **red** child, then we turn the single child to black and we are done.
- ❑ Otherwise, deletion of a black node causes a shortage in the number of black nodes, which needs to be fixed.

DELETION: EXAMPLE I

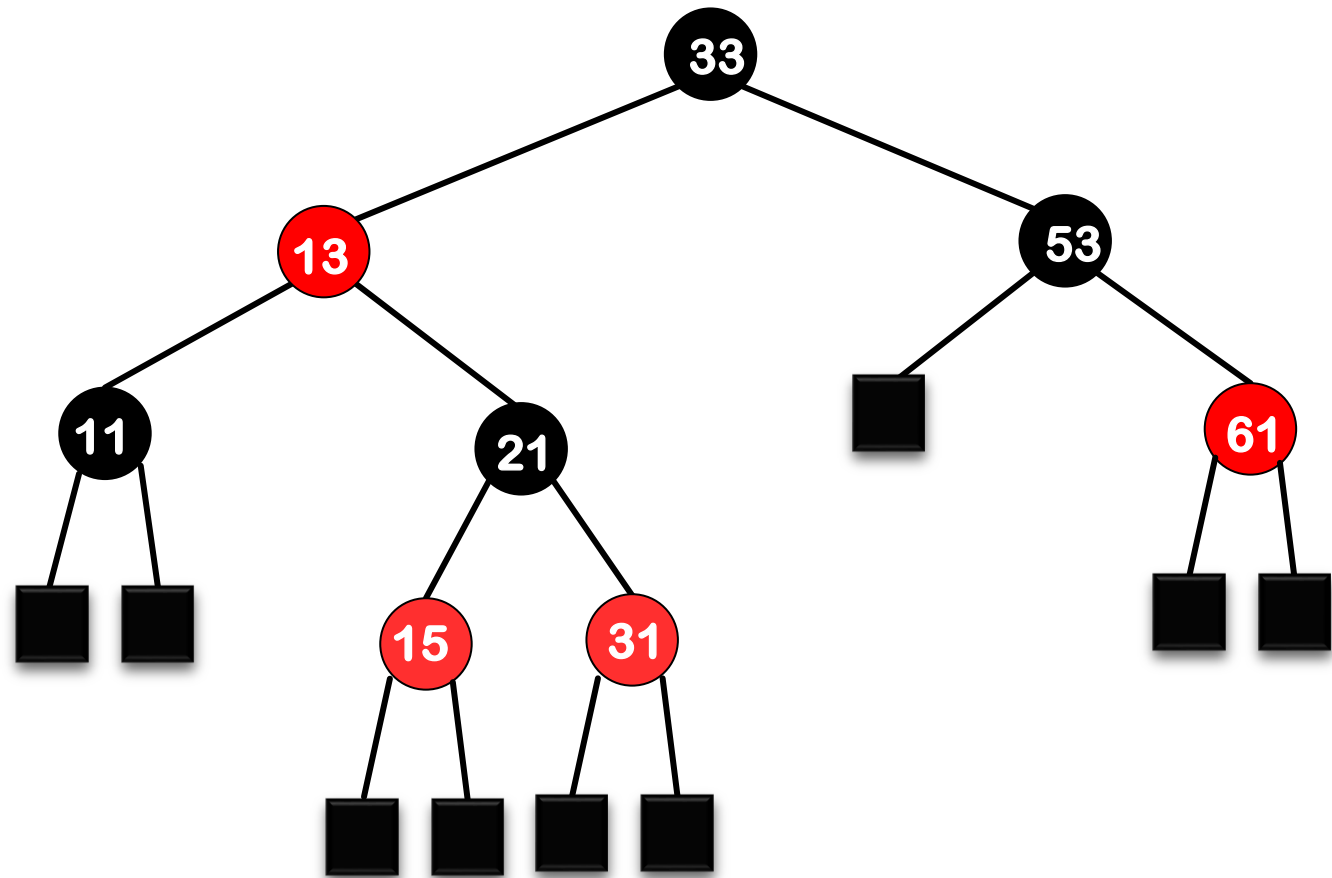
Delete 41

If **x** is **red**, then nothing else needs to be done.



DELETION: EXAMPLE I

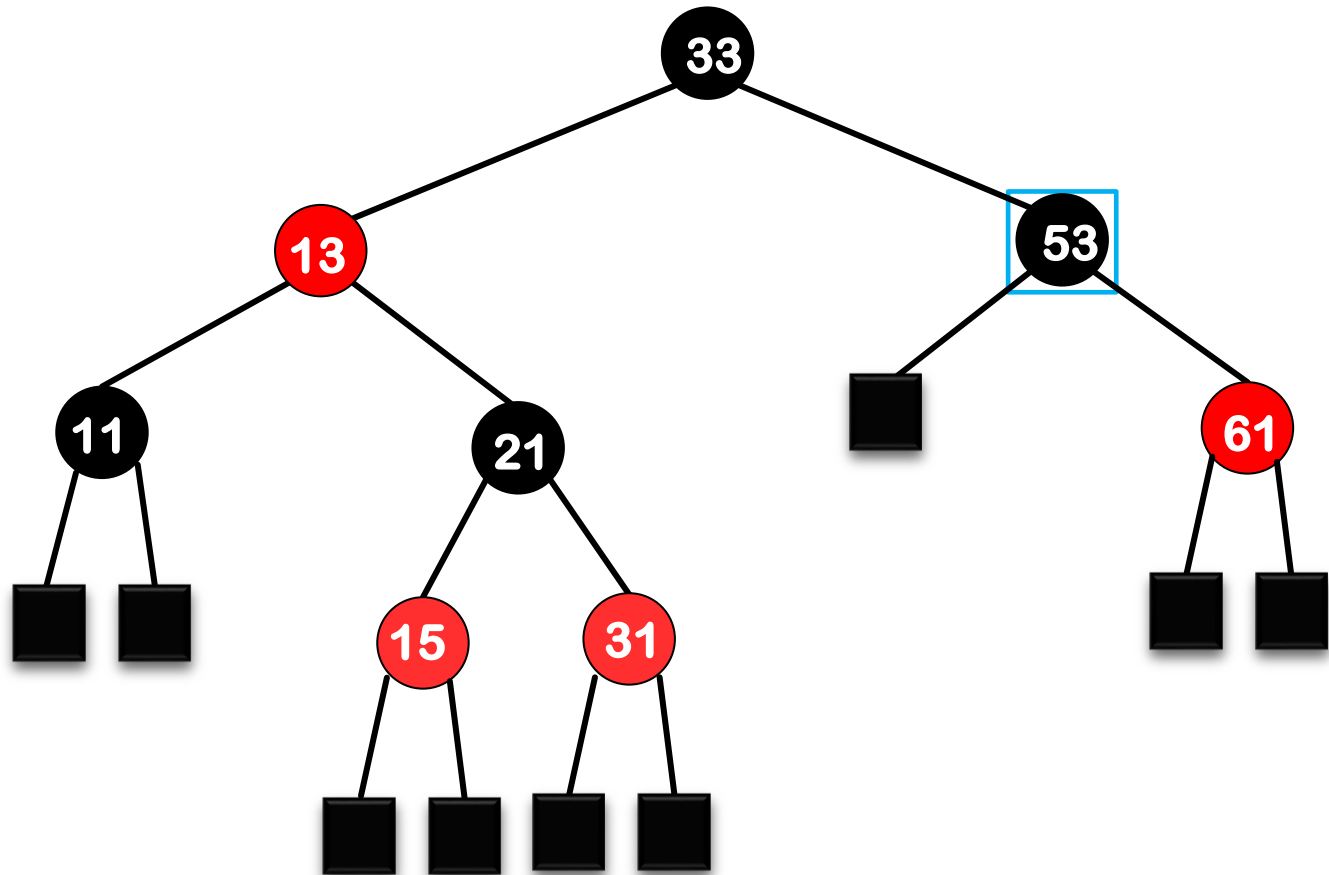
Delete 41



DELETION: EXAMPLE 2

Delete 53

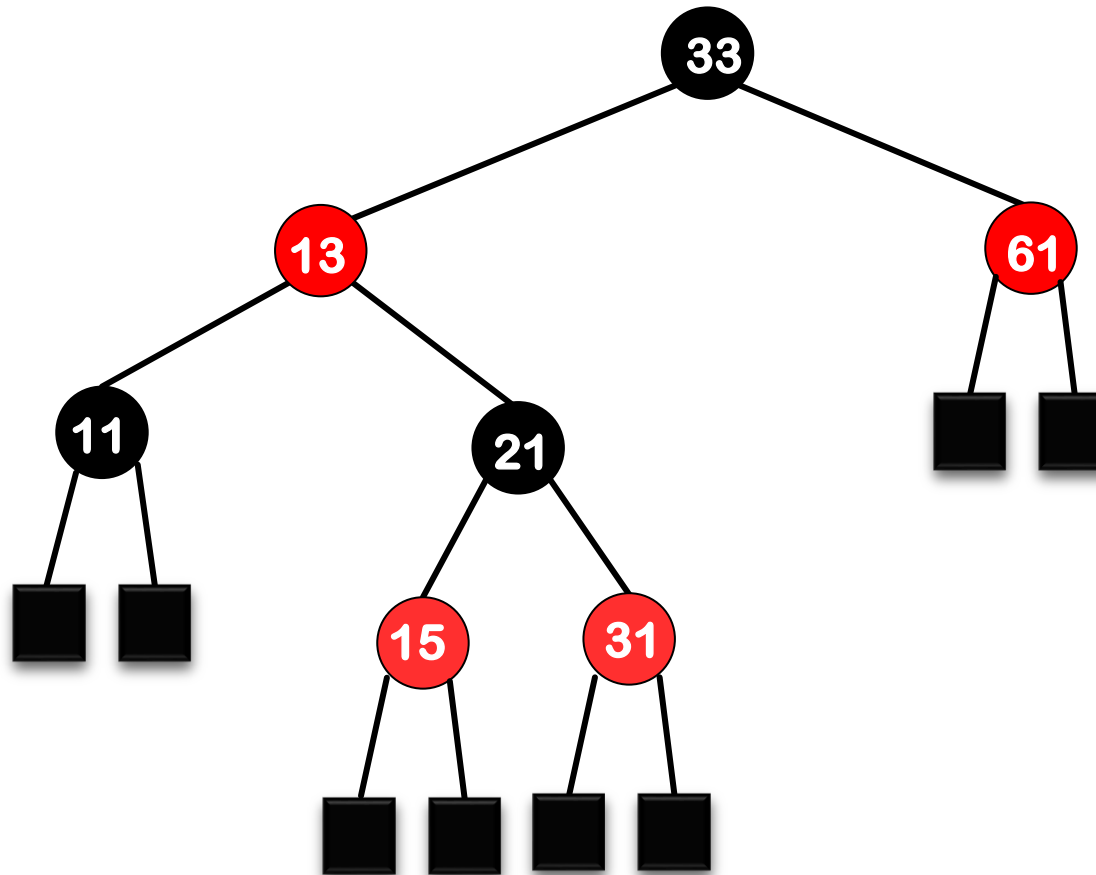
If **x** is **black** and has a single **red** child, then we turn the single child to black and we are done.



DELETION: EXAMPLE 2

Delete 53

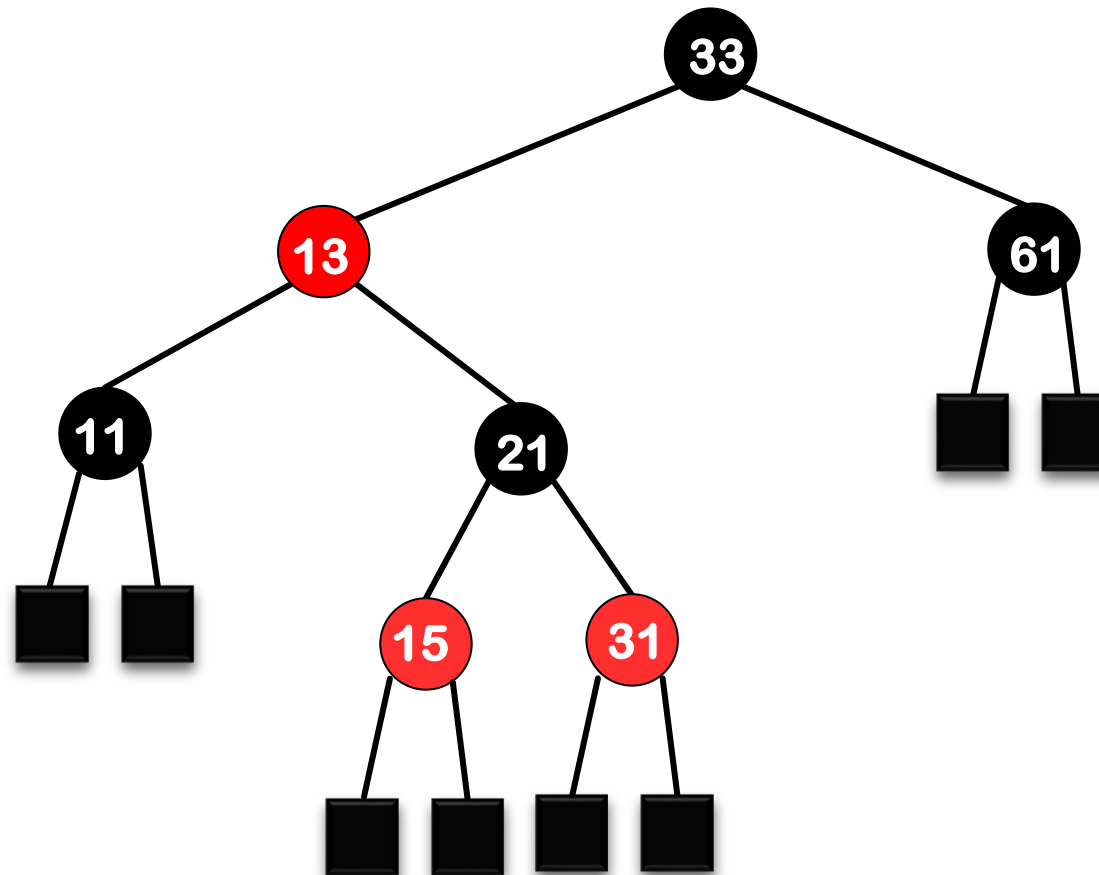
If **x** is **black** and has a single **red** child, then we turn the single child to black and we are done.



DELETION: EXAMPLE 2

Delete 53

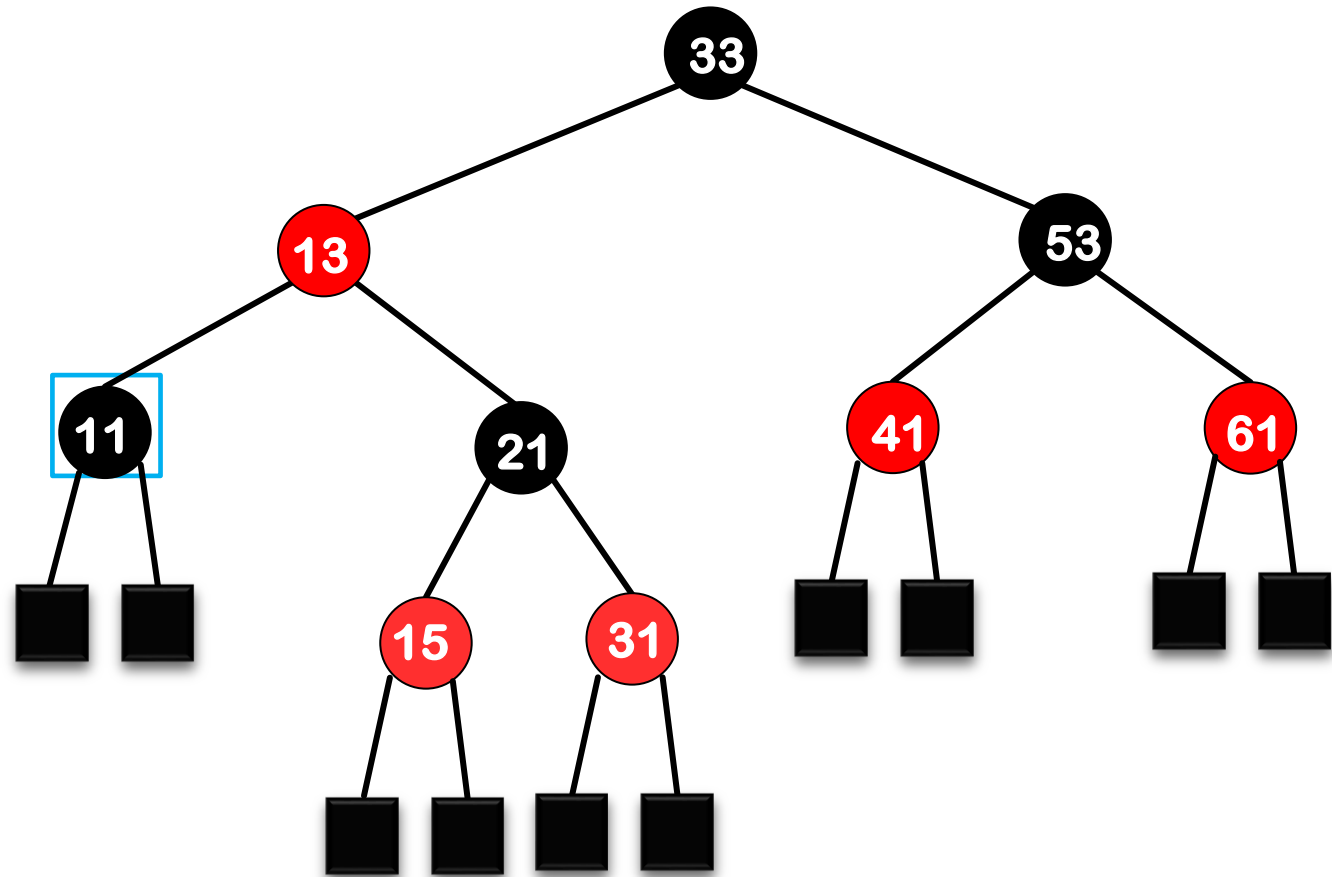
If **x** is **black** and has a single **red** child, then we turn the single child to black and we are done.



DELETION: EXAMPLE 3

Delete 11

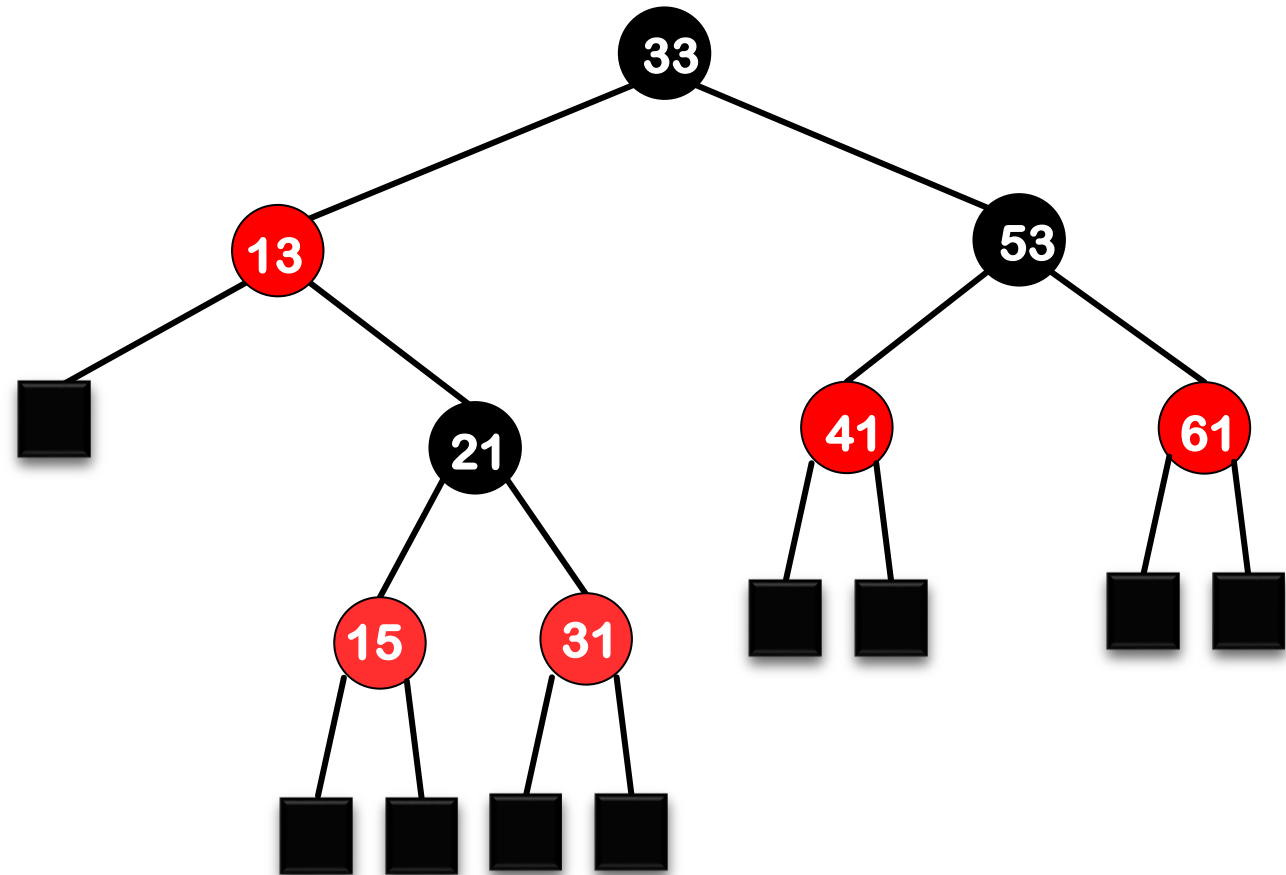
If **x** is **black** and does not have a red child: shortage in the number of black nodes, which needs to be fixed.



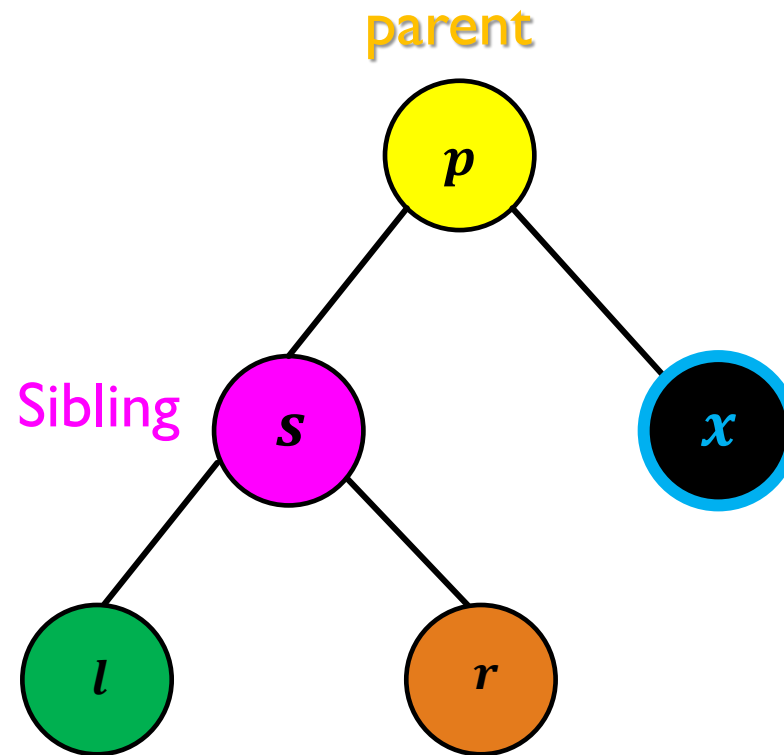
DELETION: EXAMPLE 3

Delete 11

If **x** is **black** and does not have a red child: shortage in the number of black nodes, which needs to be fixed.

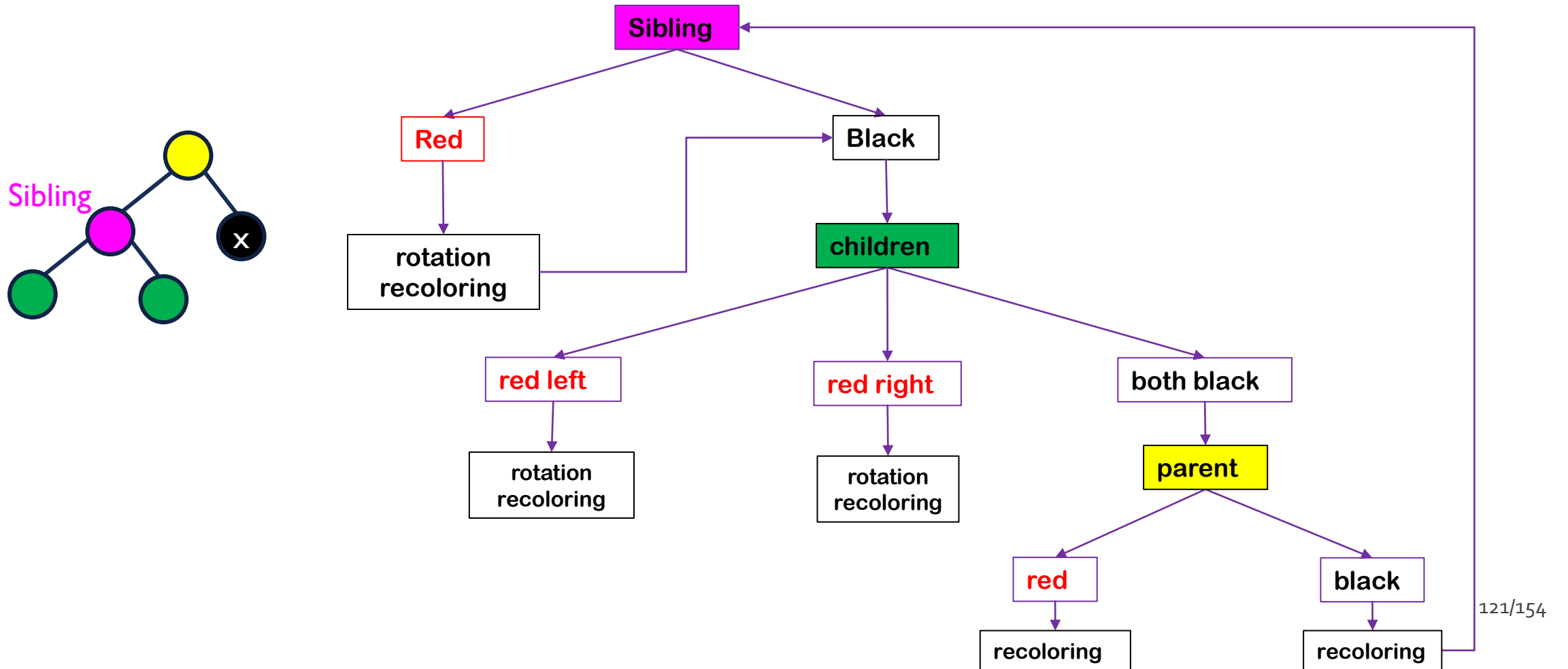


REMOVAL: x IS RIGHT CHILD

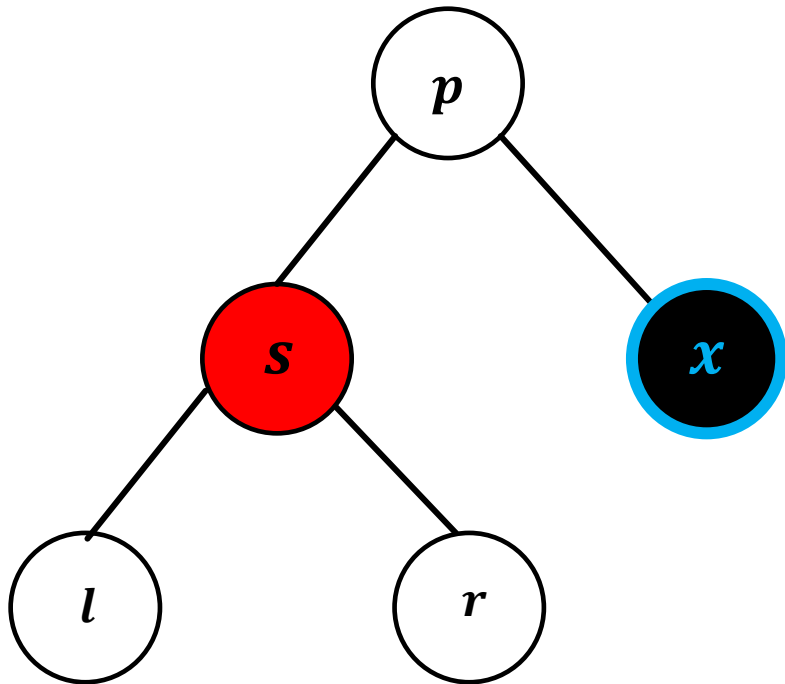


Suppose x is the right-child of its parent. (The case that it is the left child is mirror image.)

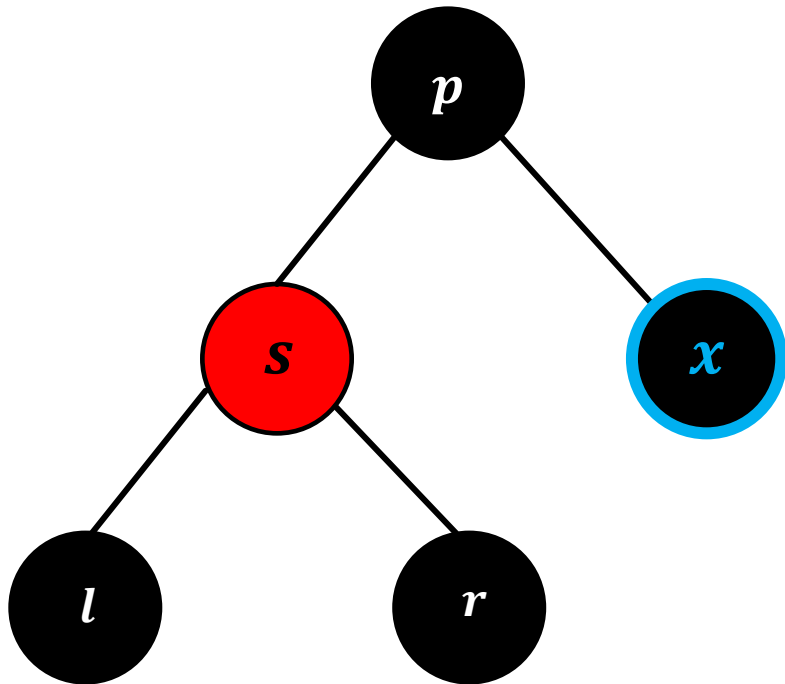
REMOVAL: CASES



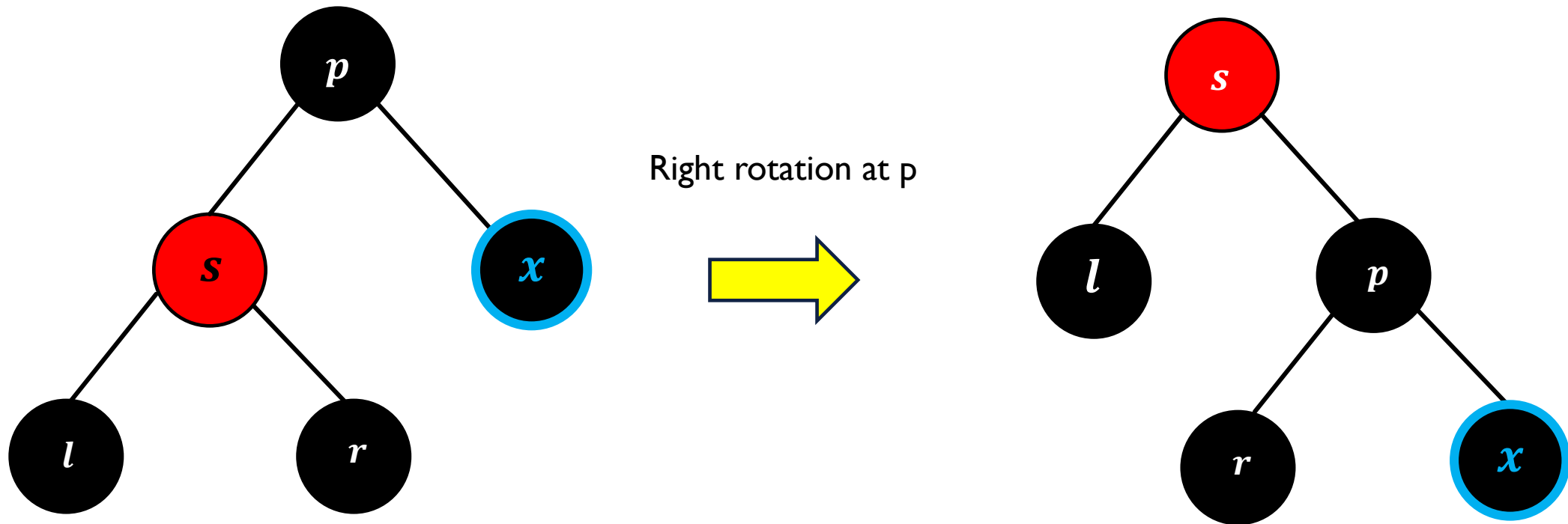
REMOVAL I: SIBLING IS RED



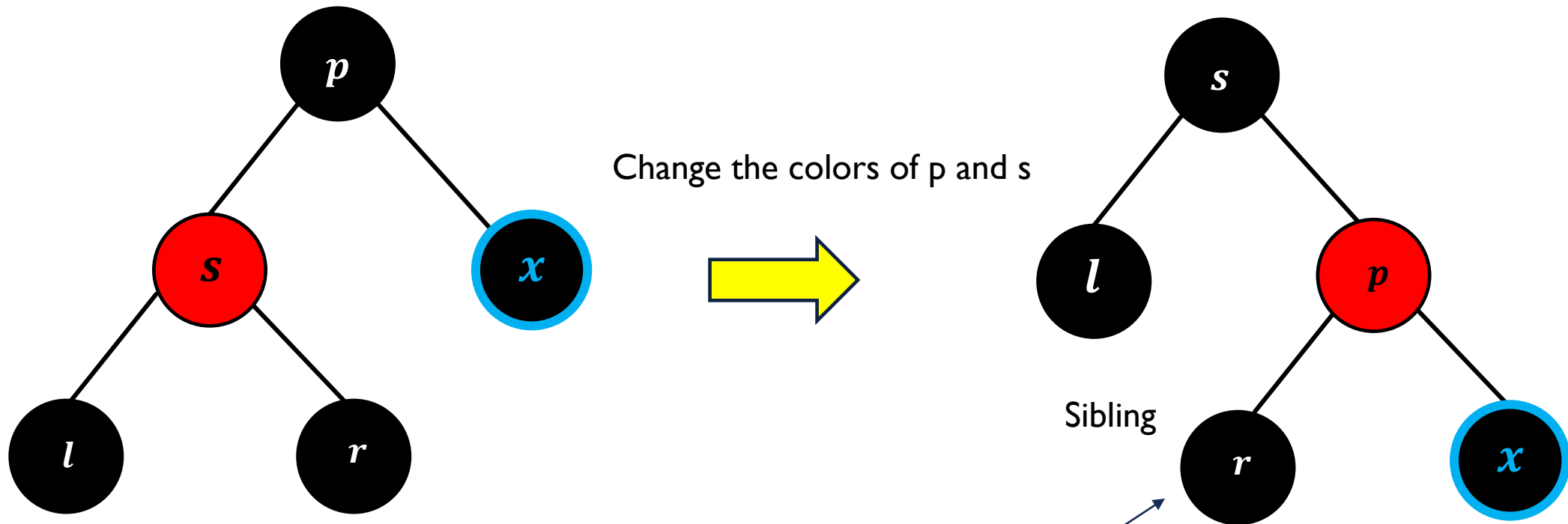
REMOVAL I: SIBLING IS RED



REMOVAL I: SIBLING IS RED

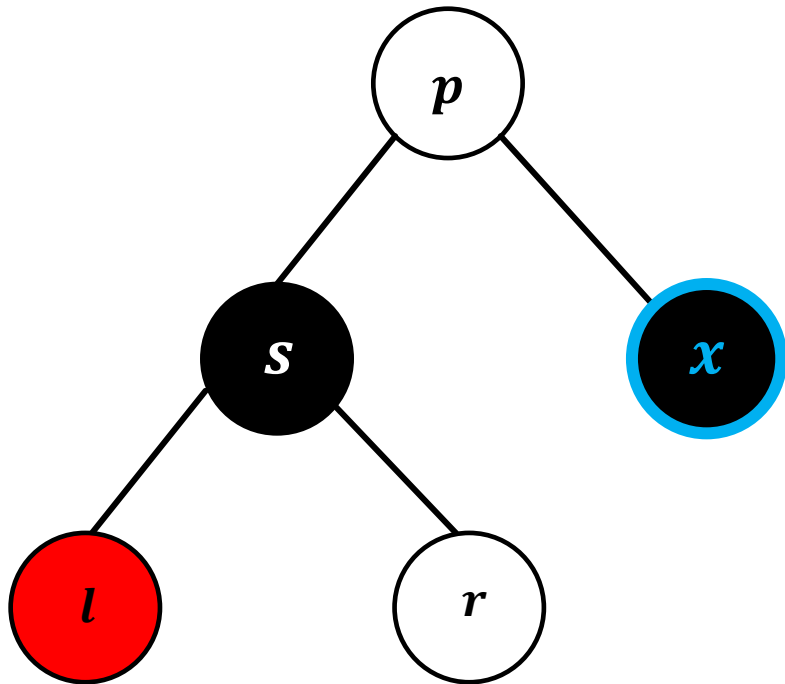


REMOVAL I: SIBLING IS RED

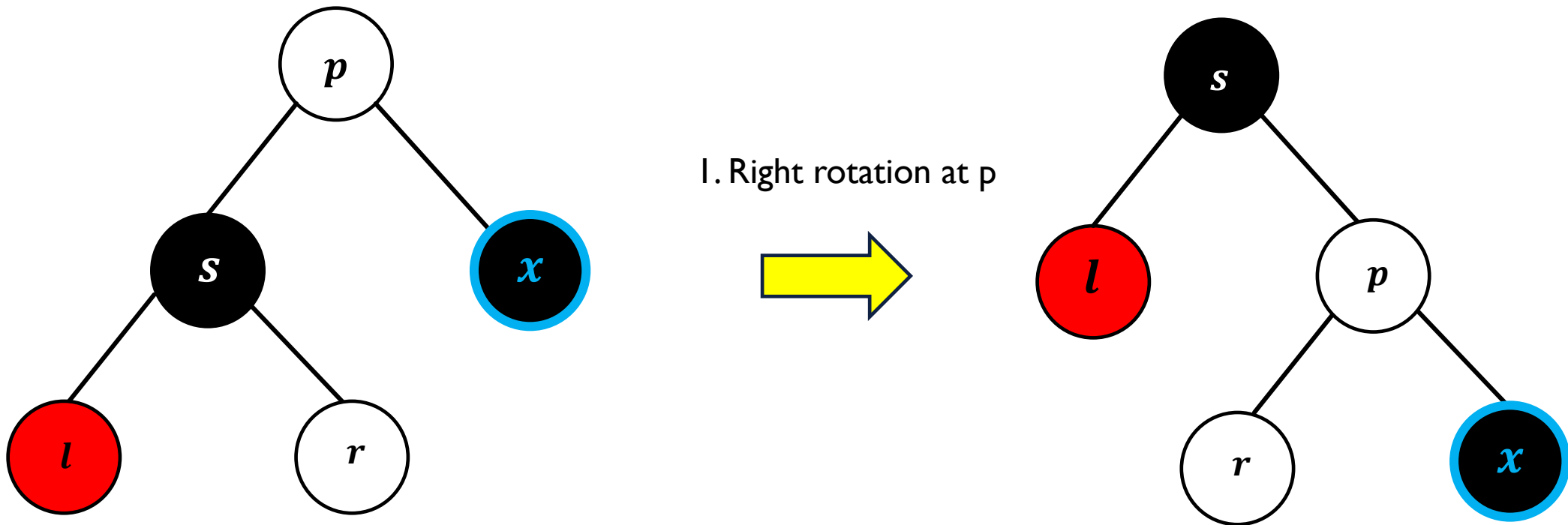


Sibling is black, one of the other cases will apply to terminate

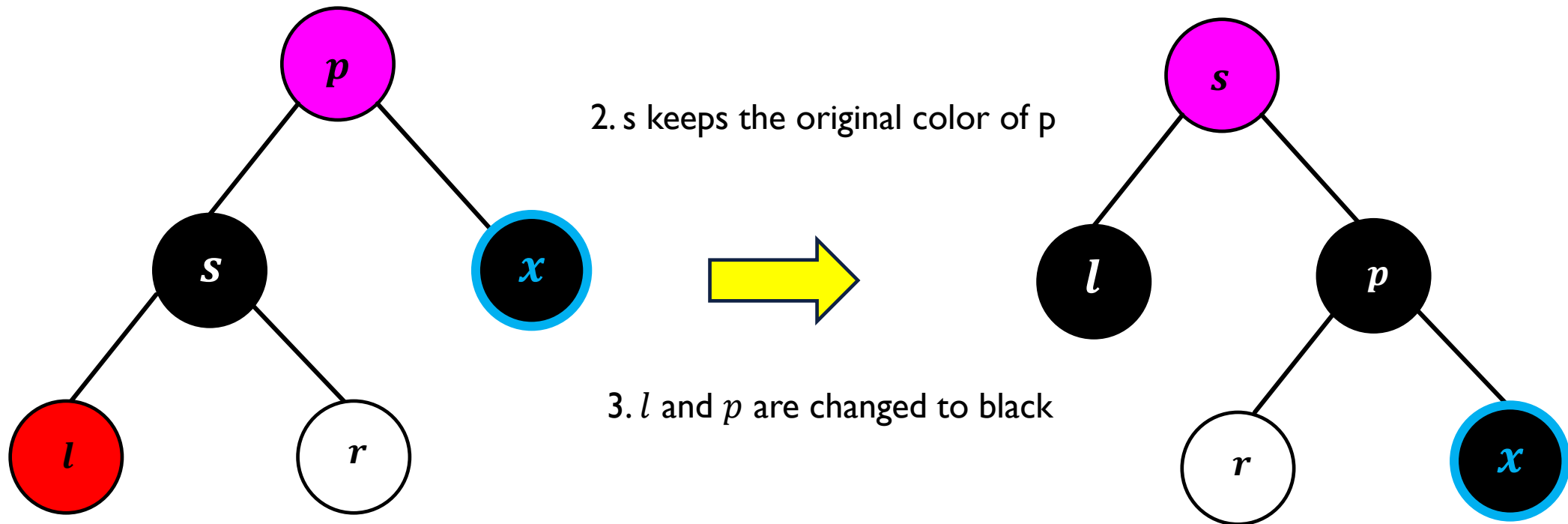
REMOVAL 2: SIBLING IS BLACK WITH A RED LEFT CHILD



REMOVAL 2: SIBLING IS BLACK WITH A RED LEFT CHILD

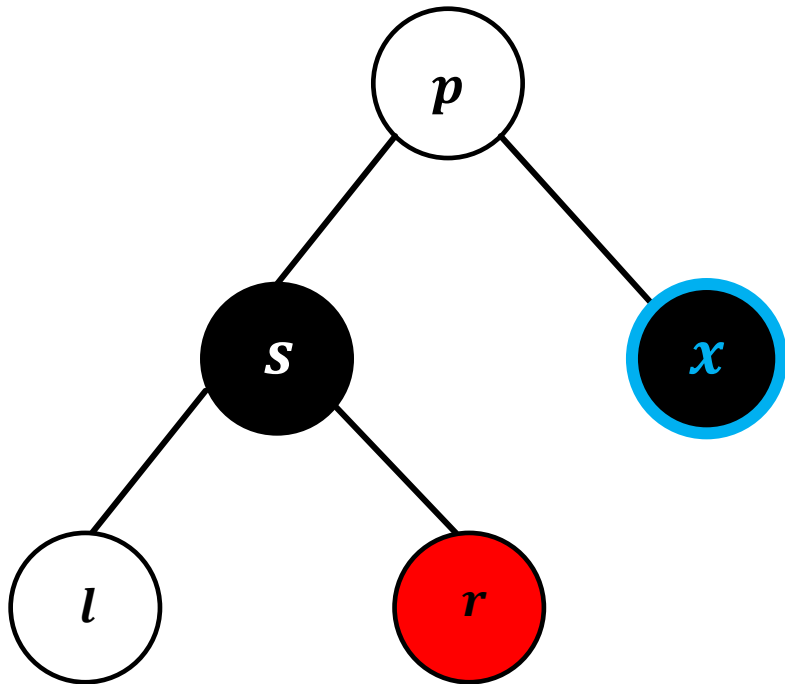


REMOVAL 2: SIBLING IS BLACK WITH A RED LEFT CHILD

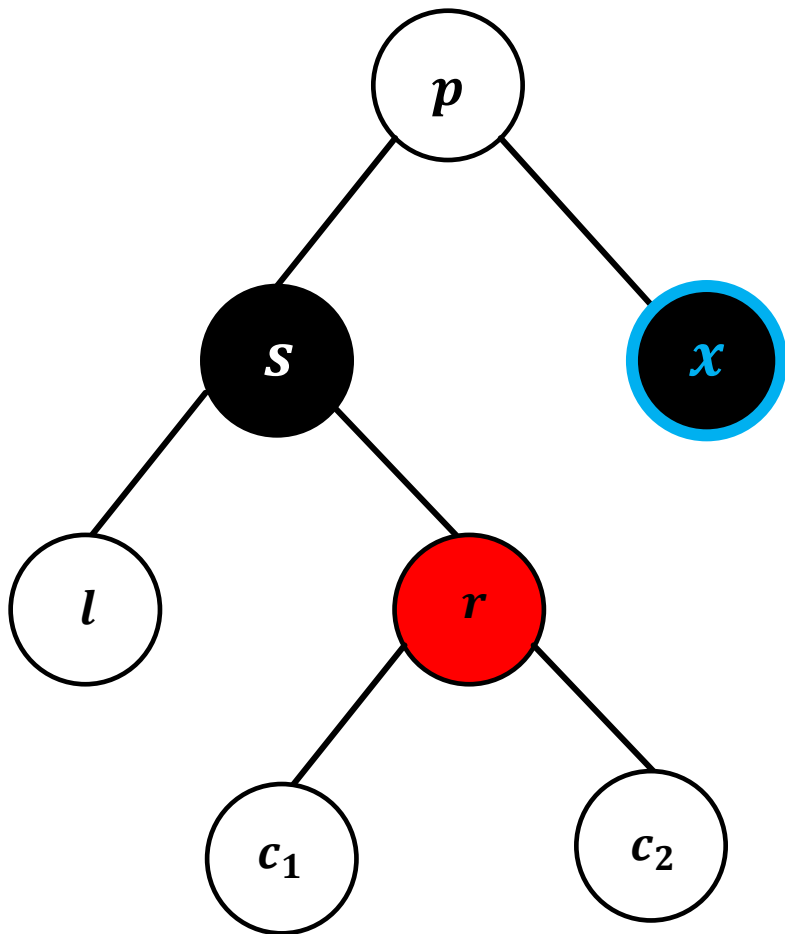


Final result

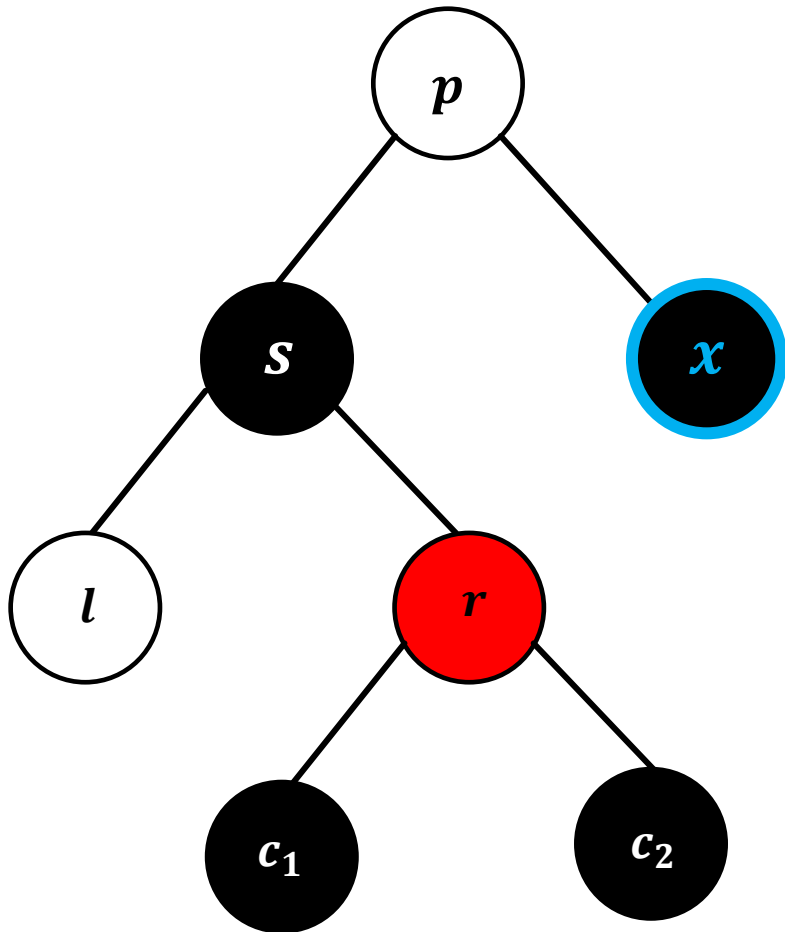
REMOVAL 3: SIBLING IS BLACK WITH A RED RIGHT CHILD



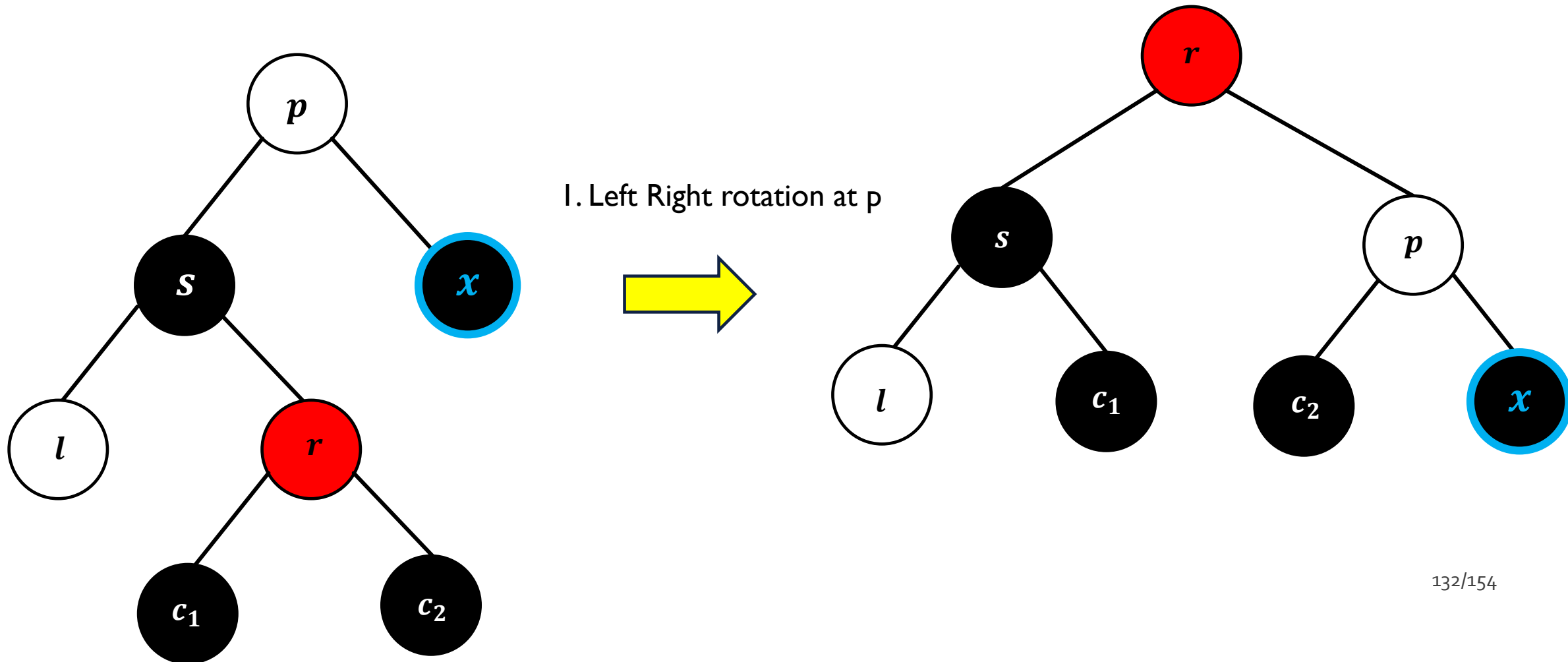
REMOVAL 3: SIBLING IS BLACK WITH A RED RIGHT CHILD



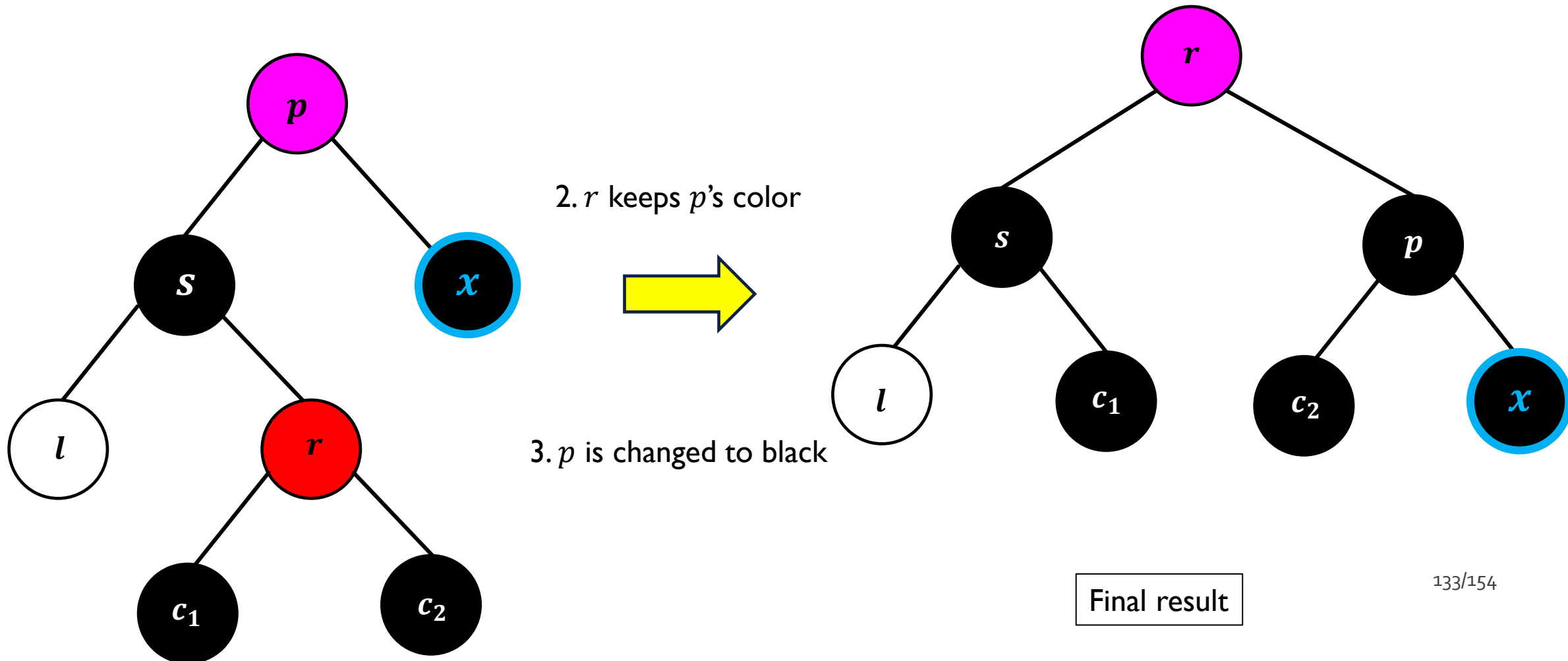
REMOVAL 3: SIBLING IS BLACK WITH A RED RIGHT CHILD



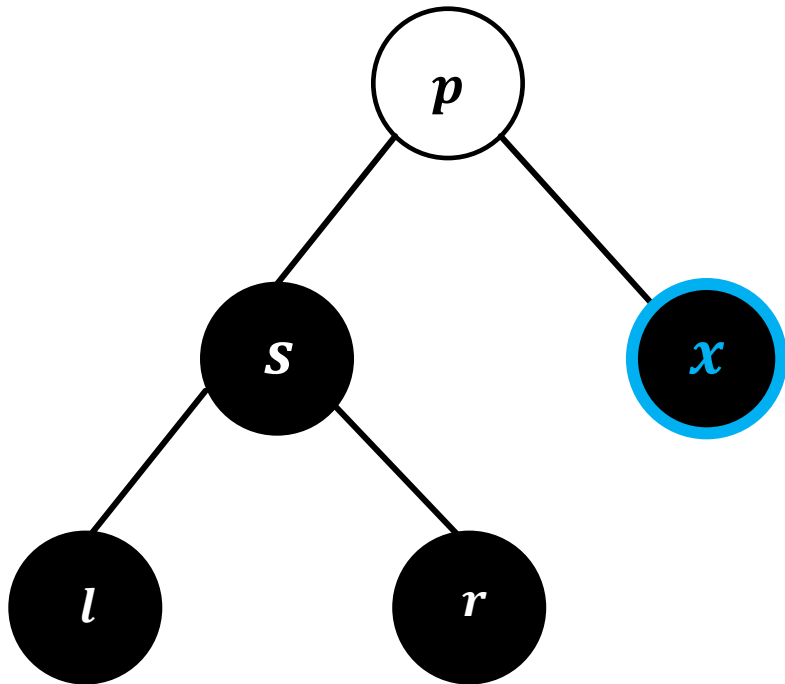
REMOVAL 3: SIBLING IS BLACK WITH A RED RIGHT CHILD



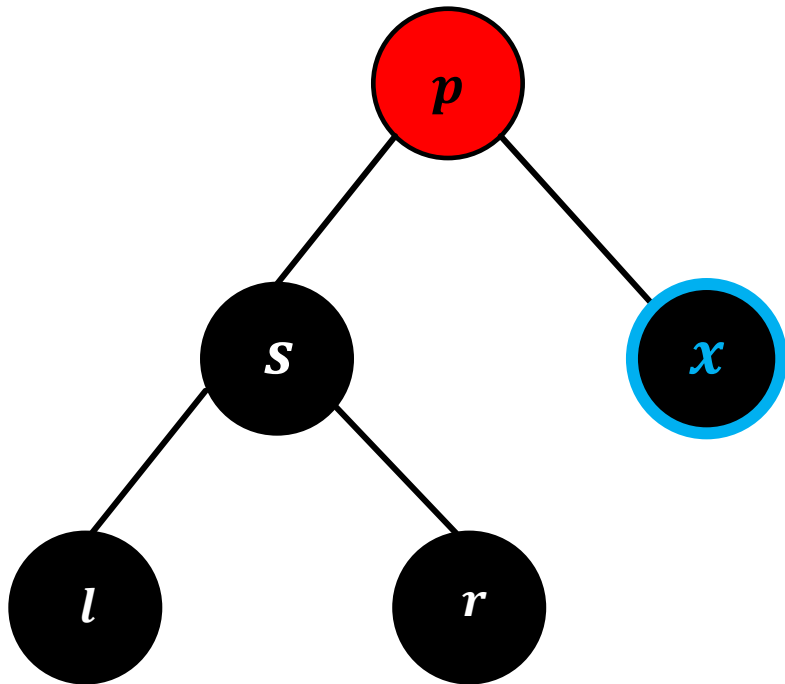
REMOVAL 3: SIBLING IS BLACK WITH A RED RIGHT CHILD



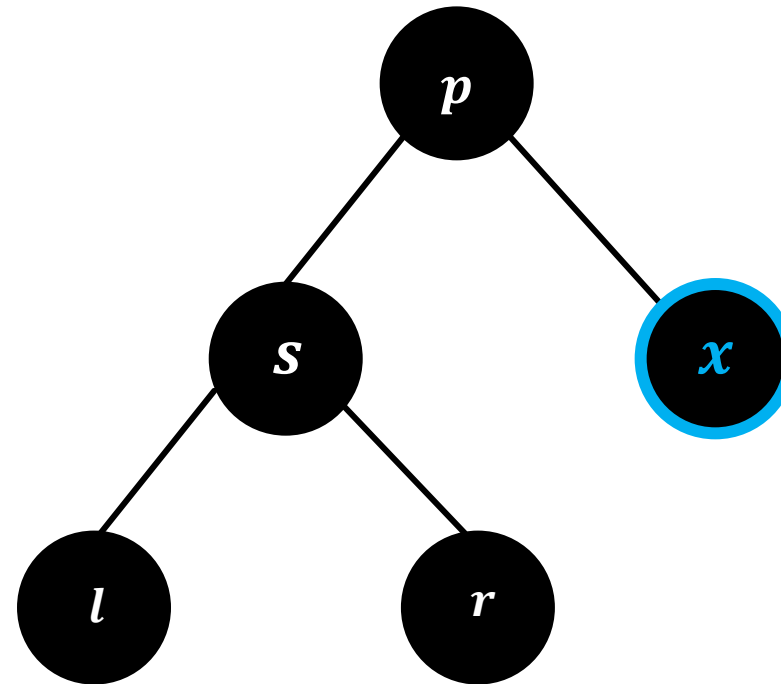
REMOVAL 4: SIBLING IS BLACK WITH TWO BLACK CHILDREN



REMOVAL 4: SIBLING IS BLACK WITH TWO BLACK CHILDREN

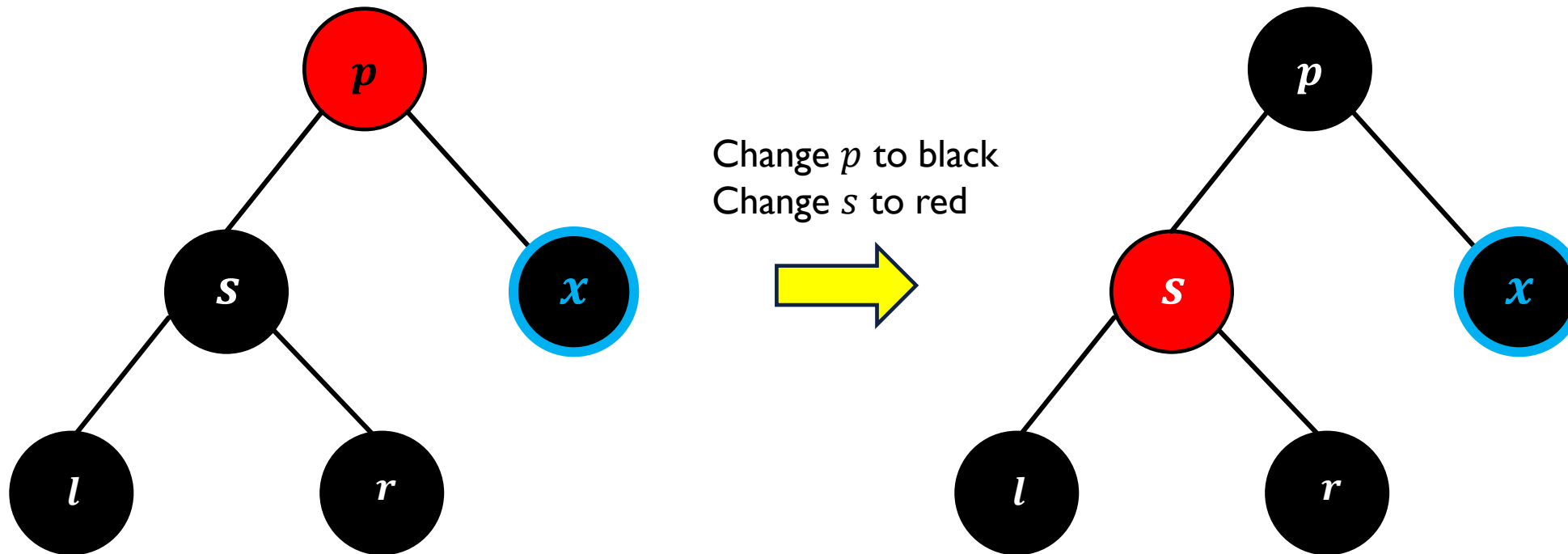


Case A

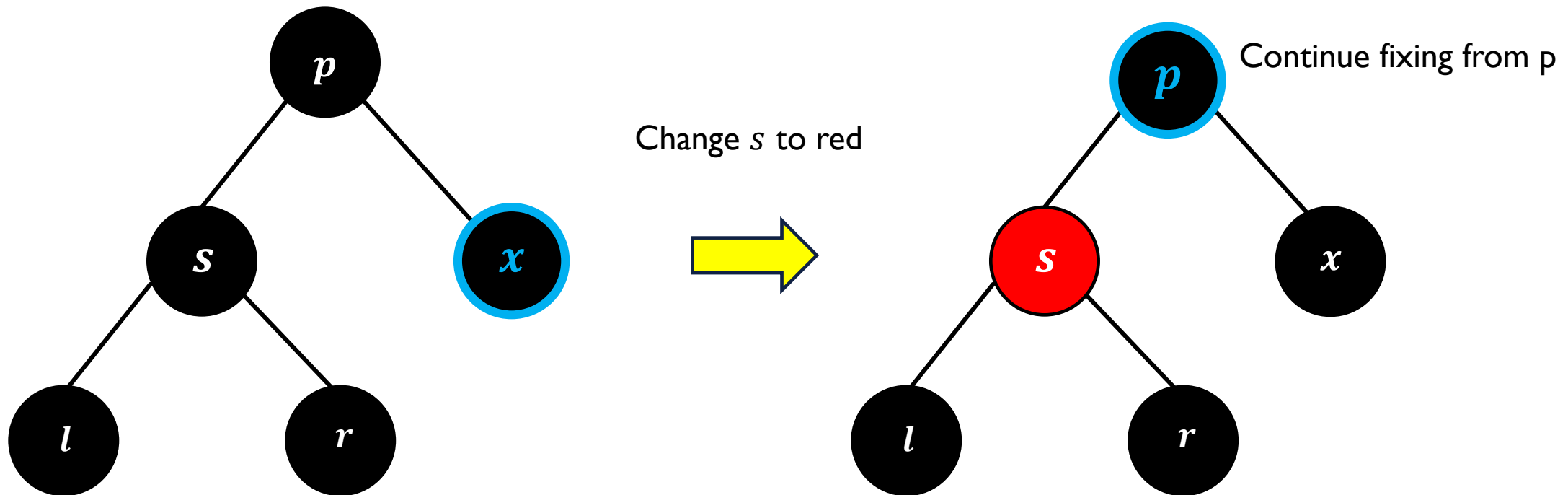


Case B

REMOVAL 4: SIBLING IS BLACK WITH TWO BLACK CHILDREN-A

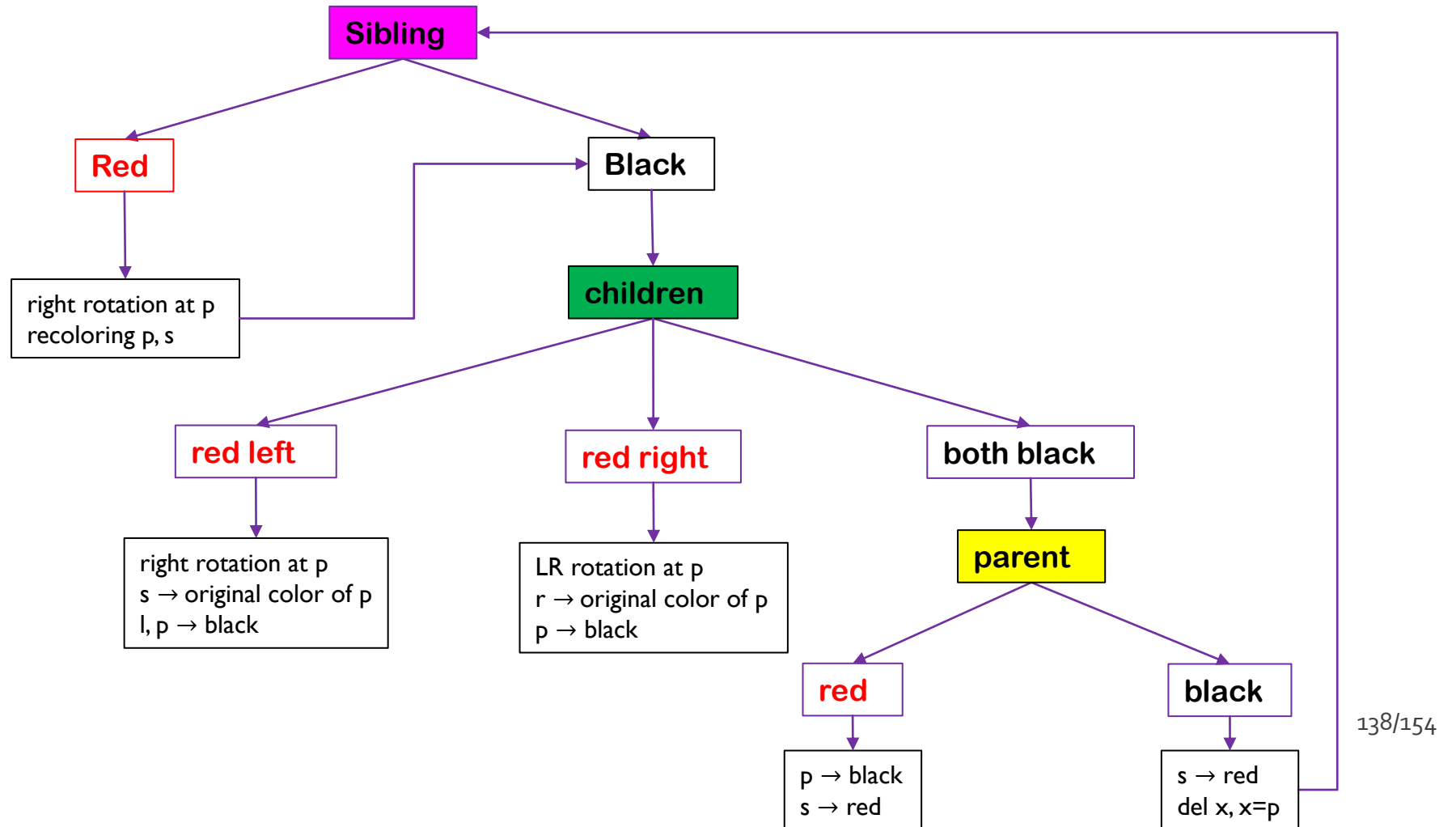
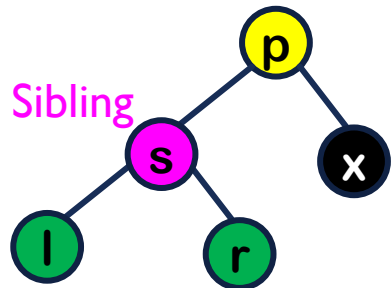


REMOVAL 4: SIBLING IS BLACK WITH TWO BLACK CHILDREN-B

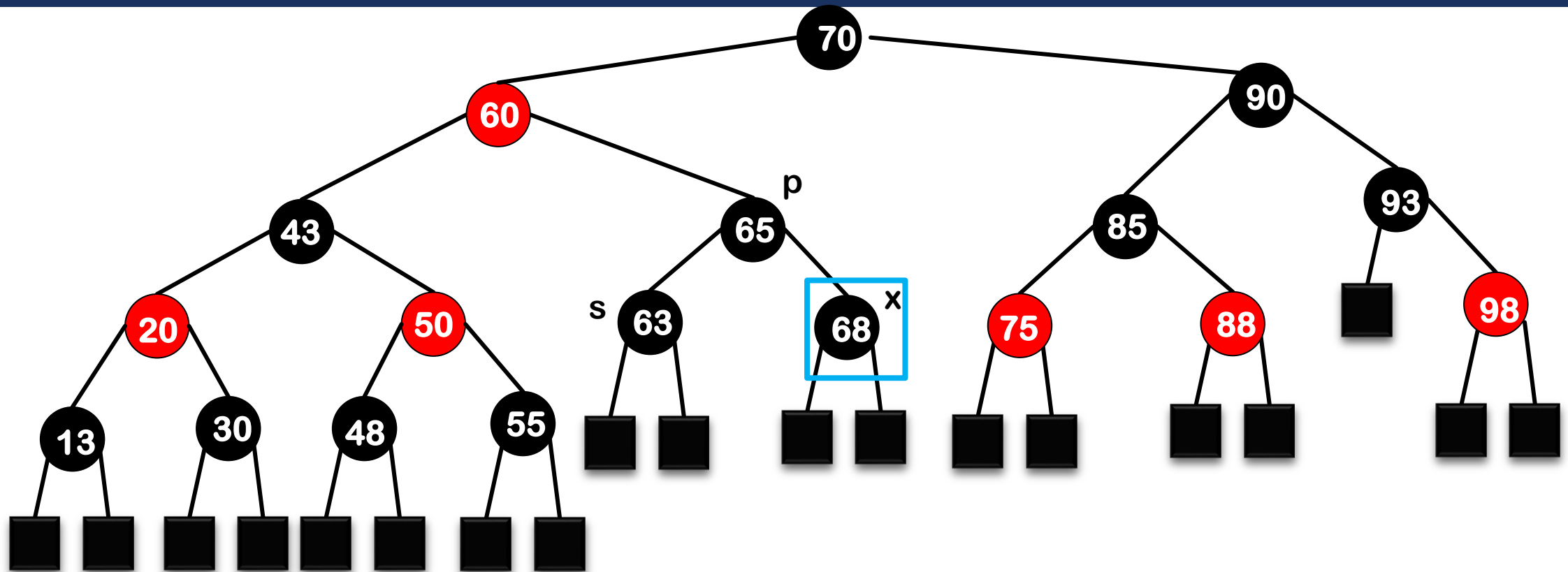


REMOVAL: CASES

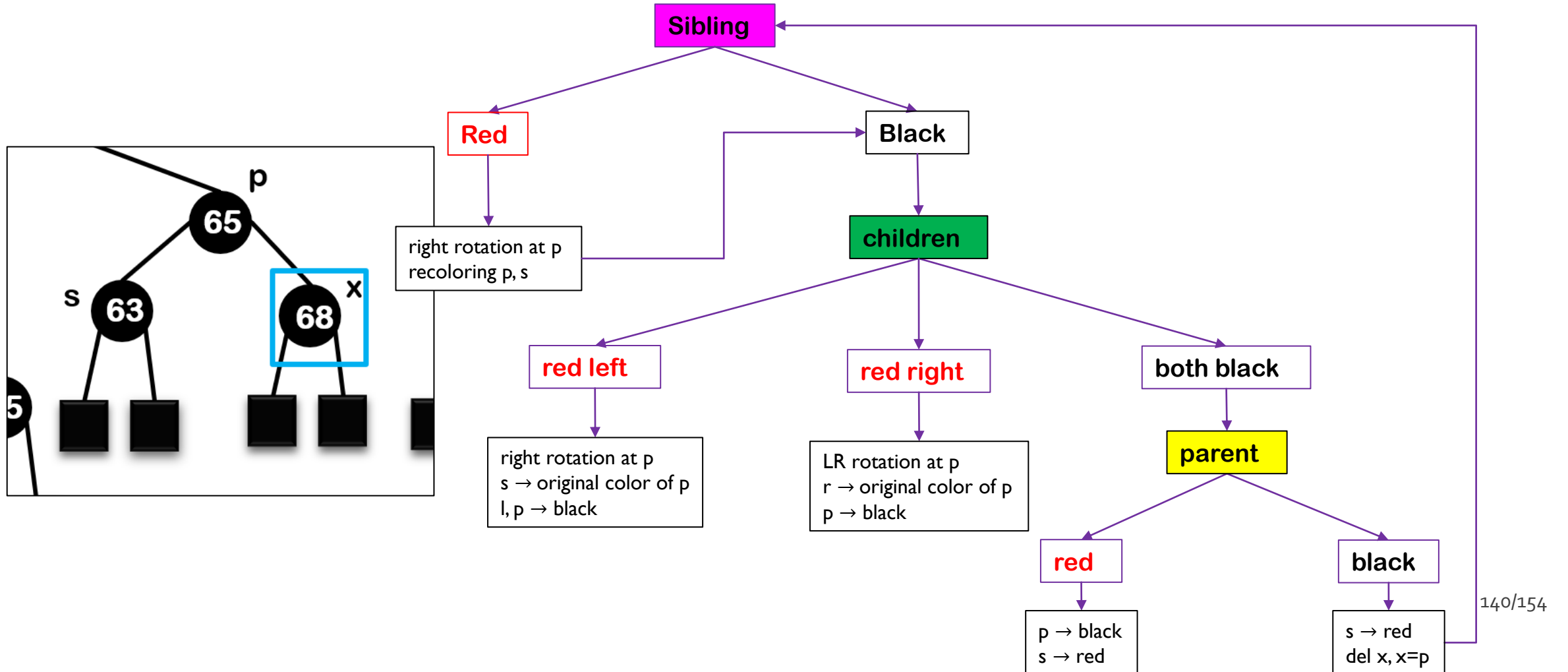
- If **x** is **red**, we are done!
- If **x** is **black** with a **red** child, recolor the child and we are done.



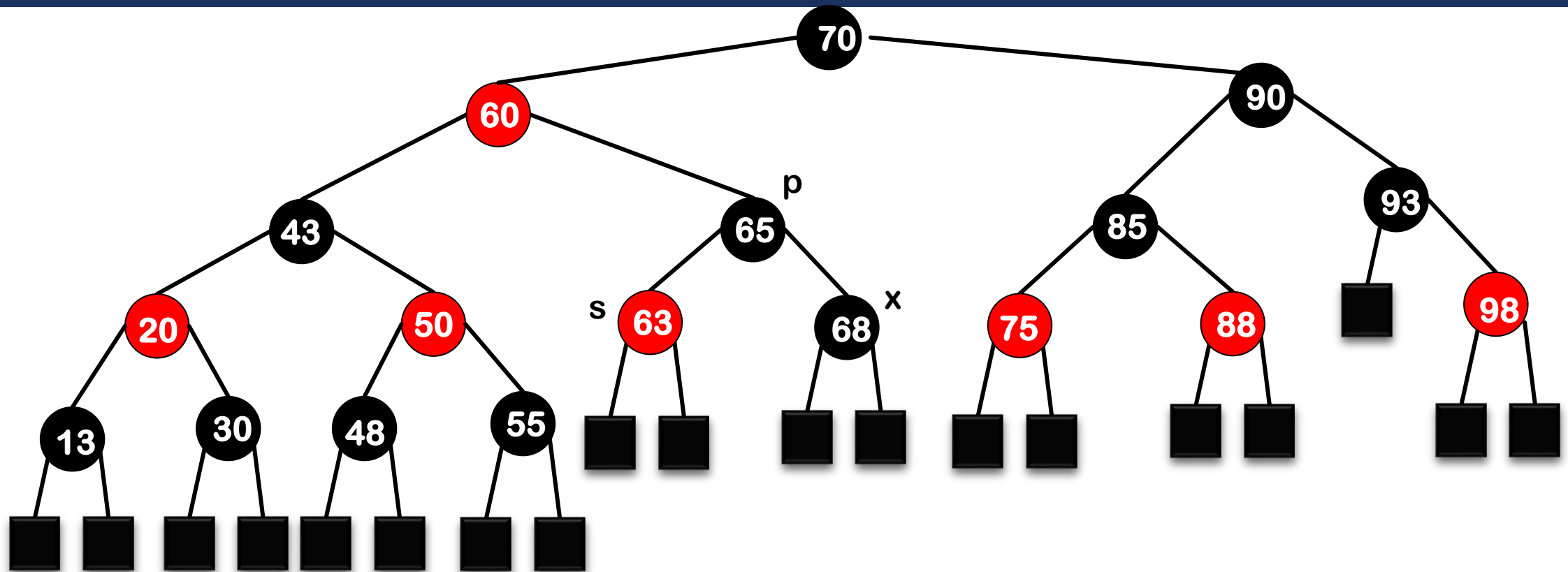
EXAMPLE



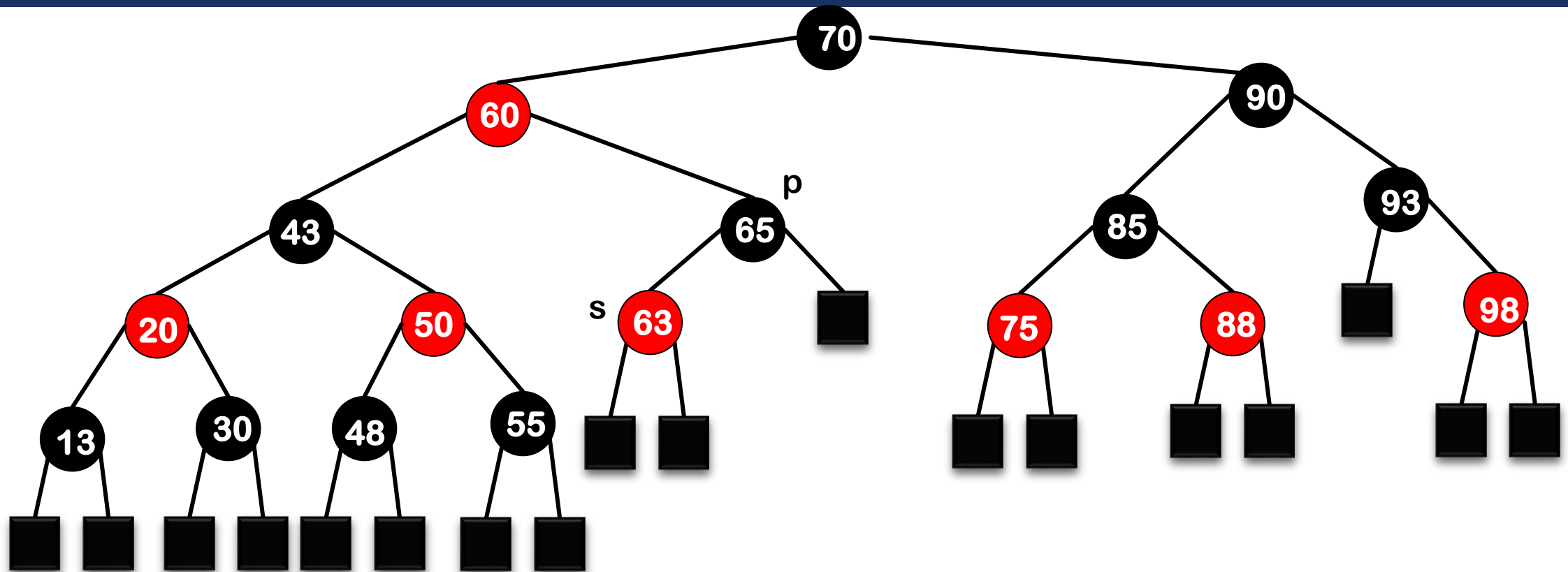
REMOVAL: CASES



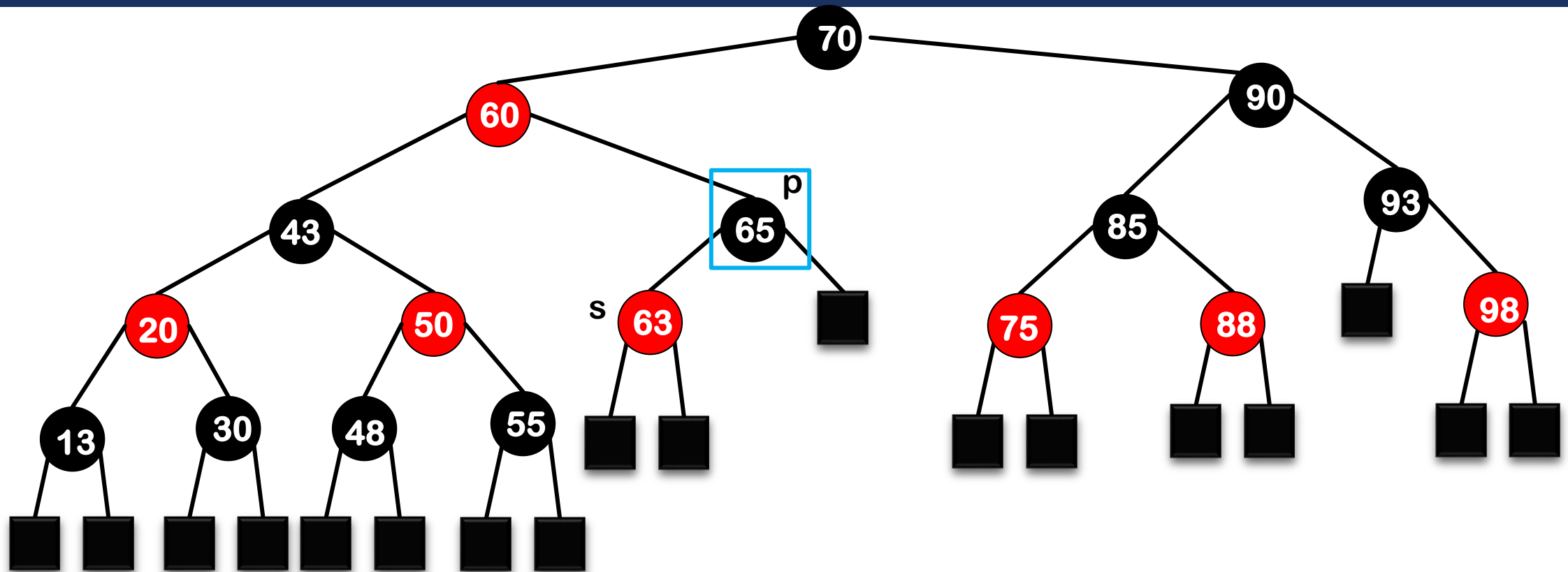
EXAMPLE



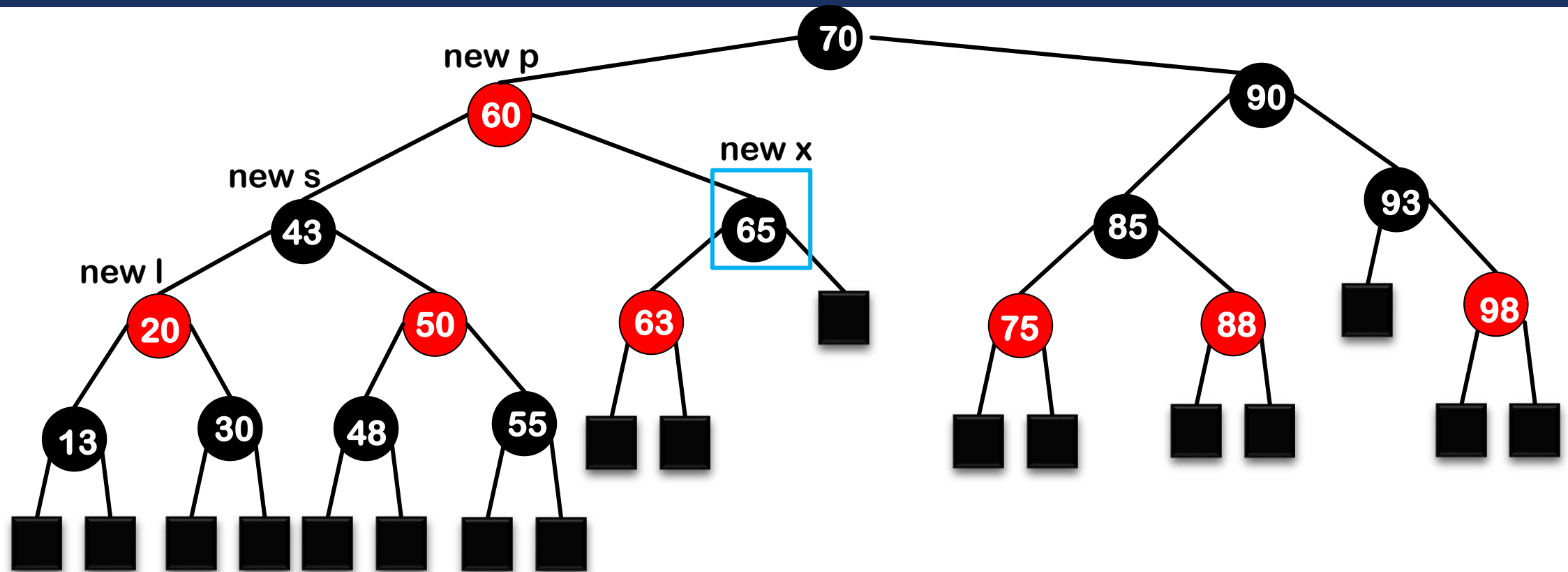
EXAMPLE



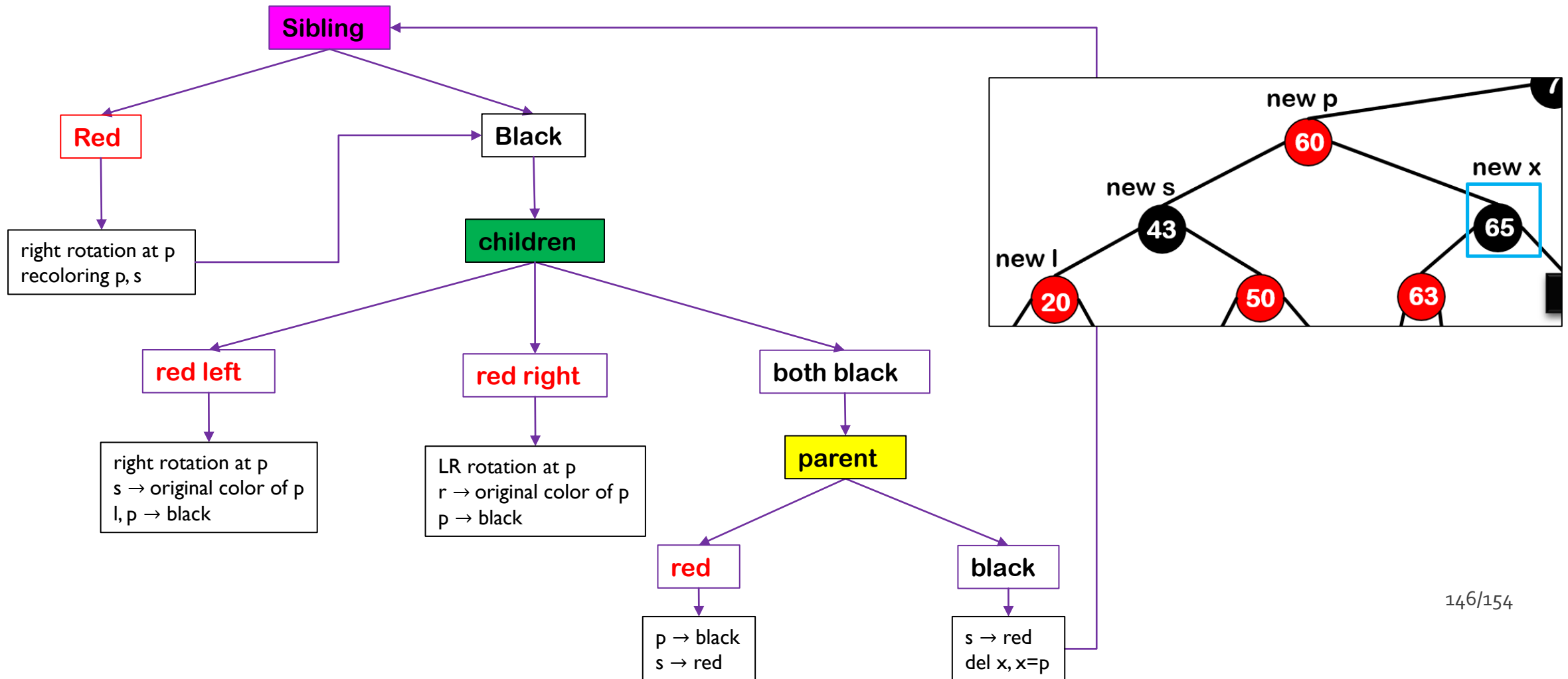
EXAMPLE



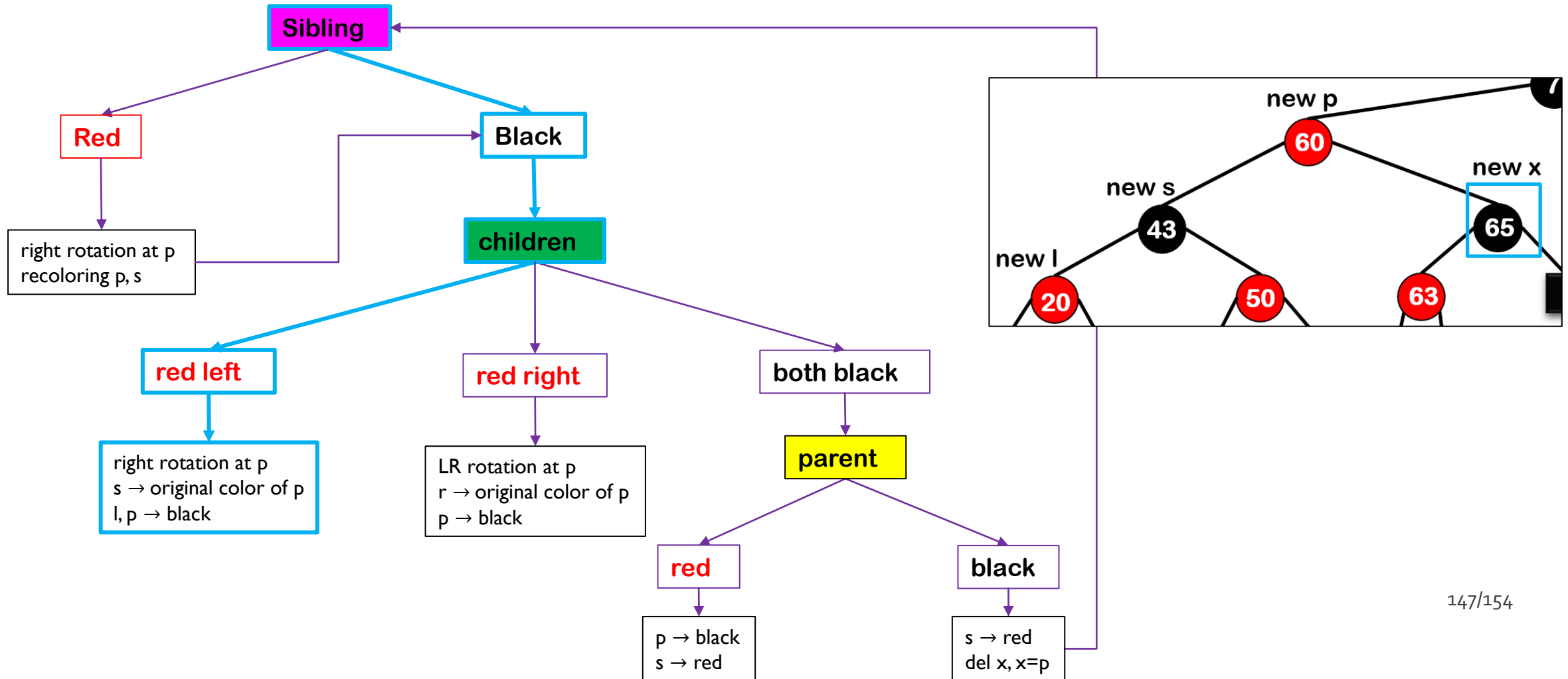
EXAMPLE



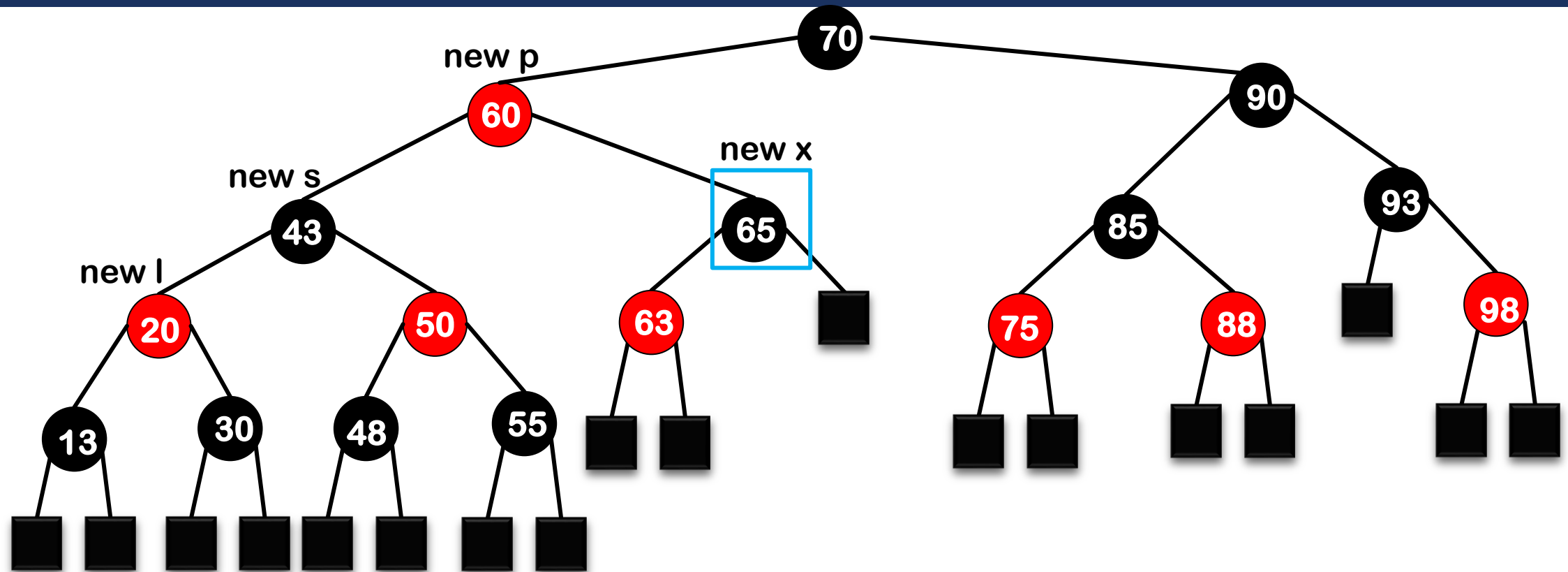
REMOVAL: CASES



REMOVAL: CASES

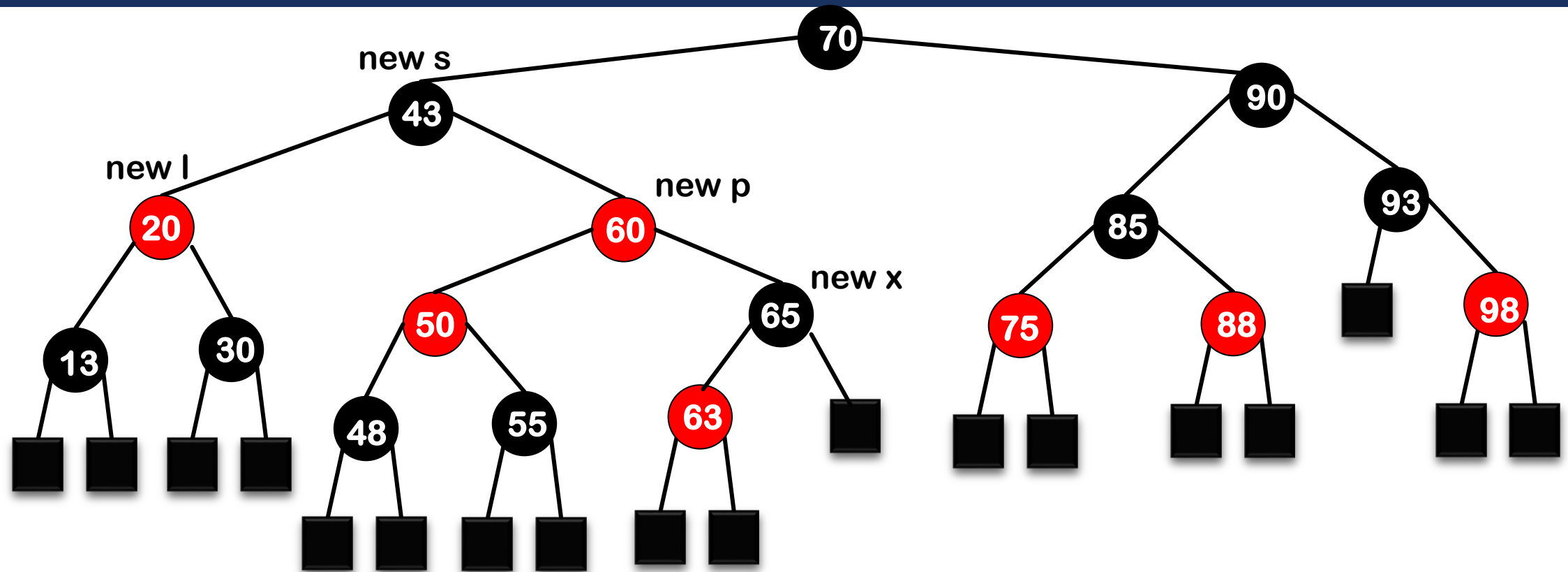


EXAMPLE



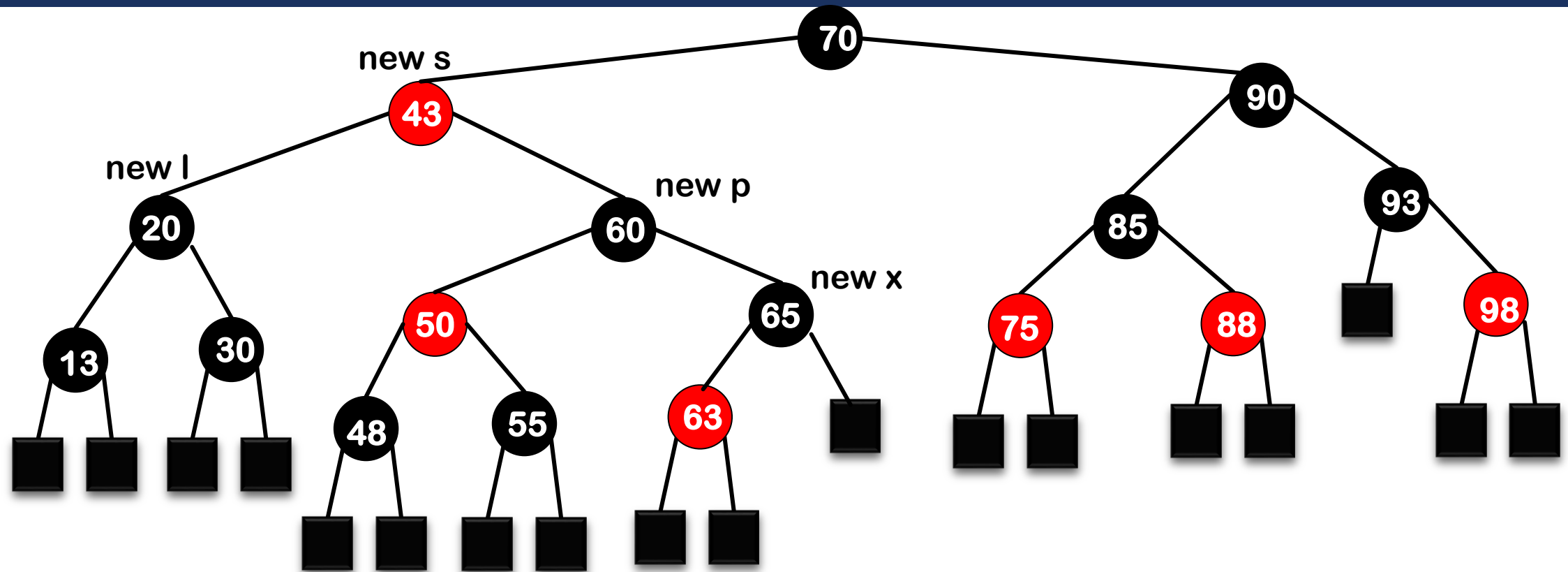
Right rotation at p

EXAMPLE



s → original color of p
l, p → black

EXAMPLE



APPLICATION: FIREWALL RULE MATCHING WITH RED-BLACK TREES

- A **firewall** has a list of rules like: Allow or block certain IP addresses or IP ranges.

Example: Block all IPs from 192.168.0.0 to 192.168.0.255.

- **Challenge:**
 1. These rules are often stored as ranges, and new rules can be added, removed, or updated.
 2. When a packet comes in, the firewall must quickly find if its IP matches any rule.

APPLICATION: FIREWALL RULE MATCHING WITH RED-BLACK TREES

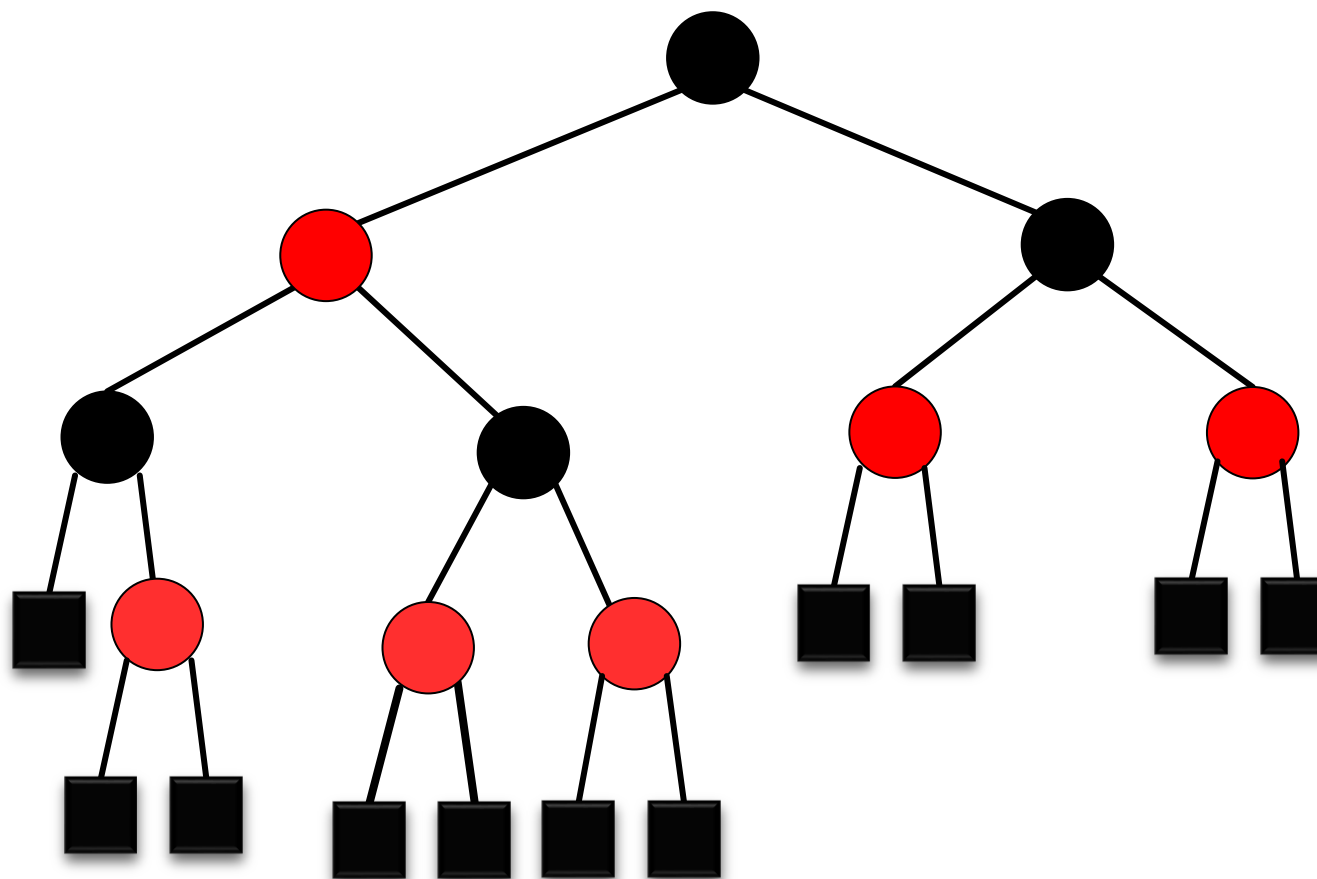
IP Address	Integer Form
192.168.0.1	3232235521
192.168.0.3	3232235523
192.168.0.5	3232235525
192.168.0.6	3232235526
192.168.0.7	3232235527
192.168.0.8	3232235528
192.168.0.9	3232235529
192.168.0.12	3232235532
192.168.0.13	3232235533
192.168.0.14	3232235534

If the IP address is A.B.C.D, its 32-bit integer value is:

$$\text{Integer} = A \times 256^3 + B \times 256^2 + C \times 256 + D$$

APPLICATION: FIREWALL RULE MATCHING WITH RED-BLACK TREES

IP Address	Integer Form
192.168.0.1	3232235521
192.168.0.3	3232235523
192.168.0.5	3232235525
192.168.0.6	3232235526
192.168.0.7	3232235527
192.168.0.8	3232235528
192.168.0.9	3232235529
192.168.0.12	3232235532
192.168.0.13	3232235533
192.168.0.14	3232235534



Thank you for Your attention!

ANY QUESTION??

HAVE A NICE DAY!